BELGIUM CAMPUS ITVERSITY

SEN381

BUGBUSTERS

CIVICCONNECT PROJECT TEAM

MILESTONE 3

Engineering Foundation & Requirements Baseline

PED v3.0 — Engineering Baseline

TEAM MEMBERS

CHRISTIAN JANSE VAN RENSBURG (601840)
MARCO REINERS (578056)
WALDO BLOM (578068)

PROJECT DOCUMENT DETAILS

COURSE	Software Engineering 381 (SEN381)
PROJECT	CivicConnect
SUBMISSION DATE	[16 October 2026]

Table of Contents

1. Problem, Stakeholder Analysis & Business Value 6
   1.1 Problem and business need 6
   1.2 Stakeholder analysis 6
      1.2.1 Product owner 6
      1.2.2 Development team (Waldo, Christian and Marco – “Bugbusters”) 6
      1.2.3 Requester 7
      1.2.4 Management 7
      1.2.5 Staff 7
      1.2.6 Admin (added during M2) 8
      1.2.7 Operator (added during M3) 8
2. Scope Baseline & Constraints 8
   2.1 Scope Baseline 8
      2.1.1 In Scope 8
      2.1.2 Out of Scope 10
      2.1.3 Deferred Scope 11
   2.2 Constraints 11
3. Requirements & Acceptance Criteria 13
   3.1 Functional Requirements 13
   3.2 Non-Functional Requirements 16
4. Architecture-Significant Requirements and Quality Drivers 17
   4.1 Identification of Architecture-Significant Requirements 17
   4.2 Selected CivicConnect ASRs and Quality Drivers 17
   4.3 Influence of ASRs on M2 Engineering Decisions 18
   4.4 Quality Requirements Not Treated as Primary ASRs 19
5. Architecture Baseline 20
   5.1 Architecture Context and Evaluation Criteria 20
   5.2 Architecture Alternatives Considered 21
   5.3 Selected Architecture and Rationale 23
   5.4 Logical Architecture and Responsibilities 23
   5.5 Architecture Diagrams 24
   5.6 Architecture Decision Record and Consequences 25
6. Data & Persistence Baseline 26
   6.1 Core Data Entities and Relationships 26
   6.2 Data Model / ERD / Schema 35
   6.3 Data Ownership and Lifecycle 36
   6.4 Integrity, Consistency and Concurrency 37

6.5 Scalability, Availability, SPOF & Backup/Recovery 38
7. Technology Stack Baseline 41
7.1 Technology Requirements and Selection Criteria 41
7.2 Alternatives considered 41
7.3 Selected Technology Stack 41
7.4 Versions and Compatibility Assumptions 42
7.5 Technology Trade-offs 42
7.6 Evidence and traceability 42
8. Initial Design Decisions 42
8.1 Design decision 1: Decoupling domain logic from external side-effects 42
8.2 Design Decision 2: Centralized state mutation Governance 43
9. Initial API / Integration Decisions 44
9.1 Interface/Integration background and context 44
9.2 Responsibilities and Information Exchange 44
9.3 Interaction Mechanism 45
9.4 Validation, Errors and Security Boundary 46
10. Deployment Compatibility 46
10.1 Deployment Direction and Environment Assumptions 46
10.2 Compatibility of Architecture, Data and Technology Decisions 48
10.3 Deployment Configuration, Secrets, Persistence/State and Networking Implications 49
10.4 Deployment Compatibility, Cost, Availability and Operational Risks 51
10.5 Deferred Deployment Decisions and Evidence Required 53
11. RTM & Traceability Foundation 55
11.1 Evolved RTM 55
11.2 Requester RTM 55
11.3 Staff RTM 55
11.4 Management RTM 56
11.5 Operator RTM (Added M3) 57
11.6 Engineering Decision Log 57
12. Risk & Forward Engineering Considerations 70
12.1 Initial Risk Register 70
12.2 Forward Engineering Considerations 76
13.Development and Application Evidence (M3 Release Candidate) 78
13.1 Identified Release candidate for M3 78
13.2 M3 Overview and Current Status 78
13.3 Repository / Module Structure 79
13.4 M2 Baseline Conformance and Controlled Changes (Assessment Area A) 79

13.4.1 Baseline conformance 79
13.4.2 Significant deviations and controlled changes 80
13.5 Collaborative Construction and Configuration Control (Assessment Area B) 80
13.6 CI and Automated Verification (Assessment Area C) 82
13.6.1 Test inventory 83
13.7 Quality Strategy Applied (Risk-Based) 85
13.8 Black-Box, Integration and System Validation (Assessment Area D) 86
13.8.1 Functional and E2E black-box records 86
13.8.2 Integration, API and state-transition records 87
13.9 Defects, Findings and Technical Debt (Assessment Area E) 91
13.10 Performance and Reliability (Assessment Area F) 99
13.10.1 Performance Test Purpose and Scope 99
13.10.2 Performance Criteria and Test Design 99
13.10.3 Test Implementation and Application Workflow 99
13.10.4 Hosted Environment, Workload and Data Preparation 100
13.10.5 Test Data, Execution Results and Verification 101
13.10.6 Engineering Interpretation, Resource Evidence and Limitations 102
13.10.7 Evidence, Traceability, Technical Debt and Release Conclusion 103
14. Staging / Test Deployment 103
14.1 Purpose and rationale 103
14.2 Deployment architecture and configuration 104
14.3 Environment variables and secret control 105
14.4 Hosted end-to-end/system verification 105
14.5 Staging differences, constraints and residual risks 106
14.6 Path to real-world deployment 107
15 Backup, Rollback and Recovery Plan 108
15.1 Scope and implementation status 108
15.2 Database and attachment backup and recovery 108
15.2.1 Database protection 108
15.2.2 Database recovery procedure 108
15.2.3 Attachment backups and consistency 109
15.2.4 Schema/version compatibility 109
15.3. Codebase, application and deployment recovery 110
15.3.1 GitHub and Source Code Protection 110
15.3.2 Application rollback 110
15.3.3 Configuration recovery 110
15.4. Whole-System Recovery and Risk Management 111

15.4.1 Coordinated System Recovery 111
15.4.2 Proposed Recovery Objectives 111
15.4.3 Recovery Risks and Mitigations 111
15.4.4 Recovery Responsibilities and Verification 112
15.5. Milestone 3 Scope and Release-Readiness Implications 112
15.6. Conclusion 113
16 Release-Quality Review & Residual Risk - Assessment area G 113
16.1 Release evidence summary: 113
16.2 Final Classification & Rationale 114
17. GitHub Governance 115
17.1 Branches 115
17.2 Pull requests (PR's) and Code review process 118
17.3 Issues 118
17.4 Milestones 118
17.5 Labels 118
17.6 CI / Automated Checks 119
17.6.1 Triggers and concurrency 119
17.6.2 Shared setup 119
17.6.3 Jobs 119
17.6.4 Integration with branch protection 120
17.6.5 Secret handling 120
17.6.6 Scope and deliberate exclusions 120
17.6.7 Evidence and traceability 121
17.6.8 Evidence of the CI gate blocking a failing change: 121
18. Architecture, Technology and Initial Design Baseline 121
18.1 Baseline Identification 121
18.2 Baseline synthesis and operational contract 121
18.3 Open / Deferred Decisions 122
19. Change Management Standard & Controlled Changes 124
19.1 Change register 124
19.2 Detailed change management per summary 125
19.2.1 Operator Stakeholder and External Requester Support (CR-001) 125
19.2.2 Requester Contact Information Access Approval (CR-002) 128
AI Usage 130
Baseline Sign off 130

1. Problem, Stakeholder Analysis & Business Value

1.1 Problem and business need

The problem is that the community focused organization currently manages service requests through different commutation platforms: email, telephone calls, WhatsApp messages, spreadsheets and paper-based protocols. This approach creates operational problems such as requests are frequently duplicated, overlooked, or lost between channels; requesters don't visibility into the status of their requests; staff struggle to coordinate and prioritise tasks; management does not have enough information about the outstanding, overdue and resolved work; there is no way to keep track of the different requests logged.

The organisation requires a digital platform that can provide a reliable, and easy way to submit, manage, monitor, and report on these different requests. This digital platform aims to solve these operational problems identified above. Stakeholders each have different business needs will interact with the application in different ways, this is further explained in the next section.

1.2 Stakeholder analysis

The project consists of 7 different identified stakeholders, each of them having different needs and ways in which they will interact with the digital platform. Some stakeholders might have conflicts or competing expectations. Below is the breakdown for each stakeholder:

1.2.1 Product owner

This is the person who wants the system in the first place. They are the "community-focused organization" that identified the problem. Their responsibility is to ensure that they clearly communicate their needs and expectations for the system that needs to be developed.

They do not submit actual issues. They review the system during the milestones to ensure that it meets the expectations and aligns with the business needs and functional requirements.

Conflict / competing expectation: The Project owner wants the digital platform to have as much functionality as possible within the fixed budget and schedule. This could clash with Development teams need for a stable scope and enough time to build a quality product that is also maintainable. If the scope is not controlled properly, this could lead to scope creep. That is why scope will be controlled and any changes in scope will not just be accepted, they will first be considered against the baseline scope identified and only in the event that scope change is deemed absolutely necessary, it will accepted and be properly documented.

1.2.2 Development team (Waldo, Christian and Marco – "Bugbusters")

This is us (Waldo, Christian and Marco) the "Bugbusters" team who are actually developing the project based on the identified requirements. Our responsibility is to ensure that we clearly understand the business need in order to identify the requirements of the project. But also, to ensure that we build a system that is maintainable, reliable and secure.

We will test the system to ensure that it works correctly. We will not add additional items to the scope before properly considering the implications of doing so, also in the event that the scope changes we will properly document the decision and the change.

Conflict / competing expectation: We must balance the speed at which we develop the application with the quality and maintainability of the end product. With a fixed schedule and free tier constraints we must choose quality and sustainability over just implementing something that works and gets the job done.

1.2.3 Requester

This is the person who experiences the actual problem i.e. a fault, security concern, lost property, or any other operational request.

Their responsibility is to provide accurate and clear information about the issue they are experiencing, and to label the issue correctly according to the request category. They should also respond to staff when asked for more information about their issue.

Their core need is to stay updated on their request. They want as much visibility as possible when it comes to their logged request, i.e. knowing when it was received, assigned, in progress and resolved. They want a system that is easy to use and easy to interact with, they don't want something is complex and difficult to use.

Conflict / competing expectation: Requesters want an experience that is simple and quick however the staff and management need sufficient information to be able to resolve the issue that the requester is experiencing. These needs clash and need to be balanced, though the digital platform that will be developed.

1.2.4 Management

Note: Each department manager (e.g., IT Manager, Facilities Manager) is not a separate stakeholder; Management overall is treated as a single stakeholder group.

Management oversees their specific category (i.e. IT sees IT related requests, Facilities Manger sees the requests associated with facility requests) of requests and monitors activity through a dashboard (see FR-301of the functional requirements section), which thereby provides the visibility of active and closed requests. In order to improve operational efficiency and reduce management workload, staff may self-assign tasks. However, Management still has the final authority to approve, override, or redirect requests as needed.

Their responsibility is to ensure that the issues assigned to their department is resolved within the expected time frame. Also, to authorize release of contact information of the Requesters to the staff only when needed, for example to get further context about an issue when it was not clearly stated (Scope item nr. 9 & out of scope item 2).

Conflict / competing expectation: Management want detailed reporting and accountability of their staff (who did what, when etc.). However, staff want minimal data entry and an interface that is easy and allows for fast data entry. Privacy of the requesters must also be ensured, information must only be provided when absolutely necessary as the requesters value their privacy of their data but at the same time if they are not specific enough about their issue, they will need to be contacted for further clarification.

1.2.5 Staff

Note: Each staff member is not a separate stakeholder; The staff overall is treated as a single stakeholder group.

The staff are the people who are actually assigned to investigate, handle and resolve the service requests made by the requester.

Their primary responsibility is to accept and claim ownership of the issue they are assigned to, including status updates, resolution details and contacting the requester in the event there is clarification needed (Scope item nr. 9 & out of scope item 2).

Their core need is to clearly see the issues that they are assigned to, and to be able to update the status of the requests they are working on. They want an interface that is easy to interact with and that requires minimal clicks and fast loading time (As they are going to be using this system a lot).

Conflict / competing expectation: Staff may resist to use this newly proposed solution in the event that it is slower than the current informal methods (WhatsApp, email, telephone calls etc.). In order to ensure that they will want to use the system, we must make the system as easy as possible to use, while still ensuring management has detailed reporting and accountability of their staff (who did what, when etc.).

1.2.6 Admin (added during M2)

M2 review: During millstone 2 we identified another stakeholder. We refer to this as the admin stakeholder. We identified something that we missed previously; how the staff and managers will be registered. Requesters will be able to self-register, but the application would be insecure if anyone could register as a manager or staff. So, this is where the admin user comes in, they register the managers and managers are mainly responsible for registering staff. However, the admin user will also be able to manage staff across different divisions.

1.2.7 Operator (added during M3)

M3 review: During Milestone 3,we identified a gap in the CivicConnect baseline. The platform assumes that requesters can access and use a digital interface. However, some of the requester stakeholders may be less tech-savvy or prefer existing communication channels. In addition, the organisation still holds incomplete or unresolved requests in email, telephone logs, WhatsApp messages, spreadsheets, and paper records. If these are not captured, service history becomes fragmented, open work may be lost, and accountability cannot be demonstrated.

To address this gap, we introduced the Operator role through change request CR-001. The Operator acts as an intermediary between external requesters (requests received via email, phone calls and WhatsApp messages) and CivicConnect. The role allows requests received through existing channels to be logged accurately into the system, and allows legacy or in-process requests to be migrated so that they are not lost. This supports inclusion, business continuity, traceability, and the organisation's need for a single controlled service-request record.

2. Scope Baseline & Constraints

2.1 Scope Baseline

2.1.1 In Scope

These are the core features and things that our application will and must be able to do. These are the things that we are promising to ensure that our system will perform, this is the baseline scope and will be adjusted as the project progresses (note in the event that the scope is adjusted we will properly document these changes):

1. User login, registration and authentication
   Requesters will be able to register and log in to access account specific functionality. They will also be able to update their personal information and settings such as if they want receive email push notifications.
   This authentication will also help with staff and management functionality as they will also have accounts. This will help with things such as staff being assigned to a specific department and also to a specific management person.

M2 Review: IN PEDv1 we established register for Requesters, Management and Staff. After further consideration we identified in PEDV2 that requesters will be able to self-register. Management will register staff for only their department. A special Admin user will register the management users and will also be able to manage staff across all departments.

2. Requesters submit a new service request
   Requesters will be able to submit a new service request containing all the required information.
3. Add images to a request
   Requesters will also be able to attach images to a specific request in the event that it is needed. However, images will be limited to a certain file size (still to be determined at a later stage what this established limitation is, as it depends on the database architecture limitations and free teer availability).
4. Controlled request categorization
   Requesters will select a request category from a list of available categories. These categories will then be used to make the request available to the relevant department. For example, IT related issues will only go to the IT department and NOT across the all the departments. This will help to better keep track of what the issue pertains to.
5. Status tracking of a request
   The requesters will be able to view the current status of their submitted request as it changes.
   Staff will be able to change the status of only the issues they are assigned to.
   Management then also has the dashboard view of the different requests and their status within their department and which staff member is assigned to the specific issue
6. View request history
   Requesters will be able to view the history of their previously submitted requests.
   Managers will also be able to view all the past requests logged within their department.
   Staff will be able to view their assigned request history.
7. Feedback via email push notification.
   As an additional feature that we will implement is the ability for requesters to receive email notifications as their status of their task changes. This feature will be able to be toggled on / off. This is also why we require a user registration with email as we will use this email address to send the push notifications to.
8. Search and filter requests
   Staff and Management will be able to search, filter and sort available service requests using relevant criteria.
9. View requester details
   Authorized staff will be permitted access to contact details associated with a requester. This is how the staff member will be able to get further clarification on an issue if required. The manager must first approve access to this information for the staff member.
10. Assign / accept responsibility
    Authorized staff will be able to accept responsibility for the appropriate unassigned requests. Meaning that they will be able to assign themselves to a specific request only within their department.
11. Management assign and reassignment
    Although staff are able to assign themselves to a task within their department, management still has the ability to assign or reassign service requests to the appropriate staff member within their department. This thereby allows managers to only assign task when needed, which allows them to spend more time on other responsibilities they have to attend to such as monitor the performance of their staff via the dashboard.
12. Update status
    The assigned staff member of a specific task will have the ability to update the status of the issue as it progresses.

13. Resolve and close request
    The assigned staff member will be able to resolve and close the request. Management is able to revert this closer if deemed the issue is not yet resolved.
14. Management dashboard and reporting
    Management will be able to view information to open, overdue, resolved and closed requests. Management will be able to analyse request information using the relevant search and filter tools available.
15. Digital platform Mobile accessibility
    For our proposed digital platform, some of the screens we will adapt for mobile as this will improve the user experience for the requesters.
16. Operator-assisted request capture (Added during M3)
    For users unable to use CivicConnect for whatever reason, the option to call an operator or to capture a request via other channels such as email or WhatsApp. Operators can now create requests on behalf of citizens.
17. Request Editing (Added during M3)
    Requesters can edit their submitted request (title, description, category, location, priority) and add/remove image attachments while the request is still in 'Submitted' status.
18. Citizen Search (Added during M3)
    Requesters can search and filter their own past requests by text, reference number and status

    M3 Scope Status Note: The listed items remain the agreed scope baseline. Their implementation and deployment readiness are evaluated separately in Sections 13.4, 13.8–13.9 and the Release Evidence Summary. In particular, operator editing (Issue #122), hosted SMTP delivery (Issue #124), password recovery (Issue #123), and manager export (Issue #121) remain incomplete or constrained. The existence of a user-interface control does not constitute verified completion.

2.1.2 Out of Scope

These are explicit decisions that that we will not be moving forward with and are excluded from the baseline scope.

1. AI chatbot
   We will not create an AI chatbot for users to communicate with.
   It is unlikely that users will make use of this feature. Also, when implementing something like this it can be complex there are certain risks such as people misusing the chatbot for personal reasons (Schuman, 2026). There are security and implementation risks associated that are not worth the potential value that it could add (Schuman, 2026).
2. Internal real time messaging system
   Requesters won't have direct communication with staff via the system. Staff will send an email or call the user only when the issue on the request is not clearly stated. The system will have the option to update the status of the issues. However the staff do have the option to leave comments to the users to update them on the issues. The comment feature is different from real time communication as the requester can not repond to the comment, the comment is merely there to inform the requester of any information relating to the update of their request.
3. Video file upload
   Requesters will not be able to attach video files to the service requests. Uploads will be restricted to images only due to storage limitations (images will take less space to store).
4. Voice input

Communication is limited to text only. Dealing with different audio files can get complex and also require a more data storage capacity. This potential feature is not worth the additional storage capacity.

5. Social media login
   We will provide one method of registration and that is email along with a password. We will not be integrating Google login, Login via Facebook. This overcomplicates the issue and requires an additional API's that could change or break. This could add to the technical debt of the application.

2.1.3 Deferred Scope

These are recognised possible future things we could implement in future milestones, but we deliberately postpone these issues due to schedule, cost, risk or missing evidence.

1. Auto assign issues to the same task

Ideally, it would be beneficial for the system to automatically group multiple requests to the same underlying issue. For example, a specific website that the community focused organization manages and is in charge of goes down. They might get multiple requests that the website is no longer accessible. So instead of dealing with each individual request we treat it as single issue. This could be advantageous, but this decision is differed as we would first like to implement the basic functionality first. This is a "nice to have".

2. Portal for requesters to review their interactions with the staff

Adding a reviews portal where requesters could review their interaction with a specific staff member if they had one. This is also another "nice to have".

2.2 Constraints

Constraint	Description	Engineering Implication
Team Size and Resources	The project must be completed by a team of only 3 members.	Because development resources are limited, we must avoid complex architectures that require more overhead. If one team member falls behind, the other two will do the workload, which means code must be readable and documented.
Schedule	The project must progress through 4 formal milestones within 7 weeks.	Time is heavily limited, so scope must be flexible. If a chosen technology is too difficult to learn or if development falls behind the schedule, then we must drop the "could have" and "should have" features rather than to miss a milestone's deadline.
Cost	The team must use free services where possible and identify operational limits.	This heavily restricts our tech stack for M2. We cannot use premium services. We must actively monitor our usage of free limits to ensure we do not hit a paywall.
Scope control	Committed scope must be baselined, uncontrolled scope creep is not acceptable.	Once M1 document is signed off, any new feature ideas cannot be silently added. It must go through a formal Change Request and impact analysis to ensure it does not interfere with our schedule or quality.
Evidence	Attributes must be defined and later supported by measurable evidence and not just claims.	We cannot just simply say the code works; we must prove it. This is implied we must allocate time to write automated test and configure continuous integration pipeline to act as a quality gate before any code merges into the main branch.
Security	Security must be treated as a lifecycle wide engineering responsibility not just a final add-on.	We must implement strict config management from day one. Things like database passwords and API keys must never be committed to our GitHub. This does require us to establish a secure environment strategy.

Technology and team capability	No technology stack is prescribed but decisions must be justified against the constraints and our team capability.	We cannot pick a popular framework just because it looks trendy. If we pick a tech stack that nobody in the team knows there is a learning curve risk and that will threaten our schedule.

3. Requirements & Acceptance Criteria

3.1 Functional Requirements

Priority:

M – Must have.

S – Should have.

C – Could have.

How the Requirement IDs work; The first number is for stakeholder when it comes to FR and category when it comes to NFRs. The second and 3rd number is just for numbering the requirement.

Requirement ID	Group of Feature	Description	Priority	Source/Stakeholder	Acceptance Criteria
FR-101	Submission	The system must allow users to submit a service request using a category mechanism.	M	Requester	When valid details are submitted when the user clicks, save/submit then a unique ID is generated.
FR-102	Tracking	The system must allow users to view the status of requests.	M	Requester	Given that the user inputs their unique ID, then the ticket status is displayed.
FR-103	History	The system should allow authenticated users to view a history of their requests	S	Requester	Given that the user is logged in, when navigating to their profile, then a list of their past request should render.
FR-104	Feedback	The system must provide feedback when a user's request is updated.	M	Requester	When a request is accepted, rejected, updated or completed, the system must generate an appropriate notification for the requester.
FR-105	Authentication & Registration	The system must allow requesters to register an account and authenticate before accessing account-specific functionality.	M	Requester	Given valid registration information, the system must create a requester account. Given valid registered credentials, the system must authenticate the requester. Invalid credentials must be rejected.
FR-106	Attachments	The system should allow requesters to attach supported image files to a service request.	S	Requester	Given a requester creating or submitting a request, when a supported image is attached, the image must be associated with the correct service request and be retrievable by an authorised user. Requesters can only attach up to 5 images, preview them and remove them during editing. (Updated during M3)
FR-201	Assignment	The system must allow staff to accept responsibility for a ticket.	M	Staff	Given that an unassigned ticket and the staff user clicks assign, then they must become the record owner.

FR-202	Workflow	The system must allow staff to update request statuses through controlled transitions.	M	Staff	Given a ticket, if updated to a different status then the status state is saved in the database.
FR-203	Staff View	The system must allow the staff to search, filter and sort request using criteria.	M	Staff	Given the staff dashboard, when filtering by criteria, then only tickets related to that criteria are displayed.
FR-204	Resolution	The system must allow staff to record actions, comments or resolution information.	M	Staff	Given that a ticket is being resolved, then the staff member must input a resolution comment before closing it.
FR-205	Request Details	The system must allow authorised staff to view the full permitted details of service requests relevant to them.	M	Staff	Given an authorised staff member opening an available request, the system must display the permitted request information associated with that request.
FR-206	Request Closure	The system must allow authorised staff to resolve and close a service request through the controlled workflow.	M	Staff	Given an authorised staff member resolving a request, when all required resolution information has been provided and a permitted closing transition is performed, the request must be recorded as resolved or closed.
FR-207	Department-Relevant Access	The system must make service requests available to authorised staff according to the request category and relevant staff responsibility or department.	M	Staff	Given a service request assigned to a defined category, authorised staff within the relevant responsibility area must be able to access it, while staff outside the authorised scope must not be able to access it.
FR-301	Dashboard	The system must provide a dashboard to identify open, overdue and closed requests.	M	Management	Given that a manager accesses the dashboard, then a visual metric of active vs closed tickets should render.
FR-302	Auditing	The system should support reporting on request lifecycle history for accountability.	S	Management	Given a completed ticket, then a timestamp audit log of all transitions should be viewable.
FR-303	Assignment & Reassignment	The system must allow authorised management to assign or reassign responsibility for a service request to an appropriate staff member.	M	Management	Given an eligible service request and authorised manager, when a valid staff member is selected and the assignment is confirmed, that staff member must become the recorded assignee and the assignment change must be recorded.
FR-304	Request Analysis	The system must allow authorised management to analyse service-request information using relevant dimensions such as category and status.	M	Management	Given an authorised manager viewing service-request information, when a supported category, status or other approved dimension is selected, the resulting information must correspond to the selected criterion.
FR-305	Contact Information Authorisation	The system must restrict staff access to requester contact information and allow authorised management to control access where additional clarification is required.	M	Management	Requester contact information must not be made available to an unauthorised staff member. Where the required access has been authorised for a request, the permitted information must become available only to the appropriate authorised staff member.
FR-401 (M3)	Operator Submission	Operators can log service requests on behalf of external community members.	M	Operator	Operators must be able to accurately capture request from citizens. Including the requests category, title, description and priority.

FR-402 (M3)	External Requester details	Capture external requester contact details.	M	Operator	Given that a citizen chooses to log a request via an operator, the operator must be able to capture the citizen's contact information (either phone number or email, and the preferred contact method) for updates on the request. Also, other details such as the requesters first name and last name.
FR-403 (M3)	Community member search	Search and link existing requesters to prevent duplicate records.	M	Operator	An operator should be able to search for request that has been previously captured to ensure the same issue doesn't get captured multiple times.
FR-404 (M3)	Operator Logged view	Dedicated view for operators to review requests logged by them.	M	Operator	Given an operator has made multiple requests, a dedicated view is necessary for them to confirm and revisit their request made by them.

3.2 Non-Functional Requirements

Requirement ID	Category	Description	Measurable Acceptance criteria	Justification
NFR-101	Security	The system must enforce authentication and data protection.	Passwords must be hashed. Data in transit must use https. Role-Based Access Control must prevent requesters from accessing staff endpoints. (measurable through test gates, binary pass/fail)	“Security is a lifecycle wide responsibility not a final add-on", Because CivicConnect handles sensitive community data such as locations and contact details, failing to secure trust early creates privacy risks.
NFR-201	Performance	The platform must respond efficiently to user inputs to prevent duplicate submissions.	95% of all standard API read and write requests must compile in less than 800 milliseconds under normal system load.	If the system responds slow, then users will click “submit/save” multiple times, leading to duplicated tickets. By setting a millisecond limit, gives the team a measurable goal.
NFR-301	Constraints	The architecture must operate for free (open source) or within low cost.	The production environment must run on free-tier services without exceeding monthly quotas.	Cost is an engineering constraint in this Project.
NFR-401	Compatibility	Given that the decision is made that the system needs to operate on multiple different platforms -The public portal must be usable on modern desktop and mobile browsers. Please note this decision will only be made in milestone 2.	The UI must pass responsive visual testing on the newest versions of Chrome, Safari, Edge and Firefox without horizontal scrolling on mobile.	Since native mobile applications are out of scope, the web interface is the only way citizens can submit a new service request. Requesters will most likely report issues on the go, using their mobile browsers, making responsiveness a crucial attribute.
NFR-501	Reliability	The system must handle erroneous inputs without crashing.	100% of the form submissions must have backend validation. The system must return human readable http error codes instead of stack traces.	If the system crashes silently on bad input, tickets are lost. Validation ensures data integrity and provides feedback to the user.

4. Architecture-Significant Requirements and Quality Drivers

4.1 Identification of Architecture-Significant Requirements

Not every CivicConnect requirement has the same influence on the architecture. For Milestone 2, the team reviewed the existing functional requirements, non-functional requirements, stakeholder needs, project constraints, risks and forward engineering considerations to identify the requirements and quality attributes that materially affect the structure and engineering direction of the system. These are treated as Architecture-Significant Requirements (ASRs) because failing to address them early could require substantial redesign, introduce significant project risk, or prevent important stakeholder and project requirements from being achieved.

The identified ASRs are used as decision criteria when evaluating the CivicConnect architecture, data and persistence approach, technology stack and initial design. Measurable expectations are retained or defined where appropriate so that the effect of these decisions can later be verified through implementation and testing. The team has deliberately selected only the drivers that materially influence the system rather than treating every non-functional requirement as an ASR.

4.2 Selected CivicConnect ASRs and Quality Drivers

ID	ASR / Quality Driver	Project Evidence / Source	Measurable / Testable Expectation	Why It Is Architecture-Significant
ASR-01	Security, Authorisation and Controlled Access	NFR-101; FR-105 Authentication & Registration; FR-207 Department-Relevant Access; FR-305 Contact Information Authorisation; new M2 Admin stakeholder; RSK-003 Unauthorised access to requester information.	Passwords must be securely hashed; protected communication must use HTTPS; unauthorised users must be denied protected functionality; role and department restrictions must be enforced server-side; requester contact information must only be accessible where the required authorisation exists.	CivicConnect contains Requester, Staff, Management and Admin roles with different levels of access. Requests are department-restricted and some requester information requires additional Management approval. Security therefore cannot exist only in the user interface; authentication and authorisation must influence application boundaries, request handling, business logic and persistence relationships.
ASR-02	Data Integrity, Lifecycle Traceability and Reliability	FR-202 Workflow; FR-204 Resolution; FR-302 Auditing; FR-303 Assignment & Reassignment; NFR-501 Reliability; RSK-008 Email notification dependency/failure; RSK-010 Loss or corruption of platform data.	Valid request, assignment and status changes must be persisted correctly; invalid input must be rejected through backend validation; required lifecycle changes must be traceable; failure of a secondary function such as email notification must not invalidate or lose the underlying request update.	A central business purpose of CivicConnect is to replace fragmented processes in which requests can be lost, duplicated or poorly tracked. Request state, assignment and audit information must therefore remain consistent across the application. This materially influences the database model, transaction boundaries, validation, audit/history storage and separation between core request processing and secondary functions such as notifications.
ASR-03	Maintainability, Modifiability and Testability	Three-person team constraint; limited milestone schedule; RSK-001 Schedule overrun; RSK-009 Team-member availability/knowledge concentration; evolving requirements across M2-M4.	Core responsibilities must be separated sufficiently for components/modules to be understood, changed and tested independently where practical. Core business logic should be testable without requiring the complete user interface or external services to execute. Significant code must remain understandable and reviewable by more than one team member.	The application must continue evolving across later milestones while being developed by only three team members. Excessive coupling or unnecessary architectural complexity would increase development, testing and knowledge-transfer effort. This driver therefore influences module boundaries, separation of concerns, dependency management and the level of architectural complexity that is acceptable.
ASR-04	Performance and Interaction Responsiveness	NFR-201 Performance; Staff stakeholder need for fast interaction and minimal delay; Management search/filter/dashboard requirements.	The existing NFR target remains that 95% of standard API read and write requests complete within 800 milliseconds under normal system load.	Staff are expected to use the system regularly for request handling, while Requesters need efficient submission and tracking. Performance therefore influences request-processing overhead, database queries and indexing,

ID	ASR / Quality Driver	Project Evidence / Source	Measurable / Testable Expectation	Why It Is Architecture-Significant
			Repeated user actions must not be required because normal operations respond too slowly.	search/filter implementation and the amount of unnecessary communication introduced between application components.
ASR-05	Cost-Effective Deployability and Resource Efficiency	NFR-301; three-person team; free/low-cost project constraint; RSK-005 Unsuitable technology/deployment platform; RSK-006 Free-tier resource/service limitations; image-upload and email features.	CivicConnect must be executable in the team's development environment and must remain deployable using a free or sufficiently low-cost environment appropriate to the educational project. Hosting, database, storage and supporting-service limitations must be identified before final commitment, and secrets/configuration must remain external to source control.	Cost and deployment limits constrain the amount of infrastructure and operational complexity the team can responsibly support. Architecture, persistence, image storage, notifications and deployment must therefore avoid requiring unjustified infrastructure or paid services and remain practical for a three-person team to configure, test and maintain.

4.3 Influence of ASRs on M2 Engineering Decisions

ASR	Architecture Influence	Data / Persistence Influence	Technology Influence	Initial Design Influence
ASR-01 Security, Authorisation & Controlled Access	Requires clear security boundaries and consistent enforcement of authentication and authorisation throughout the server-side application. Architecture alternatives must support centralised and reusable access-control rules rather than relying only on UI restrictions.	User roles, departments, staff assignments, requester ownership and controlled contact-information access must be represented in the data model. Access relationships must support both role-based and department-based restrictions.	Authentication technology must support secure credential/token or session handling, password hashing and server-side authorisation. The chosen web framework and deployment platform must support secure configuration and secrets management.	Controllers/routes and business operations must check authorisation before protected actions are performed. Reusable access-control logic should be preferred over duplicating permission rules throughout the application.
ASR-02 Data Integrity, Traceability & Reliability	Architecture must maintain a clear responsibility for request workflow and persistence so that request state does not become inconsistent across components. Secondary operations such as notification should not determine whether the underlying request update succeeds.	A structured persistence model must represent requests, status/history, assignments, departments, users and audit information. Integrity constraints, validation and transaction support must be considered when finalising the persistence approach.	The database technology must provide reliable persistent storage and appropriate integrity/transaction capabilities. This requirement will be considered when finalising the current PostgreSQL direction.	Workflow rules and validation should not be duplicated across unrelated UI/controllers. Operations that logically belong together must be handled consistently, while notification/audit responsibilities should not unnecessarily corrupt or control the core request transaction.
ASR-03 Maintainability, Modifiability & Testability	Architecture alternatives must provide meaningful separation of responsibilities without introducing unnecessary distributed complexity. Clear internal module/layer boundaries are preferable to a structure where unrelated concerns become tightly coupled.	Persistence access should be separated sufficiently from presentation and request-routing concerns so that changes to storage logic do not unnecessarily affect the complete application.	Technology choices should favour tools the team can realistically understand, test and maintain within the available schedule. The current JavaScript/Node.js/Express direction and Jest testing proposal must therefore be evaluated partly against team capability and maintainability.	MVC/separation of concerns, cohesive components and limited coupling should guide design. Core business behaviour should be testable independently where practical.
ASR-04 Performance & Interaction Responsiveness	Architecture should avoid unnecessary processing and communication overhead for ordinary request operations. Introducing distributed/network boundaries must therefore have a clear benefit rather than	Frequently used queries, authentication lookups, request filtering and dashboard operations will require suitable database relationships, indexing and query design.	The selected runtime, framework and database must be capable of meeting the existing response target for the expected project workload.	Request submission should prevent unnecessary duplicate actions; search/filter operations should use efficient queries rather than processing unnecessarily large result sets in the presentation layer.

ASR	Architecture Influence	Data / Persistence Influence	Technology Influence	Initial Design Influence
	being added only for architectural complexity.			
ASR-05 Cost-Effective Deployability & Resource Efficiency	Architecture alternatives must remain proportionate to the size of CivicConnect and practical to deploy and operate with limited resources. Infrastructure-heavy or distributed approaches require stronger justification.	Database and image-storage choices must consider free/low-cost storage limits, persistence availability and recovery implications.	Railway and Render remain deployment candidates and should be compared against cost/free-tier availability, PostgreSQL compatibility, configuration/secrets support, operational simplicity and team capability. Local execution remains useful for development and demonstration, while Docker should only be introduced if it provides sufficient project value to justify the additional complexity.	Configuration should remain externalised from source code. Image size/type controls and failure behaviour for supporting services should be designed with resource limitations in mind.

4.4 Quality Requirements Not Treated as Primary ASRs

The team also reviewed the remaining non-functional and usability concerns but did not automatically classify each one as a primary ASR. In particular, NFR-401 browser/mobile compatibility remains an important quality requirement, but its current effect is concentrated mainly on the Requester-facing presentation and interaction design rather than determining the fundamental backend or persistence architecture. The M2 decision already establishes that Requester-facing functionality will support mobile browser use while the extent of Staff and Management mobile responsiveness remains deliberately deferred. NFR-401 will therefore remain a supporting quality driver and will primarily influence UI design, browser compatibility testing and responsive-layout decisions.

Similarly, detailed CI quality gates and later production observability remain important engineering concerns but are not treated as independent architecture drivers at this stage. They will continue to influence testability, deployment and later M3 quality/release-readiness work rather than being added as separate ASRs merely to expand the list

5. Architecture Baseline

5.1 Architecture Context and Evaluation Criteria

CivicConnect requires an architecture that supports the current functional requirements and the Architecture-Significant Requirements identified in Section 4, while remaining practical for a three-person development team working within a limited schedule and free or low-cost resource constraints. The architecture must therefore provide sufficient separation of responsibilities, support the required quality attributes, and avoid complexity that is not justified by the project.

The architecture alternatives will be evaluated against the following project-specific criteria:

- **Security and controlled access (ASR-01):** The architecture must support consistent authentication and authorisation, including role-based and department-based access restrictions and controlled access to requester information.
- **Data integrity and traceability (ASR-02):** The architecture must support reliable request lifecycle, assignment, status and audit information without introducing unnecessary consistency problems.
- **Maintainability and testability (ASR-03):** Responsibilities should be sufficiently separated so that parts of the system can be understood, changed and tested without excessive coupling.
- **Performance (ASR-04):** The architecture should avoid unnecessary processing and communication overhead that could negatively affect normal request, search and dashboard operations.
- **Cost and deployability (ASR-05):** The architecture must remain practical to run and deploy using free or low-cost infrastructure with limited operational overhead.
- **Team capability and schedule:** The architecture must be realistic for three students to implement, test, document and maintain within the available milestones.
- **Proportional complexity:** Additional architectural complexity must provide a clear CivicConnect-specific benefit. Distributed components or additional infrastructure should not be introduced unless they are justified by the project requirements, risks or constraints.

The Master Project Brief requires architecture and technology decisions to be justified against the project requirements and constraints rather than selected according to preference alone. SEN381 Master Project Brief The Milestone 2 brief also states that a more distributed architecture is not automatically more advanced and that additional complexity must be justified by project evidence. SEN381_CivicConnect_Milestone_2

These criteria will be used in the following section to compare the realistic architecture alternatives before the final CivicConnect architecture is selected.

5.2 Architecture Alternatives Considered

Three realistic architecture alternatives were considered for CivicConnect: a Traditional Layered Monolith, a Modular Monolith, and a Microservices / Distributed Architecture. The alternatives were compared against the project-specific criteria identified in Section 5.1. The purpose of the comparison is not to select the most complex architecture, but to identify the approach that best satisfies the CivicConnect ASRs, project constraints and team capability.

Project-Fit Colour Scale

Best Fit	Good Fit	Acceptable Fit	Weak Fit / Added Complexity	Too Complex / Not Suitable
Strongest match for CivicConnect and its constraints.	Fits the project well with only minor limitations.	Usable, but has notable trade-offs for this project.	Possible, but introduces unnecessary limitations or complexity.	Poor fit for the current scope, team size or constraints.

Comparison of Architecture Alternatives

Evaluation Criteria	Traditional Layered Monolith	Modular Monolith	Microservices / Distributed Architecture
Security and controlled access	**Good Fit.** Centralised authentication and authorisation are relatively simple to enforce within one application.	**Best Fit.** Centralised security can be maintained while access-related responsibilities are separated into clear internal modules.	**Acceptable Fit.** Can provide strong service boundaries, but security must be managed across multiple services and network interfaces.
Data integrity and traceability	**Good Fit.** A single application and database make transactions and lifecycle consistency easier to manage.	**Best Fit.** Retains simple transaction handling while allowing request, workflow and audit responsibilities to be separated internally.	**Weak Fit / Added Complexity.** Data and transactions may need to span multiple services or databases, adding consistency and coordination complexity.
Maintainability and modifiability	**Acceptable Fit.** Can be maintainable initially, but responsibilities may become tightly coupled as the application grows.	**Best Fit.** Functional areas can be separated into modules while remaining within one application and one deployment boundary.	**Weak Fit / Added Complexity.** Provides strong service separation, but adds multiple codebases, interfaces and operational responsibilities that are heavy for this team.
Testability	**Good Fit.** Layers can be tested independently if separation is maintained correctly.	**Best Fit.** Clear modules and internal boundaries support focused unit and integration testing without distributed-system overhead.	**Acceptable Fit.** Services can be tested independently, but substantially more integration and end-to-end testing is required.
Performance and responsiveness	**Good Fit.** Internal operations use direct in-process communication with little additional overhead.	**Best Fit.** Internal module communication remains in-process while preserving clearer responsibility boundaries.	**Weak Fit / Added Complexity.** Network communication between services introduces additional latency and more failure points.
Deployment simplicity	**Best Fit.** One application is straightforward to configure and deploy.	**Best Fit.** Retains one primary deployment unit despite clearer internal structure.	**Too Complex / Not Suitable.** Several services may need to be configured, deployed, connected and monitored separately.
Free / low-cost operation	**Best Fit.** Requires relatively little infrastructure and is practical for free or low-cost hosting.	**Best Fit.** Provides additional structure without requiring multiple hosting environments or distributed infrastructure.	**Weak Fit / Added Complexity.** Multiple services, databases or supporting infrastructure may increase resource use and cost.

Evaluation Criteria	Traditional Layered Monolith	Modular Monolith	Microservices / Distributed Architecture
Three-person team capability	**Good Fit.** Relatively simple for a small team to understand and maintain.	**Best Fit.** Provides useful separation of responsibilities without introducing excessive operational complexity for three team members.	**Too Complex / Not Suitable.** Requires additional development, integration, deployment and maintenance effort that is not realistic for the current team size.
Limited milestone schedule	**Best Fit.** Can be implemented quickly, although poor separation may create later maintenance problems.	**Best Fit.** Adds useful structure while remaining achievable within the available milestone schedule.	**Too Complex / Not Suitable.** Significant time would be spent on service infrastructure and integration instead of CivicConnect functionality.
Future growth and change	**Acceptable Fit.** The application can grow, but larger changes may become harder if responsibilities become tightly coupled.	**Best Fit.** Modules can evolve independently within the application and provide a clear path for later restructuring if needed.	**Good Fit.** Offers strong theoretical scalability and independent deployment, although this capability exceeds current project needs.
Overall complexity relative to project needs	**Good Fit.** Simple and proportionate, but may provide too little internal separation as CivicConnect develops.	**Best Fit.** Provides useful architectural structure while keeping deployment and operations simple enough for the project.	**Too Complex / Not Suitable.** The additional service, network and operational complexity is not justified by the current scope, team or deployment constraints.

Comparison Outcome

The Traditional Layered Monolith is simple to build, test and deploy, but it may become harder to maintain as CivicConnect grows and responsibilities become more tightly coupled. In contrast, a Microservices / Distributed Architecture provides stronger separation but introduces unnecessary deployment, integration, monitoring and data-consistency complexity for a three-person team working within a limited schedule and low-cost environment.

For these reasons, the **Modular Monolith** was selected as the most appropriate architecture for CivicConnect. It provides a balance between the simplicity of a traditional monolith and the separation of responsibilities normally associated with more distributed architectures. CivicConnect can remain a single deployable application while internally separating major responsibilities into modules such as:

- **Authentication and User Management**
- **Request Management**
- **Assignment and Workflow**
- **Audit and Request History**
- **Notifications**
- **Management Reporting and Dashboard**

These modules can operate through controlled internal boundaries while sharing the same application and persistence environment. This allows CivicConnect to maintain straightforward deployment, transaction handling and performance while improving maintainability, testability and separation of concerns. If the system were to grow substantially in the future, these clearer internal boundaries would also make later architectural changes easier than if the system were developed as a tightly coupled monolith

5.3 Selected Architecture and Rationale

Based on the comparison in Section 5.2, the **Modular Monolith** was selected as the architecture for CivicConnect. It provides the best balance between clear separation of responsibilities and the implementation simplicity required by the current project.

CivicConnect will remain a **single deployable application**, while its major responsibilities will be separated into clear internal modules. The initial responsibility areas are:

- Authentication and User Management
- Request Management
- Assignment and Workflow
- Audit and Request History
- Notification Management
- Search, Reporting and Dashboard

These responsibility areas align with the current CivicConnect requirements for authentication, request submission and tracking, controlled workflow and assignment, auditing, notifications, search/filtering and management reporting.

The modular monolith allows these areas to remain logically separated while still sharing the same application and persistence environment. This supports simpler deployment, consistent security enforcement, straightforward transaction handling and focused testing without introducing the additional infrastructure and coordination required by independently deployed services.

The main trade-off is that the modules cannot be deployed or scaled independently. This is accepted because the current CivicConnect requirements do not justify the added complexity of a distributed architecture. If the system grows significantly in future, the clearer internal module boundaries would provide a stronger foundation for later architectural change.

5.4 Logical Architecture and Responsibilities

Within the selected Modular Monolith, CivicConnect will use clear internal responsibility boundaries so that presentation, request handling, business rules and persistence concerns do not become mixed together. The aim is to keep the application cohesive while preventing one part of the system from taking responsibility for unrelated behaviour.

The logical responsibility flow will follow these principles:

- **Presentation responsibilities** handle what each stakeholder sees and the input they provide. User-interface components should display information and collect input, but should not contain core workflow, authorisation or persistence rules.
- **Request-handling responsibilities** receive user actions and direct them to the appropriate application behaviour. They should coordinate the request rather than contain all business logic themselves.
- **Business-rule responsibilities** contain the rules that control CivicConnect behaviour, such as who may access information, how requests move through their lifecycle, and how assignment or closure decisions are handled.

- **Persistence responsibilities** handle the storage and retrieval of application data. Database operations should remain separated from presentation logic and should support the integrity and traceability requirements already identified in the PED.
- **Supporting responsibilities**, such as notifications, auditing and reporting, should interact with the core request workflow without unnecessarily controlling or duplicating it.

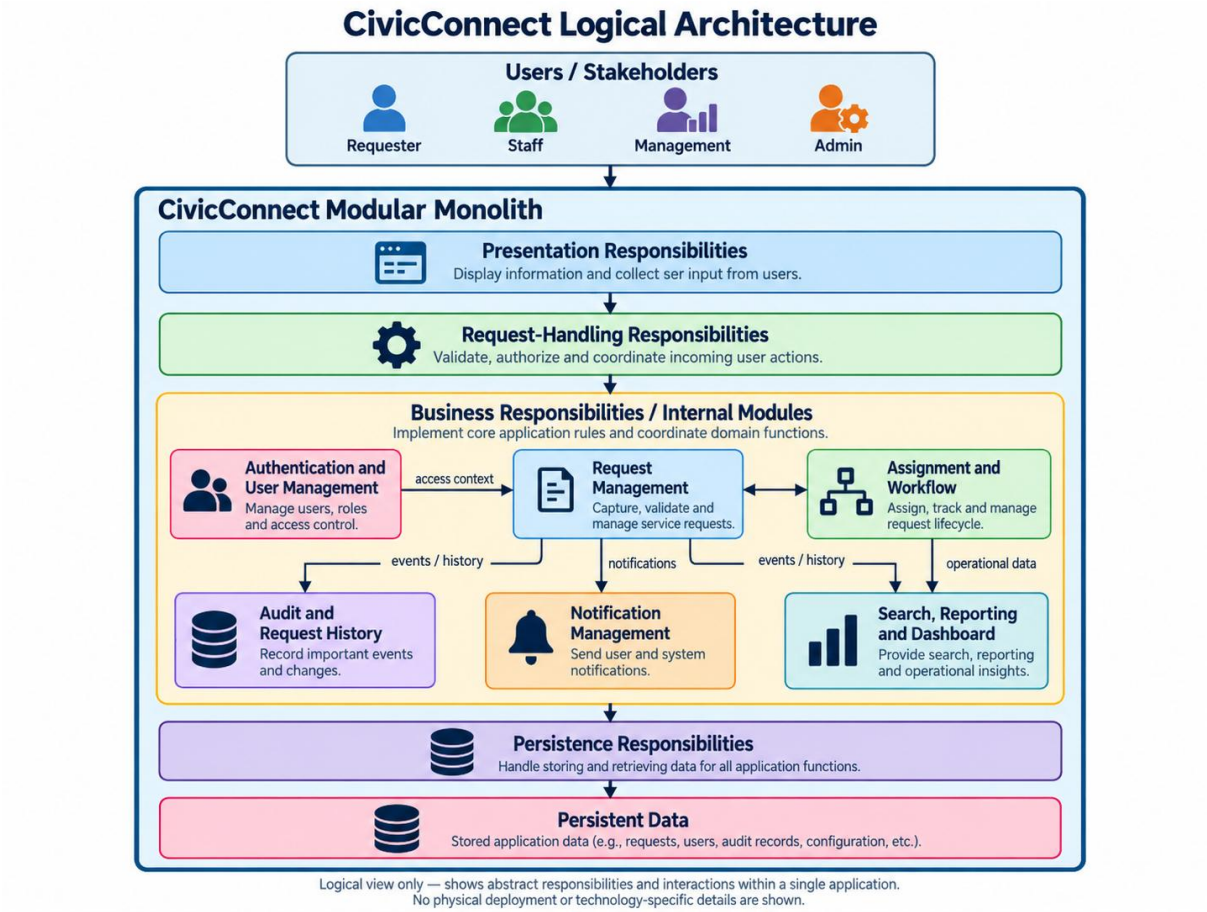
Interaction between internal modules should occur through clearly defined responsibilities. A module should use another module's functionality where required rather than reimplementing the same rule in several places. For example, request-processing functionality should rely on the established authentication and access-control responsibilities when determining whether a user may perform an action, while significant workflow changes should be passed to the audit/history responsibility rather than recorded independently in several parts of the system.

This logical separation supports the maintainability and testability goals identified in ASR-03 while still keeping CivicConnect within one application boundary. It also supports the project's requirement for controlled request lifecycle information and accountability without introducing separate network services or additional deployment boundaries. The project brief requires architecture and design decisions to remain proportionate and traceable to actual project needs rather than adding unnecessary complexity.

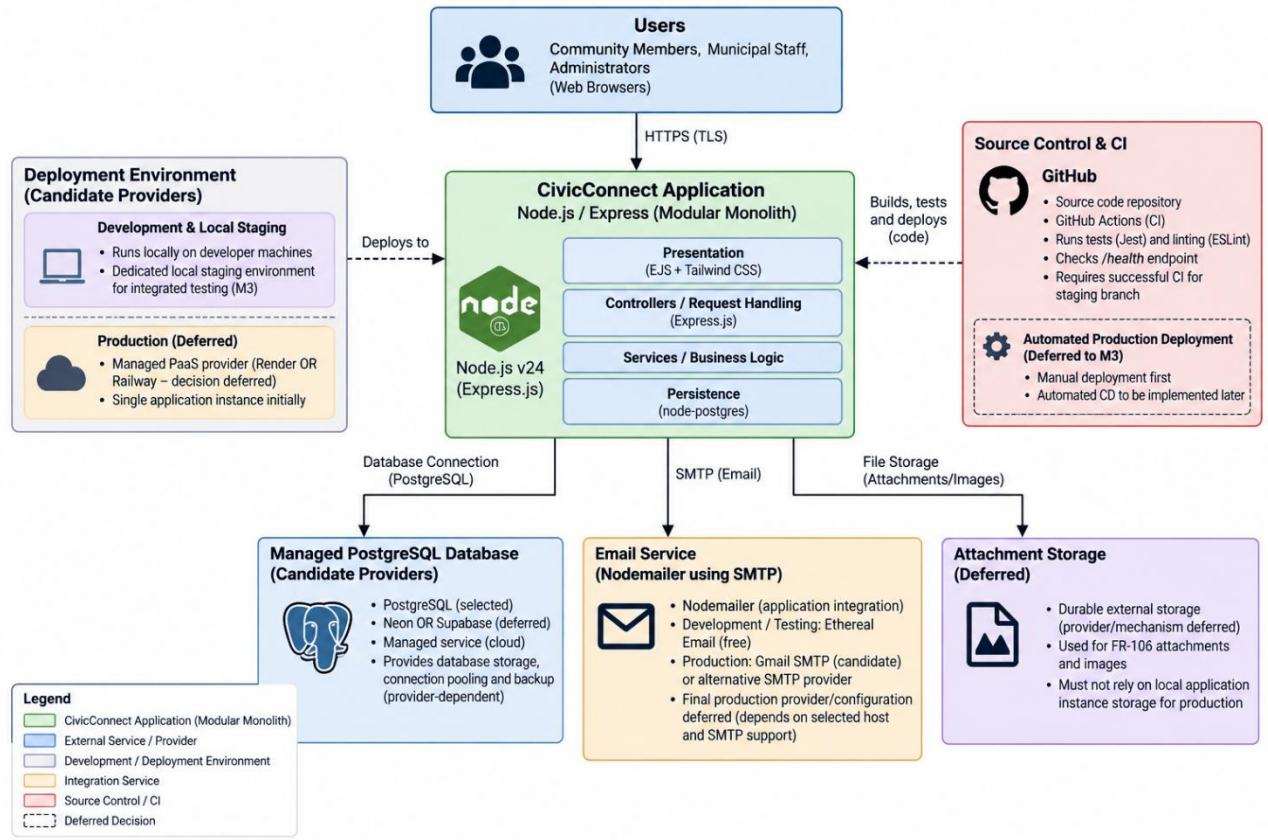
5.5 Architecture Diagrams

The architecture diagrams below show CivicConnect from two different perspectives. The first diagram shows the **logical structure and interaction of responsibilities within the Modular Monolith**, while the second shows the **physical deployment view**. Keeping these views separate avoids confusing internal modules with independently deployed services, which is explicitly required by the Milestone 2 brief.

Logical Architecture Diagram



Physical Deployment Diagrams



5.6 Architecture Decision Record and Consequences

The selection of the **Modular Monolith** as the CivicConnect architecture is recorded in a dedicated Architecture Decision Record (ADR). The ADR captures the alternatives considered, the ASRs and project constraints that influenced the decision, the rationale for selecting the Modular Monolith, and the trade-offs and consequences that follow from this choice. The ADR is maintained as a controlled project artefact in the GitHub repository and linked to the relevant PED sections and supporting evidence.

Decision Consequences

The main consequences of selecting the Modular Monolith are:

- CivicConnect will be developed and deployed as a single application, reducing deployment and operational complexity.
- The application will be separated into clear internal modules, supporting maintainability, testability and separation of responsibilities.
- Communication between modules remains within the same application process, avoiding unnecessary network overhead and distributed-system failure concerns.
- Request workflow, audit and persistence operations can remain within a common application and database boundary, simplifying transaction handling and data consistency.
- The architecture remains practical for the current three-person team, limited milestone schedule and low-cost deployment constraints.

- Internal module boundaries must be maintained during implementation; otherwise, the application could gradually become tightly coupled.
- Individual modules cannot be deployed or scaled independently.
- A failure in the main application process may affect the entire system because all modules run within the same application.
- If CivicConnect grows significantly in future, selected modules may need to be restructured or extracted. The defined internal boundaries should make this easier than if the system were implemented as a tightly coupled monolith.

If later project evidence requires a significant change to the architecture baseline, the ADR and affected PED content will be updated through the project's controlled change process.

M3 Implementation Cross-Reference: The selected modular monolith remains the implemented architecture. The single Express deployable contains separated feature modules and communicates with managed PostgreSQL and external object storage. Implementation evidence and deviations are indexed in Section 13.4. Architecture compliance does not, by itself, establish NFR-201 performance compliance (Section 13.10).

6. Data & Persistence Baseline

The following sections describe the tables that make up our database. For each table, the description states its purpose and the requirements it satisfies, names the tables it refers to (the relationships) and the tables that refer to it by means of foreign keys, lists its columns, and describes its indexes and constraints.

All primary keys are UUIDs generated by the database.

6.1 Core Data Entities and Relationships

6.1.1 The users Table

The users table is used for authentication and authorisation of all user types (requesters, staff, managers and administrators), for user profile management, and for tracking user activity.

Satisfies FR-105 (Authentication & Registration), FR-207 (Department-Relevant Access) and NFR-101 (Security).

The users table does not refer to any other table. It is referred to by the departments table (one-to-one, via manager_id), the staff_assignments table (one-to-many, via staff_id and assigned_by), the service_requests table (one-to-many, via requester_id, assigned_staff_id, assigned_by and contact_approved_by), the request_attachments table (one-to-many, via uploaded_by), the request_status_history table (one-to-many, via changed_by), the request_assignment_history table (one-to-many, via assigned_to_staff_id and assigned_by_id), the request_comments table (one-to-many, via commented_by) and the admin_audit_log table (one-to-many, via admin_id). Each of those tables uses the users table for one or more roles, such as requester, assignee, assigner, manager or the user who performed an action.

Columns

- user_id: A primary key used to uniquely identify each user.
- email: The user's email address, used to log in. Must be unique.
- password_hash: The hash of the user's password. The password itself is never stored.
- first_name: The user's first name.

- last_name: The user's last name.
- phone: The user's telephone number. Optional.
- user_type: The role of the user. Can be one of: REQUESTER, STAFF, MANAGER, ADMIN.
- is_active: Indicates whether the user is active. Setting this to FALSE serves as an alternative to deleting a user. Defaults to TRUE.
- email_notifications_enabled: Indicates whether the user receives email notifications. Defaults to TRUE.
- created_at: When the user was created.
- updated_at: When the row was most recently updated.
- last_login_at: When the user last logged in.

Indexes and constraints

- Primary key on user_id.
- Unique on email, used for fast authentication lookups.

6.1.2 The admin_audit_log Table

The admin_audit_log table records administrative actions, tracks changes made to the system, and maintains accountability for user and system management. It is an independent audit trail and is not linked to individual service requests.

Satisfies FR-302 (Auditing) and NFR-101 (Security).

The admin_audit_log table refers to the users table (many-to-one, via admin_id). It is not referred to by any other table.

Columns

- audit_id: A primary key used to uniquely identify each audit record.
- admin_id: A foreign key identifying the administrator who performed the action, in the users table.
- action_type: The action that was performed. Can be one of: USER_CREATED, USER_MODIFIED, USER_DEACTIVATED, DEPARTMENT_CREATED, DEPARTMENT_MODIFIED, STAFF_ASSIGNED, STAFF_REMOVED, MANAGER_CREATED, MANAGER_REMOVED, CATEGORY_CREATED, CATEGORY_MODIFIED.
- target_type: The kind of record the action was performed on. Can be one of: USER, DEPARTMENT, CATEGORY, ASSIGNMENT.
- target_id: The identifier of the record the action was performed on. This is not a foreign key, because it may point to a row in any of the tables named by target_type.
- changed_data: A JSON document describing what was changed.
- created_at: When the action was recorded.

Indexes and constraints

- Primary key on audit_id.
- No secondary indexes are defined in the current script.

6.1.3 The departments Table

The departments table organises service request categories by department. It allows staff to be assigned to departments and supports department-specific dashboards and reporting.

Satisfies FR-207 (Department-Relevant Access), FR-301 (Management Dashboard) and FR-304 (Request Analysis).

The departments table refers to the users table (one-to-one, via manager_id) and is referred to by the staff_assignments table (one-to-many, via department_id) and the request_categories table (one-to-many, via department_id).

Columns

- department_id: A primary key used to uniquely identify each department.
- name: The name of the department. Must be unique.
- description: A description of the department.
- manager_id: A foreign key identifying the manager of the department, in the users table. Every department has exactly one manager, and a user can manage at most one department, which makes this a one-to-one relationship.
- is_active: Indicates whether the department is active. Setting this to FALSE serves as an alternative to deleting a department.
- created_at: When the row was created.
- updated_at: When the row was most recently updated.

Indexes and constraints

- Primary key on department_id.
- Unique on name.
- Unique, not-null constraint on manager_id. This enforces the one-to-one relationship with users and also supports manager lookups for dashboard filtering and department assignment validation.

6.1.4 The staff_assignments Table

The staff_assignments table records which staff members belong to which departments and enforces department-based access control on requests. It keeps a history of moves, removals and reassignments, but a staff member can have only one active assignment at a time.

Satisfies FR-207 (Department-Relevant Access) and FR-302 (Auditing).

The staff_assignments table refers to the users table (many-to-one, via staff_id and assigned_by) and the departments table (many-to-one, via department_id). It is not referred to by any other table.

Columns

- staff_assignment_id: A primary key used to uniquely identify each staff assignment.
- staff_id: A foreign key identifying the staff member, in the users table. A user can have many assignments over time.
- department_id: A foreign key identifying the department the staff member is assigned to, in the departments table.
- assigned_by: A foreign key identifying the user who made the assignment, in the users table.
- is_active: Indicates whether this is the staff member's current assignment. When a staff member moves or is removed, the row is kept and this column is set to FALSE. Defaults to TRUE.
- assigned_at: When the staff member was assigned to the department.
- removed_at: When the staff member was removed from the department. NULL while the assignment is active.

Indexes and constraints

- Primary key on staff_assignment_id.
- Partial unique index on staff_id where is_active is TRUE. This enforces that a staff member has only one active department at a time, and is used to validate access permissions.
- Unique index on (staff_id, department_id), which prevents the same staff member being assigned to the same department more than once.

6.1.5 The service_requests Table

The service_requests table stores service requests. It is the central record for all request-related operations, tracking status progression and managing assignment.

Satisfies FR-101 (Submission), FR-102 (Status Tracking), FR-201 (Assignment), FR-202 (Workflow), FR-204 (Resolution), FR-206 (Closure), FR-301 (Dashboard), FR-302 (Auditing)

The service_requests table refers to the users table (many-to-one, via requester_id, assigned_staff_id, assigned_by) and the request_categories table (many-to-one, via category_id). It is referred to by the request_attachments, request_assignment_history, request_comments and request_status_history tables (each one-to-many, via request_id).

Columns

- request_id: A primary key used to uniquely identify each service request.
- request_number: A unique, human-readable reference number for the request.
- requester_id: A foreign key identifying the requester who submitted the request, in the users table.
- category_id: A foreign key identifying the category of the request, in the request_categories table. The category determines which department handles the request.
- title: A short title for the request.
- description: A full description of the request.
- location: Where the issue is located. Optional.
- status: The current status of the request. Can be one of: SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED, REJECTED. Defaults to SUBMITTED.
- priority: The priority of the request. Can be one of: LOW, MEDIUM, HIGH, URGENT. Defaults to MEDIUM.
- assigned_staff_id: A foreign key identifying the staff member currently assigned to the request, in the users table. NULL until the request is assigned. Earlier assignments are kept in request_assignment_history.
- assigned_at: When the current assignment was made.
- assigned_by: A foreign key identifying the manager or staff member who made the current assignment, in the users table.
- created_at: When the request was submitted.
- updated_at: When the row was most recently updated.
- expected_completion_at: When the request is expected to be completed, used to monitor how long a request should take.
- resolved_at: When the request was resolved.
- closed_at: When the request was closed.
- overdue_flag: Indicates whether the request is overdue.
- external_requester_id: A foreign key identifying the external citizen (for operator-logged requests). (Added M3)
- created_by: A foreign key identifying the operator who captured the request (UUID). (Added M3)

Indexes and constraints

- Primary key on request_id.
- Unique on request_number.
- Index on requester_id, used to find requests by requester.
- Index on assigned_staff_id, used to find staff workload.
- Index on category_id, used for departmental filtering.
- Index on status, used for dashboard filtering.
- Index on created_at, used for date-based queries.
- Index on expected_completion_at, used to monitor expected completion times.

- Index on overdue_flag, used to identify overdue requests.
- Check constraint: service_requests_exactly_one_requester (ensures either requester_id OR external_requester_id is populated). (Added M3)

6.1.6 The request_assignment_history Table

The request_assignment_history table maintains a complete audit trail of assignment changes, recording who assigned a request to whom, and when. This supports accountability and management reporting.

This table is used only to record when someone is removed from, or re-assigned on, a service request. The active assignment is held on the service_requests table.

Satisfies FR-302 (Auditing) and FR-303 (Assignment & Reassignment).

The request_assignment_history table refers to the service_requests table (many-to-one, via request_id) and the users table (many-to-one, via assigned_to_staff_id and assigned_by_id). It is not referred to by any other table.

Columns

- assignment_history_id: A primary key used to uniquely identify each assignment history record.
- request_id: A foreign key identifying the service request the record belongs to. Deleting the request deletes its assignment history.
- assigned_to_staff_id: A foreign key identifying the staff member who was assigned, in the users table. Optional.
- assigned_by_id: A foreign key identifying the user who made the assignment, in the users table.
- assigned_at: When the assignment was made.
- removed_at: When the staff member was removed from, or replaced on, the request. NULL while the assignment is current.

Indexes and constraints

- Primary key on assignment_history_id.
- Index on request_id, used to retrieve the assignment history of a specific request.
- Index on assigned_at, used for timeline-based queries.

6.1.7 The request_attachments Table

The request_attachments table stores references to files uploaded against service requests, manages file metadata, and tracks who uploaded each file. Files themselves are not stored in the database. Only the location of the file and its metadata are kept, which is recommended above directly storing images in the database (Douslimi, 2021). The problem with storing images directly in the database is that it does not scale efficiently leading to performance bottlenecks, it also requires more backend server performance (Douslimi, 2021). Since we want to limit the cost as much as possible, we stick to only storing a reference to where it will be stored. We have not yet decided where exactly the images will be stored but we are considering a cloud service such as Google Drive or One Drive, other solutions will be explored in M3, and in M3 the final decision will be made as to where we will store the actual images.

UPDATE M3: Cloudflare R2 / S#-compatible Object Storage used via @aws-sdk/client-s3 and @aws-sdk/s3-request-presigner. Only the storage key path is stored in PostgreSQL (requests/:id/:uuid.:ext), while images are served via secure, time-limited presigned URLs.

Satisfies FR-106 (Attachments).

The request_attachments table refers to the service_requests table (many-to-one, via request_id) and the users table (many-to-one, via uploaded_by). It is not referred to by any other table.

Columns

- attachment_id: A primary key used to uniquely identify each attachment.
- request_id: A foreign key identifying the service request the file was uploaded to. Deleting the request deletes its attachment records.
- file_name: The original name of the uploaded file.
- file_path: The location where the file is stored.
- file_size_bytes: The size of the file, in bytes.
- file_extension_type: The file extension or type, such as pdf or png.
- uploaded_by: A foreign key identifying the user who uploaded the file, in the users table.
- uploaded_at: When the file was uploaded.

Indexes and constraints

- Primary key on attachment_id.
- Index on request_id, used to find all attachments for a request.
- Index on uploaded_by, used for user-specific upload tracking and storage quota monitoring.

6.1.8 The request_comments Table

The request_comments table records notes, status updates, resolution details and communications for a request, providing an audit trail of actions and decisions.

Satisfies FR-204 (Resolution Information) and FR-302 (Auditing).

The request_comments table refers to the service_requests table (many-to-one, via request_id) and the users table (many-to-one, via commented_by). It is not referred to by any other table.

Columns

- comment_id: A primary key used to uniquely identify each comment.
- request_id: A foreign key identifying the service request the comment belongs to. Deleting the request deletes its comments.
- commented_by: A foreign key identifying the user who wrote the comment, in the users table.
- comment_type: The kind of comment. Can be one of: ACTION, RESOLUTION, NOTE, UPDATE. Defaults to NOTE.
- content: The text of the comment.
- created_at: When the comment was written.
- updated_at: When the comment was most recently edited.
- is_deleted: Indicates whether the comment has been deleted. Comments are flagged rather than removed, so that the audit trail is preserved. Defaults to FALSE.

Indexes and constraints

- Primary key on comment_id.
- Index on request_id, used to retrieve the comments of a request.
- Index on commented_by, used to filter comments by user.
- Index on created_at, used to retrieve comments in chronological order.

6.1.9 The request_categories Table

The request_categories table defines the types of service request and routes each type to the appropriate department. It also holds the estimated time target for resolving a request of that type, and categorises requests for reporting and analysis.

Satisfies FR-101 (Categorization), FR-207 (Department-Relevant Access) and FR-304 (Request Analysis).

The request_categories table refers to the departments table (many-to-one, via department_id) and is referred to by the service_requests table (one-to-many, via category_id).

Columns

- category_id: A primary key used to uniquely identify each category.
- name: The name of the category. Must be unique.
- description: A description of the category.
- department_id: A foreign key identifying the department responsible for requests in this category, in the departments table. A department can manage one or more categories.
- target_resolution_time: The estimated time within which a request of this category should be resolved.
- is_active: Indicates whether the category is available for new requests. Setting this to FALSE serves as an alternative to deleting a category.
- display_order: The position in which the category is listed. Defaults to 0.
- created_at: When the row was created.

Indexes and constraints

- Primary key on category_id.
- Unique on name.
- No secondary indexes are defined in the current script.

6.1.10 The request_status_history Table

The request_status_history table maintains a complete audit trail of request status changes, recording when each transition occurred and who made it.

Satisfies FR-302 (Auditing).

The request_status_history table refers to the service_requests table (many-to-one, via request_id) and the users table (many-to-one, via changed_by). It is not referred to by any other table.

Columns

- status_history_id: A primary key used to uniquely identify each status change.
- request_id: A foreign key identifying the service request whose status changed. Deleting the request deletes its status history.
- old_status: The status before the change. Can be one of: SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED, REJECTED.
- new_status: The status after the change. Can be one of the same values as old_status.
- changed_by: A foreign key identifying the user who changed the status, in the users table.
- changed_at: When the status was changed.

Indexes and constraints

- Primary key on status_history_id.
- Index on request_id, used to retrieve the status progression of a request.
- Index on changed_at, used for timeline queries.

6.1.11 The external_requesters Table (Added M3 – Documented in CR-001)

The external_requesters table stores contact details for citizens who do not have a CivicConnect account but who log service requests through Operators (via telephone, email, WhatsApp or in-person channels). It allows external requesters to be linked to the requests captured on their behalf, and supports tracking of their preferred contact method for follow-up communication.

Satisfies FR-402 (External Requester Details) and FR-401 (Operator Submission).

The external_requesters table does not refer to any other table. It is referred to by the service_requests table (one-to-many, via external_requester_id), since a single external requester may be linked to more than one captured request over time.

Columns

- external_requester_id: A primary key used to uniquely identify each external requester.
- first_name: The external requester's first name.
- last_name: The external requester's last name.
- email: The external requester's email address. Optional, but at least one contact method must be provided.
- phone: The external requester's telephone number. Optional, but at least one contact method must be provided.
- preferred_contact_method: The contact method the external requester prefers to be reached by (for example, EMAIL or PHONE).
- created_at: When the external requester record was created.

Indexes and constraints

- Primary key on external_requester_id.

6.1.12 The request_contact_access_requests Table (Added in M3 – Documented in CR-002)

The request_contact_access_requests table tracks formal staff requests to view the contact information of a requester (internal or external). It supports the management-controlled approval workflow required by FR-305, ensuring that staff can only access requester contact details after the relevant manager has explicitly reviewed and approved the request. It also preserves an auditable record of who requested access, why, and who reviewed it.

Satisfies FR-305 (Contact Information Authorization) and FR-302 (Auditing).

The request_contact_access_requests table refers to the service_requests table (many-to-one, via request_id), the users table (many-to-one, via requested_by and reviewed_by) and the departments table (many-to-one, via department_id). It is not referred to by any other table.

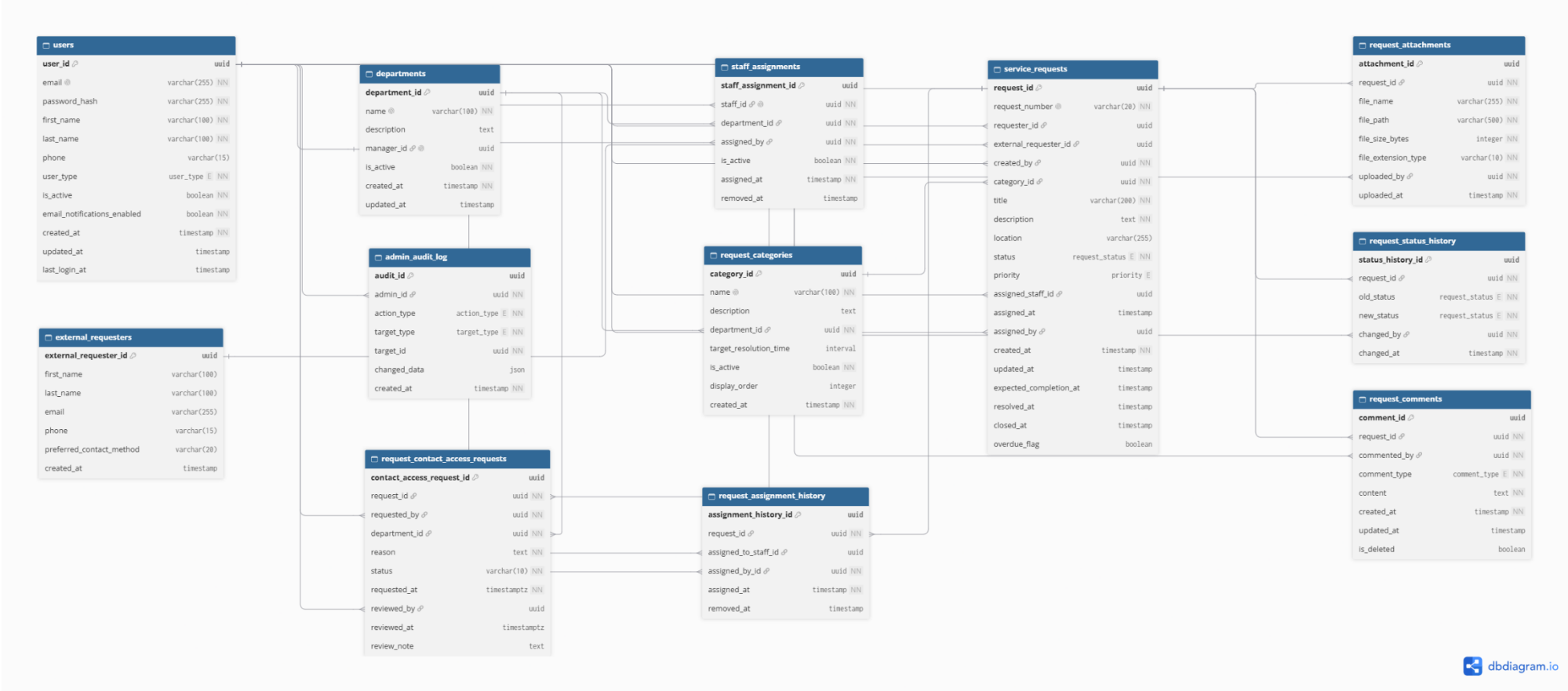
Columns

- contact_access_request_id: A primary key used to uniquely identify each contact access request.
- request_id: A foreign key identifying the service request the access request relates to. Deleting the request deletes its contact access requests.
- requested_by: A foreign key identifying the staff member who requested access to the requester's contact information, in the users table.
- department_id: A foreign key identifying the department the requesting staff member belongs to, in the departments table.
- reason: The reason the staff member requires access to the requester's contact information. Must not be blank.
- status: The current state of the access request. Can be one of: PENDING, APPROVED, REJECTED, CANCELLED. Defaults to PENDING.
- requested_at: When the access request was submitted.
- reviewed_by: A foreign key identifying the manager who reviewed the access request, in the users table. NULL while the request is still pending.
- reviewed_at: When the access request was reviewed. NULL while the request is still pending.
- review_note: An optional note recorded by the reviewing manager, explaining the decision.

Indexes and constraints

- Primary key on contact_access_request_id.
- Index on (department_id, status, requested_at), used to retrieve pending and reviewed access requests per department for the management dashboard.
- Unique index on (request_id, requested_by), used to prevent a staff member from creating more than one pending access request for the same service request.
- Index on (requested_by, requested_at), used to retrieve the access request history of a specific staff member.
- Check constraint: contact_access_reason_not_blank (ensures reason is not empty or whitespace only).
- Check constraint: contact_access_valid_status (ensures status is one of PENDING, APPROVED, REJECTED or CANCELLED).
- Check constraint: contact_access_review_time (ensures reviewed_at is NULL or is not earlier than requested_at).
- Check constraint: contact_access_review_state (ensures reviewed_by and reviewed_at are only set when status is APPROVED or REJECTED, and that status transitions remain consistent).

6.2 Data Model / ERD / Schema



The diagram is also viewable online at dbdiagram.io here: https://dbdiagram.io/d/CivicConnect_BugBusters-6ab7b2250f25a52d0113c01b

6.3 Data Ownership and Lifecycle

Entity	Created / owned by	Read / updated by	Lifecycle / retention rule
users	Requester self-registration. Staff registered via manger / admin. Manger profile created via admin user.	Own profile; Management/Admin for authorised administration.	Accounts are not hard-deleted by default. “is_active” is set to false when access is removed. This preserves request, assignment and audit history.
admin_audit_log	System-generated for administrative actions.	Admin/Management for audit and accountability.	Append-only. Stores action_type, target_type, target_id, changed_data and timestamp. Used for administrative user, department, category and assignment changes.
departments	Management/Admin.	Management, Staff, Admin.	Departments are deactivated with is_active = false rather than deleted where they are linked to requests, categories or staff assignments. manager_id enforces one manager per department.
staff_assigments	Management/Admin. Staff can also auto assign but managers can reassign after that, manager assignment is final.	Management, Admin, relevant Staff.	Tracks which department a staff member belongs to. Active assignment is controlled by is_active and removed_at. Historical membership remains for accountability.
service_requests	Requester creates the request.	Requester sees own request; assigned Staff and Management see permitted details according to role/department.	Central operational record. Status lifecycle is SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED and lastly CLOSED, with REJECTED as a exception state. Requests should not be hard-deleted in normal operation because they support tracking, reporting and audit.
request_assignment_history	System-generated on controlled status transition.	Management/Staff for accountability.	Append-only. Records who assigned whom and when. Supports FR-303 and FR-302.
request_attachments	Requester uploads; system stores metadata and file path.	Requester, assigned Staff/Management where authorised.	Attached images belong to the parent request. They are retained with the request.
request_comments	Staff add comments; Important to note requesters cannot add comments.	Requester may see permitted feedback; Staff/Management see relevant comments.	Comments are retained for audit and resolution evidence. “is_deleted” supports soft deletion rather than physical removal.
request_categories	Management/Admin.	Requesters use categories when submitting; Staff/Management use them for routing and reporting.	Categories may be deactivated with “is_active = false”. Existing requests retain their category reference.
request_status_history	System-generated for administrative actions.	Staff/Management for audit and lifecycle visibility.	Append-only and immutable. Every status change must create a history row. It must not be silently overwritten.

6.4 Integrity, Consistency and Concurrency

CivicConnect must preserve the correctness of request, assignment, status, comment and audit data even when multiple users interact with the same request at the same time. The integrity and concurrency approach therefore focuses on database constraints, application-managed transactions, validation at the correct layer, and optimistic concurrency control for the request workflow.

Integrity constraints

The following integrity rules are enforced across the data model:

- **Entity integrity:** Every table has a primary key. Core entities such as users, service_requests, request_categories, departments and staff_assignments cannot contain NULL primary keys.
- **Referential integrity:** Foreign keys enforce relationships such as service_requests.requester_id relates to users.user_id, service_requests.category_id relates to request_categories.category_id etc.
- **Domain integrity:** Controlled values are enforced with CHECK constraints or equivalent application validation. For example, service_requests.status must be one of SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED or REJECTED.
- **Business integrity:** A request cannot move directly from SUBMITTED to CLOSED without passing through the permitted workflow states. A staff member may only update a request assigned to them and within their department. Requester contact information is only released when contact_information_approved = true and contact_approved_by is set.
- **Uniqueness:** users.email is unique. A partial unique index on staff_assignments(staff_id) WHERE is_active = true enforces that a staff member has only one active department assignment at a time, while still preserving historical assignment records.
- **Soft deletion over hard deletion:** Users, departments and requests are deactivated rather than physically deleted where they are linked to audit, assignment or request history. This preserves accountability and referential integrity.

Transactional consistency

The most important consistency boundary is the service request status transition. When an authorised staff member changes a request from one status to another, the following operations must be treated as one atomic transaction:

1. Validate the requested transition against the workflow rules.
2. Update service_requests.status, updated_at.
3. Insert a record into request_status_history.
4. Insert a resolution or progress comment into request_comments where required.
5. Insert an assignment history record where assignment or reassignment occurs.
6. Commit the transaction.

If any step fails, the entire transaction is rolled back. This prevents partial states such as a request being marked RESOLVED without a resolution comment or status history record.

Email notification is deliberately outside this transaction. The status change is the primary business operation; notification is a secondary concern. If SMTP delivery fails, the status transition is not rolled back. The failure is logged, and the notification may be retried or marked as failed for audit purposes.

Concurrency control

Concurrency control is the set of mechanisms used to manage simultaneous access to shared data so that multiple users or processes do not interfere with each other and leave the data in an inconsistent state (Databricks, 2026). It is a core part of transaction management and database isolation (Databricks, 2026). Without it, operations that happen at the same time can produce incorrect results, such as lost updates, dirty reads, or inconsistent request states (Databricks, 2026).

Concurrency control matters for CivicConnect because multiple stakeholders can act on the same service request at the same time. Staff can update status, Management can reassign or approve contact information, and Admins can manage users and departments. If two users change the same request without coordination, one change can overwrite the other.

For example:

- A manager reassigns a request to a different staff member while the original staff member is updating the status. The assignment and status could become inconsistent.

There are two main approaches to concurrency control. Pessimistic concurrency control locks data before it is changed, for example using SELECT FOR UPDATE, so other transactions must wait (Databricks, 2026). This is safe but can reduce performance and cause waiting or deadlocks (Databricks, 2026). Optimistic concurrency control does not lock rows in advance (Databricks, 2026). Instead, it checks whether the data changed since it was read, typically using a version column. An update only succeeds if the version still matches. If zero rows are affected, the system detects a conflict and can ask the user to refresh or retry. Databases also provide isolation levels such as Read Committed, Repeatable Read, and Serializable, which define how strictly concurrent transactions are separated (Databricks, 2026).

We have not yet determined which concurrency control approach will be used for CivicConnect. Optimistic concurrency control is a candidate because it avoids long locks and fits the low-to-medium volume expected for the platform. Pessimistic locking may be considered for critical sections where conflicts are likely. The decision will be weighed in Milestone 3 after evaluating trade-offs between correctness, performance, complexity, free-tier database capabilities, and team capability. Until then, the baseline assumes that concurrency control is required. The final choice and implementation evidence will be recorded in M3.

M3 Concurrency Implementation: The request submission persistence pathway uses a PostgreSQL transaction and year-specific advisory transaction lock during request-number allocation. This serialises conflicting number-allocation work within the defined lock scope, supporting unique request-reference assignment under concurrent submissions. PT-001 created 90 distinct request references with no observed duplicate or missing persistence records. This is evidence for the exercised path, not proof of all concurrency interleavings. Lock contention is a possible performance contributor but has not been demonstrated as the cause of the observed latency.

6.5 Scalability, Availability, SPOF & Backup/Recovery

CivicConnect is a low-to-medium volume community service request platform built by a three-person team on free-tier or low-cost infrastructure. The operational baseline therefore prioritises simplicity, recoverability and acceptable availability over high-availability enterprise architecture.

Scalability

The selected Node.js/Express application is stateless at the request-handling level. This means it can be restarted or redeployed without losing application state. For the expected initial workload, a single free-tier application instance is sufficient.

Scaling considerations:

- **Database connections:** Node.js is single-threaded, and opening a new PostgreSQL connection per request would exhaust free-tier connection limits. CivicConnect will use a connection pool through node-postgres and the managed provider's connection pooler.
- **Database scaling:** PostgreSQL provides vertical scaling and indexing. The existing indexes on service_requests support dashboard filtering, status queries, assignment lookup and requester history. Further optimisation, such as materialised views, is deferred until real usage data exists.
- **Free-tier limits:** Storage, bandwidth, database size and email sending limits must be monitored. Image upload size limits and attachment storage are open decisions that directly affect scalability and cost.

Availability

CivicConnect targets **best-effort availability** on free-tier infrastructure. No formal uptime SLA is claimed in M2.

Availability measures:

- The Express application exposes a /health endpoint used by CI and later by monitoring.
- The hosting platform can automatically restart failed application instances.
- Deployment is controlled through GitHub Actions and the staging branch, reducing the risk of broken releases reaching production.
- Email notification failure does not block request submission, status updates or resolution. This provides graceful degradation of a secondary feature.

Potential bottlenecks identified

The database bottleneck analysis identifies the points in the persistence layer where CivicConnect is most likely to experience performance decline, resource exhaustion, or unexpected cost as usage grows.

1. Dashboard and Search Query Cost
This bottleneck arises when Management dashboards, request lists, and search or filter operations scan large tables or trigger N + 1 loading of comments and attachments. It is important because slow dashboards directly affect Staff and Management productivity. Mitigation is to rely on the existing indexes on service_requests, prefer joins or batched queries over N + 1 access, and defer materialised views and read replicas until real usage data justifies them.

2. Growth of Append-Only Audit and History Tables
This bottleneck concerns tables such as request_status_history, request_assignment_history, request_comments, and admin_audit_log, which only grow and never shrink. It is important because continuous growth can slow history queries over time and consume free tier storage, threatening both performance and cost constraints. Mitigation is that normal indexing is sufficient at current volume, while partitioning, archiving, or retention rules for non critical audit data are deferred until usage evidence shows they are needed.

3. Free-Tier Resource limits
This bottleneck refers to the limits on database size, connections, bandwidth, storage, and backup retention imposed by free tier providers. It is important because the system may work in development but hit a paywall or degrade in production, which would violate the low cost constraint and risk unexpected redesign. Mitigation is to monitor database size, connection usage, and storage, keep large binaries out of PostgreSQL, store images as metadata plus external file references, finalise image size

limits and attachment storage before production, and keep backups within provider limits while testing restore procedures.

4. Attachment Metadata Coupling
This bottleneck involves request_attachments storing metadata and file paths, where storing files locally on the application instance makes them non-durable and lost on redeploy, while external storage adds another dependency. It is important because lost attachments undermine the evidence needed for request resolution and auditability. Mitigation is to store only metadata and a stable path or reference in PostgreSQL, store actual images in durable external object storage, and confirm the final storage decision in Milestone 3.

Backup and Recovery

Recovery Time objective:
The target we are going to aim for is 8 hours.

The system is important for operational continuity but is not life-critical. An 8-hour restoration window is acceptable for the initial baseline.

Recovery Point Objective:
The target we are going to aim for is 24 hours.

Daily backups mean up to one day of request data could be lost in a catastrophic failure. This is acceptable for the initial low-volume context.

Backup approach:

- CivicConnect will rely on the selected managed PostgreSQL provider's automated daily backups.
- Backups are stored separately from the application instance.
- Database backups cover all relational data: users, departments, staff assignments, service requests, request categories, status history, assignment history, comments, attachments metadata and admin audit logs.
- Attachment backup depends on the final image storage decision. If images are stored in external object storage, that provider's retention and backup settings must be configured.

Point-in-time recovery:

Still to be determined. Based on database hosting and deployment, it will be considered if we will make use of point in time recovery, for now we defer this decision.

Recovery procedure:

1. Detect database failure or data corruption.
2. Notify the team and pause production writes if possible.
3. Restore the latest automated backup to a new database instance.
4. Update the application connection string through environment variables or secrets.
5. Run smoke tests against /health, login, request submission and status transition.
6. Resume service and record the incident in the audit or engineering log.

Testing requirement:

A restore test must be performed at least once before production release. A backup that has never been restored is not considered verified evidence. This aligns with the assignment 3 production-readiness concern that backup and recovery procedures must be tested, not merely documented.

7. Technology Stack Baseline

7.1 Technology Requirements and Selection Criteria

CivicConnet's technology stack was not evaluated on industry trends but on survivability criteria for a 3-person team operating on a short timeline. The main selection drivers include:

- Cognitive load (human focus) and context switching. The architecture must minimize the number of disparate languages, frameworks, and build pipelines the team must maintain.
- Operational cost (NFR-301). The stack must have a memory and compute usage small enough to avoid penalties and out-of-memory crashes on standard PaaS free tiers which is usually capped at 512MB to 1GB RAM per instance or service.
- Concurrency, the persistence layer must support transaction isolation and Optimistic Concurrency Control (OCC) to mitigate, "lost update" anomalies during a case where simultaneous ticket assignments/updates exist.
- Ecosystem maturity, tools should offer strong out-the-box CI/CD support without requiring shell scripting (the practice of writing shell scripts, which are text files containing a sequence of commands designed to be executed by a Unix shell a command-line interpreter).

7.2 Alternatives considered

- Decoupled SPA (React/ Next.js) + API Backend: Rejected, while this is standard for enterprise systems, an SPA introduces distributed state management, configuration overhead and require maintaining two separate CI/CD build pipelines. Given our schedule, this doubles deployment complexity and is not needed for what is essentially a forms and dashboard application.
- Java Servlets / Spring Boot: Rejected, the JVM's baseline memory and garbage collection overhead makes it a problem to free-tier cloud environments. Spring Boots heavy abstraction layers would also slow down initial scaffolding compared to a lighter runtime environment, thus making our milestone schedule tighter.

7.3 Selected Technology Stack

The team has baselined a JavaScript-centric MVC Web-Application to maximize developer power and minimize the integration boundaries.

- Runtime and API framework: Node.js (v24) with Express.js. Node's asynchronous, event driven I/O model is suited for CivicConnect, which is heavily I/O bound, database queries and SMTP calls, rather than CPU processing bound.
- Frontend Presentation: We chose server-side rendering (SSR) via EJS (Embedded JavaScript) coupled with Tailwind CSS. This removes client-side state synchronization issues and delivers fast initial page loads. Tailwind also eases development by having the advantage of allowing us to adapt the application to different screen sizes, which satisfies NFR-401 (Compatibility).
- Database and persistence: PostgreSQL. This is chosen for its strict relational integrity, it's support for row-level locking (concurrency control mechanism that allows multiple transactions to access and modify different rows within the same table) and advanced JSONB indexing which provides unstructured audit payloads are required later.
- Database access layer: Node-Postgres, using transaction blocks to guarantee atomicity during request state transitions.
- Integration: We chose Nodemailer for SMTP, which allows environment switching between Ethereal for testing and then Gmail for production.

- Quality Gates: We chose Jest for unit/integration testing and ESLint for static analysis, this is directly integrated into GitHub Actions.
- Production Libraries: Added in M3 @aws-sdk/client-s3 and @aws-sdk/s3-request-presigner for durable object storage, bcrypt for password hashing, multer & sharp for image parsing/validation, and express-session for secure RBAC session handling.

7.4 Versions and Compatibility Assumptions

- Runtime parity: Node.js v24 is pinned via .nvmrc (The .nvmrc file is a configuration file used by Node Version Manager (NVM) to specify the required Node.js version for a project. It is a simple text file placed in the project's root directory containing a version string, such as v18.16.1) The GitHub Actions CI pipeline is configured to read this file to prevent the, "works on my machine" deployment anomalies.
- Database connection pooling. Because Node.js is single-threaded, opening a new TCP connection to PostgreSQL per request will drain free-tier database connection limits. We assume the selected hosting provider Supabase/Neon will provide a connection pooler (Supabase=Supavisa; Neon=PgBouncer both built-in) at the infrastructure level.

7.5 Technology Trade-offs

Advantage: Unifying the stack under JavaScript (Node/Express/EJS) allows all team members to comfortably debug across the request lifecycle. Server-side rendering reduces the areas for security vulnerabilities because the server controls the DOM generation.

Trade-off: EJS relies on full-page reloads, which means a less dynamic UX compared to React/Next.js. We are trading client-side polish for simplicity, thereby prioritizing backend stability and security.

7.6 Evidence and traceability

- ADR reference: ADR-004 (Technology Stack Selection – MVC Node/PostgreSQL).
- Implementation evidence: .nvmrc, package.json, ESLint configuration and the express application passing the Jest CI health check on the testing branch.

8. Initial Design Decisions

8.1 Design decision 1: Decoupling domain logic from external side-effects

Problem / Context:

When a staff member resolves a ticket (FR202 and FR206) the system performs a primary domain operation (updates database). However, it must also trigger side-effects: writing to the audit log (FR-302) and dispatching an email (FR-104). Then tying a network operation (SMTP) directly to the database transaction creates performance risks; if the email server times out, the database transactions remain open thereby holding locks and degrades the system throughput.

Alternatives considered:

Assignment 2 evaluated Direct Service Orchestration synchronous, procedural execution against an Observer / Event-Based Approach asynchronous decoupling.

Selected Pattern: Observer / Event-Based Approach

The RequestWorkflowService executes the database transaction. When there is a succesful commit, then it fires an in-memory event for example, RequestStatusChanged. The NotificationService and AuditLogService act as decoupled listeners that react to this event asynchronously in the background.

Why it fits CivicConnect:

“Eventual consistency” is enforced for secondary actions and aligns with the API integration decision which means that SMTP failure must not roll back the main database state. The user gets an HTTP 200 OK response as soon as the database updates while the slow SMTP dispatch happens safely outside the request thread.

Affected components:

- RequestWorkflowService, event publiser.
- NotificationService, subscriber/ SMTP Handler
- AuditLogService, subscriber

Trade-off / Complexity:

In-memory event emitters do not survive crashes. If the Node.js process dies then milliseconds after the database commits but before the email is sent, then the email is lost. Implementing a persistent message queue like RabbitMQ would solve this but it violates our constraint to avoid operational complexity. We accept this risk as it is appropriate for a not so critical community portal.

Implementation evidence and traceability:

- ADR Reference: ADR-005 (Event-driven side-effect management).
- Evidence: src/shared/events/eventBus.js (AuditEventBus extending Node's EventEmitter) is fully implemented, decoupling RequestStatusChanged events from SMTP dispatch.

8.2 Design Decision 2: Centralized state mutation Governance

Problem / Context:

The service request lifecycle: SUBMITTED > ASSIGNED > IN_PROGRESS > RESOLVED > CLOSED, involves authorization rules. By placing if / else statements inside individual UI controllers creates logic duplication, thus making it likely that a security vulnerability exists when modifying state transitions in the future.

Alternatives considered:

Assignment 2 compared the State Pattern against a procedural Centralised Workflow / Transition Service.

Selected Pattern: Centralised Workflow / Transition Service

We implement a finite state machine configuration within a singular RequestWorkFlowService. All request mutation attempts are routed through this boundary layer, which validates the payload against an allowed transitions map before generating the SQL update query.

Why it fits CivicConnect:

The State Pattern requires creating classes for every single state for example, OpenState, ResolvedState. This brings unneeded object allocation overhead and boilerplate for a static 5 step lifecycle. A centralised service provides the same security and consistency through simple guard clauses, ultimately optimizing for maintainability.

Affected components:

- RequestWorkflowServices, boundary layer and rules engine.
- RequestController, responsible for HTTP parsing and delegation.

Trade-off / Complexity:

The service class can become dense. We will mitigate this by extracting the validation rules into utility functions thus ensuring the central service acts only as an orchestrator and not a monolithic script.

Implementation evidence and traceability:

- ADR Reference: ADR-006 (Centralised Workflow FSM).
- Evidence: workflow.service.js and requests.model.js act as the finite state machine boundary enforcing ALLOWED_TRANSITIONS.

M3 Verification Note: DEC-012 / ADR-005 decouples secondary email and audit side-effects from the core database operation; in-memory event delivery nevertheless retains durability risk (TD-06). DEC-013 / ADR-006 centralises request workflow transitions, with evidence in request workflow tests referenced in Section 13.8. These controls support maintainability and integrity, but notification availability on Render remains limited by Issue #124.

9. Initial API / Integration Decisions

9.1 Interface/Integration background and context

For CivicConnect we added the requirement that requesters have to option to receive feedback when a request status changes (FR-104; in-scope feature: "Feedback via email push notification" from PED v1). The Request Management module (FR-202, FR-206) is the producer of status-change, while an email notification component is the consumer responsible for delivering the message to the requester.

During our Assignment 2 research, three integration approaches were identified and evaluated: HTTP/REST API, asynchronous event/message (broker-based), and SMTP. The asynchronous broker approach was rejected due to the operational complexity for a non-critical messaging system (it is intended for applications where message delivery must be ensured, such as financial transactions or healthcare communications). The final choice between REST-based API and SMTP was deferred to Milestone 2 pending cost and scalability analysis.

For M2 we have now reached a meaningful implementation decision. The decision has been finalised: SMTP via Nodemailer is the chosen choice. As it is a Node.js library that handles SMTP connections directly (fits our tech stack). This is also a stable library that is widely used (Patel, 2026). Further justification follows below:

9.2 Responsibilities and Information Exchange

The Request Management component is responsible for detecting a controlled status transition (e.g., "In Progress" to "Resolved") and triggering the notification. It does not need to know how the email is ultimately delivered.

The Notification component (implemented as a Nodemailer service module) is responsible for:

- Resolving the requester's email address from the Requester ID.
- Composing the email message from the status-change data.
- Connecting to the configured SMTP server and dispatching the message.
- Logging the outcome (success or failure) for auditing and troubleshooting.

The information exchanged is derived from assignment 2 includes:

- Requester ID (used to look up the email address).
- Previous status and new status.
- Timestamp of the status change.
- Staff member ID (for name and department lookup).
- Optional comment added by the staff member (FR-204).

Structure of the email notification :

‘Good day [requester name],

This email is with regards to [request name] with request id of [request id]. The status of the request has changed from status [prev status] to [new status]. Staff member ` [staff member name] ` had this to say:

[comment by the staff member]

Kind regards,

CivicConnect, [department name] department’

9.3 Interaction Mechanism

The selected mechanism is direct SMTP integration using Nodemailer. This approach was chosen over a managed REST-based email API for the following project-specific reasons:

Factor	**SMTP + Nodemailer**	**REST API (e.g SendGrid)**
Vendor lock in	Minimal, only the SMTP credentials need to change if the provider is switched.	Higher, application depends on the provider's API contract and version
Cost	Nodemailer is free and open-source; the only cost is the SMTP service (many free/low-cost options exist, e.g., Gmail for low volume).	Free tiers exist but scale into per-email pricing; cost increases with volume
Operational Complexity	Moderate, Nodemailer handles connection, authentication, and message dispatch.	Moderate, requires managing API keys, rate limits, and provider-specific error codes.
Future sustainability	High, switching SMTP providers requires only configuration changes, not code changes.	Lower, switching providers may require code changes to accommodate different API contracts.

Testing and production configuration:

- **Development/Testing:** Ethereal Email, a free fake SMTP service designed for Nodemailer. Nodemailer can automatically generate test credentials via nodemailer.createTestAccount(), and sent messages are captured in a web interface without being delivered to real recipients. The preview URL from nodemailer.getTestMessageUrl(info) allows us as developers to inspect the message format and content.

- **Production:** Gmail SMTP (via service: "gmail" preset) is the initial production target for low-volume sending. An App Password (16-character, generated with 2-Step Verification enabled) is used for authentication. The service: "gmail" shortcut automatically configures the SMTP host and port settings. A centralised transport factory will switch between Ethereal and Gmail based on the NODE_ENV environment variable, so the application code remains identical across environments.

9.4 Validation, Errors and Security Boundary

This section outlines the planned approach for validation, error handling, and security for the SMTP-based email notification integration. Implementation of these controls is scheduled for Milestone 3.

Validation

The notification service will validate that the requester's email address has been successfully verified before attempting to send. This prevents unnecessary SMTP connection attempts when the recipient address is missing or malformed. Address validation is a recognised security control for email libraries, as user-supplied addresses can be an injection surface if not properly handled (Patel, 2026).

Error/Failure Behaviour

If the SMTP connection fails or the server returns an error, the notification service will log the failure and mark the notification as failed in the audit trail.Notification failure will not roll back the status transition.

For transient SMTP failures (e.g., temporary network issues), a single retry may be attempted. Nodemailer's direct transport module supports automatic retry with a configurable delay for messages that cannot be sent immediately, handling errors and greylisting (Nodemailer, 2025). Duplicate notifications will be avoided by tracking a notification ID for each status-change event. This addresses the concern about "spamming the user with emails unnecessarily".

Junk mail placement is a failure mode even when SMTP reports successful delivery. To reduce this, we plan to possibly implement SPF, DKIM, and DMARC authentication, keep email content simple and factual, and encourage requesters to whitelist the sender address (This decision is not finialzied, and will still be reviewed in M3).

Security Boundary

SMTP credentials (username and App Password) will never be committed to the repository. They will be injected via environment variables or CI secrets. This is the recommended practice for Nodemailer configurations, where sensitive options such as passwords can be overwritten using environment variables (Nodemailer, 2025)

Connections to Gmail SMTP will use the recommended secure ports and TLS/STARTTLS settings as per Nodemailer's official documentation, to protect credentials and message content in transit.

Email content must respect the requester's and staff member's privacy (NFR-101, FR-305). The message template will include only the information necessary for the notification and will not expose internal identifiers or sensitive data beyond what the requester is entitled to see.

Change/Version Implications

Nodemailer's SMTP transport documentation notes that with SMTP, switching providers is a matter of changing the configuration object or connection URL, whereas API-based sending creates vendor lock-in (Nodemailer, 2025).

10. Deployment Compatibility

10.1 Deployment Direction and Environment Assumptions

CivicConnect is being engineered as a low-to-medium volume, cloud-hosted web application for a community-focused organisation. The deployment approach prioritises simplicity, recoverability, acceptable availability and low operational cost rather than enterprise-scale redundancy or distributed

infrastructure. This is consistent with the current PED, which assumes that a single application instance is sufficient for the expected initial workload.

The current direction is to use a managed Platform-as-a-Service (PaaS) environment for the eventual production deployment. Railway and Render remain deployment candidates, but the final provider has not yet been selected. The choice will depend on factors such as runtime support, PostgreSQL compatibility, resource limits, configuration and secrets support, operational simplicity and cost.

Project Conditions

Contextual factor	Current assumption
Scale	CivicConnect is expected to operate at a low-to-medium workload. One application instance is considered sufficient initially.
Team	The system is being developed by a three-person team, so the deployment infrastructure must remain practical to configure, maintain and troubleshoot.
Schedule	Deployment decisions must support the limited milestone schedule without introducing unnecessary infrastructure complexity.
Cost	Free or sufficiently low-cost hosting and supporting services are preferred for the project.
Availability	Best-effort availability is acceptable at this stage. No enterprise-level uptime SLA is claimed.
Scalability	Future scaling should remain possible, but additional infrastructure will only be introduced when supported by actual evidence or usage requirements.

These assumptions align with ASR-05: Cost-Effective Deployability and Resource Efficiency, which requires CivicConnect to remain executable in the team's development environment and deployable using a free or sufficiently low-cost environment appropriate to the project. The ASR also requires hosting, database, storage and supporting-service limitations to be identified before final commitment and requires the deployment approach to remain practical for the three-person team to configure, test and maintain.

Runtime Environment

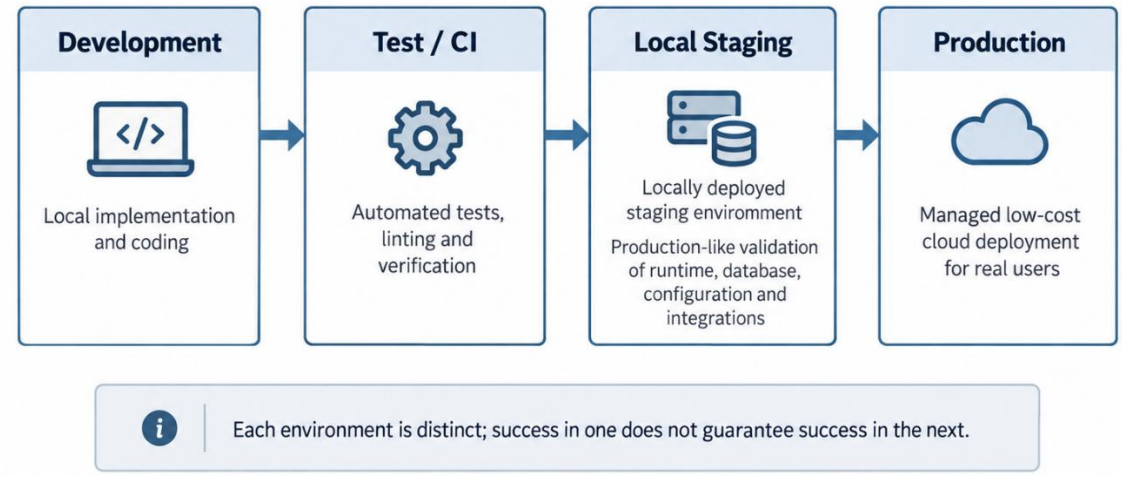
The runtime environment follows the technology baseline already established in the PED:

- Node.js v24 with Express.js for the application runtime.
- EJS and Tailwind CSS for server-side presentation.
- PostgreSQL for persistent data, accessed through node-postgres.
- Nodemailer/SMTP for email notifications.
- GitHub Actions, Jest and ESLint for CI and verification rather than production runtime functionality.

The application is assumed to remain stateless at the request-handling level, while persistent business data is stored outside the application process. This allows the application to be restarted or redeployed without losing persistent system data.

Environment Progression

CivicConnect will distinguish between the following environments:



Before moving towards production deployment, CivicConnect will first be deployed locally in a dedicated staging environment. This staging environment will be used to run the application as an integrated system under conditions that are as close as reasonably possible to the intended production configuration. This will allow the team to verify areas such as runtime compatibility, database connectivity, environment-specific configuration and supporting integrations before committing to a final cloud deployment environment.

Using a local staging environment also allows the team to identify deployment-related problems without prematurely committing to a production hosting provider. This follows the Master Project Brief requirement to distinguish between development, test, staging and production environments because successful operation in one environment does not automatically prove that the system will operate correctly in another.

The exact production host, managed database provider, attachment-storage solution and automated deployment process remain deliberately open until sufficient evidence is available. The current PED already records the Render-versus-Railway hosting decision and automated deployment as deferred decisions.

10.2 Compatibility of Architecture, Data and Technology Decisions

The deployment direction established in Section 10.1 is compatible with the architecture, persistence and technology decisions already baselined for CivicConnect. The selected solution can initially operate as a single application instance in the local staging environment and later move to a managed cloud production environment without requiring a fundamental redesign. The table below shows how the main engineering decisions already made in the PED support this deployment direction.

Engineering Area	Current Decision	Deployment Compatibility
Architecture	Modular Monolith	CivicConnect remains a single deployable application while responsibilities are separated into internal modules. This fits the single-instance deployment direction and avoids the extra networking, coordination and operational complexity associated with distributed services.
Application Runtime	Node.js v24 with Express.js	The application can run as a single web process in both the local staging environment and later PaaS production deployment. Pinning Node.js v24 through .nvmrc also supports runtime consistency across environments.
Presentation	EJS with Tailwind CSS	EJS is rendered by the same Node.js application, so no separately deployed frontend application is required. Tailwind CSS remains part of the application presentation layer and introduces no additional runtime service.

Persistence	PostgreSQL	Persistent business data is kept outside the application process and can later be hosted through a managed PostgreSQL service. This allows the application to be restarted or redeployed without losing persistent data.
Database Access	node-postgres with connection pooling	node-postgres fits the selected Node.js runtime and supports transaction handling and controlled PostgreSQL access. Connection pooling also helps reduce the risk of exhausting managed-provider connection limits.
Application State	Stateless request handling	Persistent business state is not tied to the running application instance, which supports restart, redeployment and possible future horizontal scaling.
Email Integration	Nodemailer using SMTP	Email is treated as an external supporting integration rather than a separate CivicConnect deployment unit. SMTP configuration can therefore vary between environments without changing the core application architecture.
Verification / CI	GitHub Actions, Jest and ESLint	Automated verification can occur before changes progress into staging. These tools support the deployment process without adding runtime infrastructure to the deployed application.

The selected Modular Monolith, Node.js/Express runtime, PostgreSQL persistence and supporting integrations are therefore plausible for the deployment direction defined in Section 10.1. The current PED already establishes CivicConnect as a single deployable application, with persistent data separated from the application process and deployment complexity intentionally kept proportionate to the project.

10.3 Deployment Configuration, Secrets, Persistence/State and Networking Implications

The deployment direction established in Sections 10.1 and 10.2 creates several configuration, security, persistence and networking implications that must be considered before CivicConnect progresses from local staging to production. The purpose of this section is to identify those implications at the level currently known in M2 without prematurely finalising provider-specific production configuration.

Area	Current Implication
Runtime Configuration	Node.js v24 is pinned through .nvmrc so that developer, CI and later deployment environments use the same runtime version. Environment-specific behaviour should be controlled through configuration rather than changes to application code.
Environment Variables	Values that differ between development, staging and production must be externalised from the source code. This includes database connection details, session configuration, SMTP configuration and other provider-specific settings.
Secrets	Sensitive values must not be committed to the GitHub repository. The current CI process already injects SESSION_SECRET using GitHub Actions repository secrets. Production secrets will later be supplied using the selected deployment platform's secure environment-variable or secret-management mechanism.
Database Persistence	PostgreSQL remains the persistent store for CivicConnect business data. Persistent data must remain independent of the running Node.js application instance so that restarting or redeploying the application does not remove system records.
Application State	Request handling is intended to remain stateless at application-instance level. If CivicConnect later runs across more than one application instance, session state will need to be externalised so that users are not tied to a particular instance.
Attachment Persistence	Images must not rely on temporary application-instance storage for production because locally stored files may be lost when an application instance is restarted or redeployed. The current PED therefore anticipates durable storage for attachments, while the final storage mechanism remains deferred.
Database Connectivity	The Node.js application requires network connectivity to PostgreSQL. node-postgres connection pooling will be used, together with provider-side pooling where available, to avoid exhausting managed database connection limits.
Client Connectivity	Users will access CivicConnect through the web application. Production access will require a secure public endpoint, while the local staging deployment remains locally hosted for controlled validation.
SMTP Connectivity	The application requires outbound network access to the configured SMTP service. Development/testing can use Ethereal, while Gmail SMTP is currently the initial production direction. SMTP credentials must be supplied through environment configuration rather than source code.
External Service Failure	Network-dependent supporting services must not unnecessarily control core CivicConnect transactions. In particular, an SMTP failure must not roll back a successfully completed request-status change.
Health and Failure Detection	The Express application already provides a /health endpoint, which can be used during staging and later deployment to confirm that the application starts and responds successfully.

These implications establish the deployment requirements currently known in M2. Node.js runtime consistency, externalised configuration, controlled secrets, persistent PostgreSQL storage, stateless request handling, connection pooling and failure isolation are already aligned with the current PED baseline. The exact production values, provider-specific secret management, final attachment-storage mechanism and any additional networking configuration will be finalised only once the relevant deployment providers and services are selected.

10.4 Deployment Compatibility, Cost, Availability and Operational Risks

The deployment products and supporting services currently selected or being considered for CivicConnect were reviewed against the deployment direction established in Section 10.1. The assessment records each product's intended use, compatibility with the existing architecture and technology stack, expected cost for the intended project usage, practical availability limitations and material operational risks. Where a final provider has not yet been selected, current candidates are assessed separately rather than treated as final decisions. EJS and Tailwind CSS are packaged within the application and do not introduce a separate hosted service or recurring deployment cost, so they are not listed separately below.

Product / Service	Intended Use	Compatibility	Cost for Intended Use	Availability / Service Limitations	Material Operational Risk	Related PED Risk
Node.js v24 + Express.js	Runtime and web framework for the single CivicConnect application process.	Compatible with the selected Modular Monolith and PaaS deployment direction. Node.js is suitable for HTTP/database/SMTP I/O, while Express runs in the same deployable application boundary.	US$0 software licence cost. Node.js is free and open source; Express is an open-source Node.js framework (Node.js, 2026; Express.js, 2026).	Availability depends on the hosting platform rather than Node.js/Express themselves. Runtime parity is controlled through the pinned Node.js version.	Environment/runtime mismatch or unsupported provider runtime could cause deployment failure even when the application works locally.	RSK-005; proposed RSK-011
Render - Free Web Service	Candidate cloud host for the Node.js/Express application.	Partially compatible. Supports Node web services and provides 512 MB RAM on the Free plan. However, Free web services block outbound traffic on SMTP ports 25, 465 and 587, which conflicts with the current Gmail SMTP/Nodemailer production direction (Render, 2026-a; Render, 2026-b).	US$0/month on Free. The next 512 MB paid web-service compute plan is US$7/month (Render, 2026-b).	750 Free instance-hours per workspace/month. Free web services spin down after 15 minutes without inbound traffic, cannot use persistent disks, and may be suspended when free instance hours are exhausted (Render, 2026-a).	Cold starts and free-tier limits may reduce responsiveness or availability. Direct Gmail SMTP will not work on Render Free. Local filesystem data is non-durable.	RSK-005; RSK-006; RSK-007; RSK-008
Railway - Free	Alternative cloud host for the Node.js/Express application.	Partially compatible. Supports public HTTP/HTTPS deployment and provides up to 1 vCPU and 0.5 GB RAM per service on Free. SMTP is disabled on Free, Trial and Hobby, so the current Gmail SMTP/Nodemailer production direction cannot operate on those tiers (Railway, 2026-a; Railway, 2026-b).	US$0/month with US$1 of free resource credit/month. Hobby is US$5/month; Pro is US$20/month. SMTP requires Pro or above (Railway, 2026-a; Railway, 2026-c).	Free has one replica, 0.5 GB RAM and 0.5 GB volume storage per service. Free-plan restart policy is limited, and workloads can be stopped if usage limits are reached (Railway, 2026-a; Railway, 2026-d).	The US$1 monthly credit may be insufficient for continuous operation. Retaining direct SMTP would require moving to Pro, increasing cost materially.	RSK-005; RSK-006; RSK-008
PostgreSQL	Primary relational persistence technology for users, requests, workflow, history and audit data.	Fully compatible with the Node.js application through node-postgres and with the relational integrity/transaction requirements already baselined in the PED.	US$0 software licence cost. PostgreSQL is free and open source; hosting cost depends on the selected managed provider (PostgreSQL Global Development Group, 2026).	The database software itself does not provide a hosted availability guarantee; availability, storage, backup and recovery depend on the managed provider selected.	A provider outage, capacity limit or recovery failure affects most core CivicConnect functions because PostgreSQL is the central persistent store.	RSK-006; RSK-010
Neon - Free PostgreSQL	Candidate managed PostgreSQL provider.	Compatible with standard PostgreSQL connectivity and the application's pooled connection approach. Neon's serverless model and connection options fit the low-volume deployment direction.	US$0/month. Free includes 50 CU-hours per project/month, 0.5 GB storage per project and 5 GB egress/month (Neon, 2025).	Free compute uses scale-to-zero. Free includes Instant Restore history up to 6 hours or 1 GB of changes, whichever is smaller, and limited monitoring history (Neon, 2025).	Scale-to-zero may introduce wake-up latency. Storage/compute/egress limits may be exceeded as usage grows. Restore capability must still be tested against the PED's recovery targets.	RSK-006; RSK-010; proposed RSK-011
Supabase - Free	Alternative managed PostgreSQL provider;	Compatible with PostgreSQL and supports managed Postgres connectivity. It is	US$0/month. Free includes 500 MB database	Free projects can be paused after 1 week of low activity. Automatic	Automatic pausing can reduce immediate availability. Lack of	RSK-006; RSK-010;

Product / Service	Intended Use	Compatibility	Cost for Intended Use	Availability / Service Limitations	Material Operational Risk	Related PED Risk
	could also provide file storage if later selected.	technically suitable, but its Free-plan availability and backup limits require attention before production use.	size per project, 5 GB egress and 1 GB file storage (Supabase, 2026a; Supabase, 2026-a).	backups are not included on the Free plan, and Free projects enter read-only mode after exceeding the 500 MB database-size quota (Supabase, 2026-a; Supabase, 2026b).	included automatic backups conflicts with the current recovery assumption unless the team implements another verified backup approach or moves to a paid plan.	proposed RSK-011
Nodemailer	Application library used by the Notification component to send email through SMTP.	Highly compatible with the Node.js stack and does not require a separate CivicConnect deployment unit. SMTP provider details can be changed through configuration.	US$0 licence cost. Nodemailer uses the MIT-0 licence (Nodemailer, 2026).	No independent service availability because Nodemailer runs inside the application. Actual availability depends on the SMTP provider and whether the host permits outbound SMTP.	A compatible library can still fail operationally if the chosen host blocks SMTP or if the external SMTP provider is unavailable.	RSK-005; RSK-008
Ethereal Email	Development/testing SMTP service for verifying email generation without sending to real users.	Compatible with Nodemailer and appropriate for development, CI or local staging email testing.	US$0. Ethereal describes itself as a completely free fake SMTP service for email testing (Ethereal, 2026).	Messages are captured for preview rather than delivered to real recipients; therefore it is not a production notification service.	Successful Ethereal testing does not prove that production SMTP credentials, host networking or real-world deliverability will work.	RSK-008; proposed RSK-011
Gmail SMTP	Current low-volume production SMTP direction for requester email notifications.	Compatible with Nodemailer, but only if the selected application host allows outbound SMTP. This conflicts with Render Free and Railway Free/Hobby.	No separate SMTP subscription is assumed if an existing personal Gmail account is used. If a business-domain mailbox is later required, Google Workspace Starter's standard list price is US$7/user/month in South Africa (Google, 2026-a).	Personal Gmail accounts may hit sending limits when sending more than 500 emails in a day or more than 500 recipients in a single message (Google, 2026-b).	Sending limits, authentication problems, spam filtering or host-level SMTP blocking may stop notifications. Email failure must remain isolated from the core request transaction.	RSK-006; RSK-008
GitHub Actions	CI verification for linting, tests and controlled progression to staging.	Compatible with the existing GitHub repository, Node.js runtime and Jest/ESLint workflow. It adds no production runtime service.	For public repositories, standard GitHub-hosted runners are free. GitHub Free private repositories include 2,000 Actions minutes/month and 500 MB artifact storage (GitHub, 2026).	Availability for the project is mainly constrained by included usage, workflow health and GitHub service availability. Larger runners remain chargeable even when standard public-repository use is free.	Exhausted included usage or failed workflows can block merges/releases and delay deployment, although they do not stop an already-running application.	RSK-001; RSK-005; RSK-006

Overall Deployment Risk Position

The current products are technically compatible with the Modular Monolith, Node.js/Express and PostgreSQL architecture, but the provider research identifies two material issues that must be carried forward:

1. The current free cloud-hosting assumption conflicts with the selected SMTP integration. Render Free blocks outbound SMTP ports, while Railway permits SMTP only on the Pro plan (US$20/month) (Render, 2026a; Railway, 2026b).Therefore, the team cannot currently claim that free Render/Railway + Nodemailer + Gmail SMTP forms a fully compatible production deployment. Before production, the team must either select a hosting configuration that permits SMTP, accept the required hosting cost, or reconsider an HTTPS-based transactional email provider.
2. The managed PostgreSQL provider must be selected against the project's actual recovery requirements. In particular, Supabase Free does not include automatic backups, which conflicts with the current PED wording that assumes automated provider backups (Supabase, 2026). Neon Free provides limited restore capability, but this still needs to be validated against the team's stated recovery objectives rather than treated as proven recovery evidence.

10.5 Deferred Deployment Decisions and Evidence Required

Several deployment decisions are deliberately not finalised during Milestone 2 because the required implementation, staging and provider evidence does not yet exist. These decisions are deferred to avoid making premature commitments that could introduce unnecessary cost, incompatibility or rework. The table below records the remaining deployment decisions and the evidence that must be produced before each decision can responsibly be finalised

Deferred Decision	Current Position	Why the Decision Is Deferred	Evidence Required Before Finalisation	Expected Finalisation
Production Application Hosting Provider	Render and Railway remain the main PaaS candidates. No final production host has been selected.	Both support the Node.js deployment direction, but Section 10.4 identified differences in free-tier resource limits, availability behaviour and SMTP support. In particular, the current direct Gmail SMTP approach is not compatible with the intended free tiers.	Deploy the integrated Node.js/Express application in a representative environment; measure memory/resource usage; verify PostgreSQL connectivity and pooling; verify environment-variable and secret handling; determine whether the selected email approach is supported; compare actual operational cost and availability behaviour.	M3, before production deployment.
Managed PostgreSQL Provider	PostgreSQL is baselined, but the final managed provider has not been selected. Neon and Supabase remain candidates.	The database technology is already decided, but provider-specific limits differ in storage, connection pooling, inactivity behaviour, backup and recovery capability.	Establish a working application database connection; test pooled connections; review actual database size and expected growth; verify provider backup/restore functionality; perform a restore test; confirm that the selected provider can support the current 8-hour RTO and 24-hour RPO expectations.	M3, before production release.
Attachment / Image Storage Mechanism	FR-106 attachments remain an open deployment decision. The application instance must not be treated as durable production storage.	Local application storage may be lost during redeployment, while storing large binary data in PostgreSQL may increase database size and performance cost. External object storage introduces an additional service and operational dependency.	Implement representative image uploads; establish expected file sizes and usage volume; measure database impact if database storage is considered; test a durable external-storage approach; verify access control, supported file types, quotas, backup/retention and cost.	M3, before attachment functionality is considered production-ready.
Production Email Delivery Approach	Nodemailer remains the application-level email integration. Ethereal is suitable for testing and Gmail SMTP is the current production direction, but the final production delivery configuration is not yet viable with the intended free hosting options.	Section 10.4 identified that Render Free blocks outbound SMTP and Railway restricts SMTP to higher plans. A final decision therefore depends on the selected host and whether direct SMTP remains practical.	Test email delivery from the selected deployment environment; confirm outbound-network restrictions; verify authentication and secret handling; assess actual delivery reliability; compare the cost of SMTP-enabled hosting against an HTTPS/API-based transactional email alternative if required.	M3, once the production host is selected.

Deferred Decision	Current Position	Why the Decision Is Deferred	Evidence Required Before Finalisation	Expected Finalisation
Production Environment Variables and Secrets Configuration	Secrets are currently externalised and GitHub Actions already uses repository secrets for CI. Final production values and provider-specific secret configuration remain deferred.	The final secret-management mechanism depends on the selected hosting, database and external-service providers.	Produce a complete list of required environment variables; configure them in the selected host; verify that no secrets are committed to GitHub; test startup with missing/invalid configuration; confirm database, session and email credentials are correctly injected.	M3, during deployment configuration.
Automated Production Deployment / CD	GitHub Actions currently provides CI and protects the staging branch. Automated deployment from staging/main to the production host is not yet implemented.	The CI pipeline is deliberately minimal in M2, and automated deployment is deferred until the hosting provider is baselined.	Finalise the production host; establish a manual deployment successfully first; verify environment variables and secrets; define release approval and rollback behaviour; then test an automated deployment pipeline with a controlled release.	M3, after the hosting provider is selected and manual deployment is proven.
Production Monitoring and Operational Observability	CivicConnect currently provides a /health endpoint, but full production monitoring and observability are not implemented.	Production observability was intentionally excluded from the M2 CI scope. Appropriate monitoring depends partly on the capabilities of the final deployment platform.	Deploy the application; verify /health; determine what logs, uptime information and failure alerts the selected host provides; define minimum monitoring and failure-detection evidence needed for release.	M3, before production readiness is approved.
Rollback and Recovery Procedure	Database recovery targets and a draft recovery process exist, but production rollback and recovery have not yet been proven.	The actual procedure depends on the selected hosting and database providers. Documentation alone is not sufficient evidence that recovery works.	Complete at least one database restore test; verify application reconnection after restoration; perform smoke tests against /health, login, request submission and status transition; establish how an unsuccessful application release will be rolled back.	M3, before production release.
Horizontal Scaling and External Session Storage	The initial deployment remains a single application instance. Horizontal scaling is not currently required.	The expected workload is low-to-medium, and additional instances would increase operational complexity. Session storage would need to be externalised if multiple application instances are introduced.	Collect actual workload/resource evidence; observe memory, CPU and response behaviour; determine whether a single instance becomes insufficient; only then evaluate multiple instances and external session storage.	Deferred beyond the initial deployment unless M3 evidence demonstrates a need.

These decisions are deliberately deferred rather than unresolved by omission. The current M2 baseline establishes enough architecture, technology and deployment direction for implementation to proceed, while leaving provider-specific and operational decisions open until representative evidence exists. This is consistent with the existing PED baseline, which already defers the production host, attachment-storage architecture and automated deployment until later evidence is available.

M3 Implementation Update: For the M3 staging baseline, CivicConnect uses Render for the hosted Node.js/Express modular monolith, Neon PostgreSQL for persistent relational data, and Neon S3-compatible object storage for attachments. The deployed configuration is an assessment staging environment, not a certified municipal production environment. Render Free outbound SMTP restrictions remain an unresolved notification limitation (Issue #124; RSK-016; TD-10). The PT-001 performance test was executed locally against Neon Staging and does not measure the hosted Render service.

11. RTM & Traceability Foundation

11.1 Evolved RTM

According to the CivicConnect Master Project Brief, the team must maintain one evolving Requirements Traceability Matrix (RTM), with requirements linked from their stakeholder/source through later design, implementation, testing and acceptance/release evidence. The Milestone 1 brief requires the initial RTM to establish traceability from the stakeholder/source to the requirement and acceptance criteria, while providing a structure that can be expanded with later lifecycle evidence.

In this document, the source/stakeholder, requirement description, priority and acceptance criteria are already defined in the Functional and Non-Functional Requirements tables. The RTM therefore uses the Requirement ID as the link to these definitions and extends the traceability structure to future design/architecture, GitHub Issue/Pull Request, implementation, verification/testing and acceptance/release evidence. Fields relating to future milestones are marked as pending until the relevant controlled evidence is produced.

11.2 Requester RTM

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-101	Request Management module; PostgreSQL service_requests + request_categories; Modular Monolith	PR #87	requests.service.js, requests.model.js	submission.test.js, requests.functional.test.js	Pending - M4
FR-102	Request Management + Audit / Request History; service_requests status tracking	PR #87, #98, #112	requests.controller.js:requesterDetail	submission.test.js, requests.functional.test.js	Pending - M4
FR-103	Request Management + Audit / Request History; requester-linked request persistence	PR #87, #98, #104	requests.service.js:listForRequester, search query	requests.functional.test.js	Pending - M4
FR-104	Observer / Event-Based Pattern (ADR-005); NotificationService; Nodemailer / SMTP	PR #94, #109	eventBus.js, email.service.js, notifications.listener.js	notifications.test.js	Pending - M4
FR-105	Authentication & User Management module; ASR-01; users table; Node.js / Express security boundary (ADR-004)	PR #88	auth.service.js, auth.middleware.js, bcrypt	auth.test.js	Pending - M4
FR-106	request_attachments persistence design; external durable file storage; final storage provider deferred to M3	PR #110, #112	requests.attachments.service.js, S3 integration	attachments.service.test.js, attachments.integration.test.js	Pending - M4

11.3 Staff RTM

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-201	Assignment & Workflow module; service_requests assignment model + assignment history	PR #92	workflow.service.js:assignToSelf	assignment.test.js, staff-access.test.js	Pending - M4

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-202	Centralised Workflow FSM / Transition Service (ADR-006); RequestWorkflowService; status-history persistence	PR #84	requests.model.js:updateStatus FSM	request-workflow.regression.test.js	Pending - M4
FR-203	Search, Reporting & Dashboard module; indexed request filtering by status/category/assignment	PR #95, #108	requests.model.js:searchStaffRequests	staff-search.test.js	Pending - M4
FR-204	Request Workflow + request_comments; resolution/action audit information	PR #84, #104	requests.model.js:addComment	requests.controller.test.js	Pending - M4
FR-205	ASR-01 Controlled Access; Request Management + department/role authorisation boundary	PR #97	requests.service.js:getForStaff, contact restriction	staff-access.test.js, staff-access.integration.test.js	Pending - M4
FR-206	Centralised Workflow FSM (ADR-006); RequestWorkflowService; resolution/closure persistence	PR #84	requests.model.js:updateStatus	requests.service.test.js	Pending - M4
FR-207	ASR-01; Department-Based Access; staff_assignments + departments + request_categories (DEC-007)	PR #95	requests.access.js:requireStaffAccess	department-access.test.js	Pending - M4

11.4 Management RTM

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-301	Search, Reporting & Dashboard module; service_requests + departments; status/overdue query design	PR #90	reporting.service.js:getDashboardStats	dashboard.test.js, reporting-stats.integration.test.js	Pending - M4
FR-302	Audit & Request History module; request_status_history, request_assignment_history, request_comments, admin_audit_log; ADR-005	PR #98	audit.service.js, admin_audit_log	audit.test.js, history.test.js	Pending - M4
FR-303	Assignment & Workflow module; service_requests + request_assignment_history; ASR-02	PR #106	workflow.service.js:managerAssign	management-assignment.test.js	Pending - M4
FR-304	Search, Reporting & Dashboard module; category/status/department persistence and indexed query design	PR #90, #96	reporting.service.js:getDashboardActivity	dashboard-activity.test.js	Pending - M4
FR-305	ASR-01 Controlled Access; management authorisation; contact_information_approved, contact_approved_by, contact_approved_at (DEC-007)	PR #97	requests.contact-access.service.js	contact-access.test.js, contact-access.integration.test.js	Pending - M4

11.5 Operator RTM (Added M3)

Requirement ID	Design / Architecture	Issue / PR	Implementation Evidence	Verification / Test	Acceptance / Release Evidence
FR-401	Requests Module; operatorSubmitRequest; external_requesters + service_requests (created_by); Modular Monolith	PR #87	src/modules/requests/requests.controller.js (operatorSubmitRequest), requests.service.js:createServiceRequest	src/modules/requests/__tests__/submission.test.js	Pending - M4
FR-402	Validation Schema; validateExternalRequester; DB constraint service_requests_exactly_one_requester	PR #87	src/modules/requests/requests.schema.js, src/modules/requests/views/partials/external-requester-fields.ejs	src/modules/requests/__tests__/submission.test.js	Pending - M4
FR-403	Requests Module; indexed external_requesters search ILIKE query	PR #87	src/modules/requests/requests.model.js (searchExternalRequesters, findExternalRequesterById)	src/modules/requests/__tests__/submission.test.js	Pending - M4
FR-404	Requests Module; operatorLoggedRequests; creator-filtered lookup	PR #87	src/modules/requests/requests.controller.js:operatorLoggedRequests, views/operator/logged-requests.ejs	src/modules/requests/__tests__/submission.test.js	Pending - M4

11.6 Engineering Decision Log

The Engineering Decision Log records significant decisions made during Milestone 1 so that their rationale, trade-offs, risks and later consequences remain traceable throughout the project. Where insufficient evidence currently exists to support a responsible decision, the decision is deliberately deferred and the information required before commitment is identified. In accordance with the CivicConnect Master Project Brief, significant decisions are recorded together with their context, constraints, alternatives, rationale, risks, supporting evidence and later consequences.

Field	Entry
Decision ID	DEC-001
Description	Requirement Identification Convention
Context	CivicConnect requirements must remain identifiable and traceable across the PED, RTM and later engineering artefacts. The team therefore needs a consistent naming and numbering convention.
Constraints	Requirements must use unique and stable identifiers and must remain traceable throughout later design, implementation, testing and change activities.

Alternatives	1. Use a single sequential numbering scheme for all requirements. 2. Separate Functional and Non-Functional Requirements and use numbering groups to indicate stakeholder or quality category.
Decision	Functional Requirements will continue to use stakeholder-based numbering, with FR-1xx for Requester requirements, FR-2xx for Staff requirements and FR-3xx for Management requirements. Non-Functional Requirements will use category-based numbering.
Rationale	The numbering structure allows the team to identify the primary stakeholder or quality category associated with a requirement while maintaining stable identifiers for traceability. As the scope now contains related behaviour across Requesters, Staff and Management, this separation also reduces ambiguity when requirements are later linked to design, implementation, testing and change evidence.
Trade-offs	The convention improves organisation and traceability but requires the team to maintain the numbering structure consistently as new requirements are added or changed.
Risks	1. Incorrect, duplicated or inconsistently referenced IDs could break links between the requirements tables, RTM, GitHub issues, later tests and release evidence.
Evidence	Functional Requirements table, Non-Functional Requirements table and RTM within the PED.
Later Consequence	The same requirement IDs will continue to be used in architecture/design artefacts, GitHub issues and PRs, implementation evidence, tests and acceptance evidence.

Field	Entry
Decision ID	DEC-002
Description	GitHub Branching and Review Strategy
Context	The team requires a controlled GitHub workflow that prevents uncontrolled changes to stable code and documentation while allowing three team members to work concurrently. During M2, automated CI and the need for controlled progression toward staging required the original M1 branching strategy to evolve.
Constraints	The main branch must remain highly protected. Changes must enter controlled branches through Pull Requests. However, requiring two independent reviewers for every development-level change creates a bottleneck for a three-person team.
Alternatives	1. Retain only main and dev with two approvals on all protected branches. 2. Retain main and dev but reduce approval requirements for dev (main remains unchanged). 3. Introduce testing and staging branches and apply stronger controls as code moves closer to release.
Decision	The project will use main, staging, testing and dev as controlled branches, together with temporary feature/documentation/fix branches. main and staging require 2 independent approvals. dev and testing require 1 independent approval. The testing branch additionally requires the CI status check to pass before merging.
Rationale	The revised approach preserves peer review while reducing collaboration bottlenecks during active development. Stronger controls are applied as code progresses toward staging and production.
Trade-offs	More branches increase branch-management and promotion overhead, while reduced approvals on dev and testing provide less manual review than main and staging.
Risks	Poor branch management may cause divergence, merge conflicts or confusion about which branch represents the current integration state. Reducing approvals also increases reliance on CI and effective peer review.
Evidence	GitHub branch rulesets, protected branch configuration, Pull Request history, Section 14.1 and CI evidence.
Later Consequence	M2 Amendment: The original M1 main + dev strategy has been expanded. The testing branch is now the current code-integration branch and requires one independent approval plus successful CI. The staging branch exists but its deployment integration remains deferred to M3.

Field	Entry
Decision ID	DEC-003
Description	Exclusion of High-Complexity Communication and Media Features
Context	CivicConnect could support additional interaction mechanisms such as an AI chatbot, internal real-time messaging, video uploads and voice input.

Constraints	The project is limited by schedule, team size, cost, storage capacity, security considerations and the requirement to prioritise the core service-request workflow.
Alternatives	1. Include all proposed communication/media features. 2. Include selected additional features. 3. Exclude them and focus on the core request workflow.
Decision	AI chatbot functionality, internal real-time messaging, video uploads and voice input remain excluded from the committed baseline.
Rationale	These features are not necessary to satisfy the core CivicConnect business need and would introduce additional implementation, storage, security, testing and maintenance complexity.
Trade-offs	Users receive fewer communication/input options, but the team can focus resources on the core request lifecycle, security, traceability and quality.
Risks	Some users may prefer richer communication mechanisms. Adding them later would increase scope and would require formal change analysis.
Evidence	PED Out-of-Scope section.
Later Consequence	Architecture, storage and interface design do not need to support video, audio, chatbot or internal real-time messaging functionality.

Field	Entry
Decision ID	DEC-004
Description	Defer Final Technology Stack and Architecture Selection
Context	During M1, CivicConnect had identified authentication, departmental routing, image attachments, dashboards, notifications, role-based access and responsive requester-facing interfaces, but insufficient evidence existed to responsibly commit to a final architecture and technology stack.
Constraints	Technology and architecture selection had to consider requirement fit, team capability, learning curve, cost, security, testing support, maintainability and deployment compatibility.
Alternatives	1. Select a preferred stack and architecture immediately in M1. 2. Make a provisional selection based mainly on familiarity. 3. Defer final selection until M2.
Decision	During M1, the final technology stack and architecture were deliberately deferred to M2.
Rationale	Deferring prevented premature commitment before requirements, ASRs and constraints were sufficiently established.
Trade-offs	Deferral preserved flexibility but reduced the time available for implementation once the choices had to be finalised.
Risks	Deferring too long could create schedule pressure, while deciding too early could result in an unsuitable solution.
Evidence	M1 PED scope, constraints, requirements and forward-engineering considerations.
Later Consequence	M2 Resolution: This decision is now resolved. CivicConnect has selected a Modular Monolith architecture and a JavaScript-centric MVC technology stack consisting of Node.js v24, Express.js, EJS, Tailwind CSS, PostgreSQL and node-postgres. See DEC-009, DEC-010 and DEC-011.

Field	Entry
Decision ID	DEC-005
Description	Defer CI / GitHub Actions Configuration
Context	During M1, the final build process, technology stack and testing framework had not yet been selected, so configuring CI at that stage would have relied on assumptions.
Constraints	The CI workflow must correspond to the actual runtime, dependency installation process, testing framework and quality checks used by CivicConnect.

Alternatives	1. Configure GitHub Actions during M1 using assumed technologies. 2. Create only a placeholder workflow. 3. Defer exact CI implementation until the technology stack was known.
Decision	During M1, CI implementation was deliberately deferred.
Rationale	This avoided configuring a workflow that might later require significant rework.
Trade-offs	Deferral delayed automated quality evidence but avoided throwaway implementation.
Risks	Delaying CI too long could allow defects or inconsistent code to enter integration branches without automated checks.
Evidence	M1 decision log and M2 CI implementation evidence.
Later Consequence	M2 Resolution: GitHub Actions is now implemented as the initial CI quality gate. Node.js v24 is pinned through .nvmrc; npm ci installs dependencies; ESLint/Prettier performs static checks; and Jest executes automated tests. The CI status check must pass before merging into testing. Production Continuous Deployment remains deferred to M3.

Field	Entry
Decision ID	DEC-006
Description	Use Email-Based Registration Rather Than Social Login
Context	CivicConnect requires user identity for authentication, requester ownership, staff/management roles, department membership and email notifications.
Constraints	External authentication providers introduce additional APIs, configuration, security considerations and testing requirements.
Alternatives	1. CivicConnect-managed email/password registration. 2. Third-party/social login. 3. Support both approaches.
Decision	CivicConnect will use an email-based registration and authentication mechanism. Social login will not form part of the committed baseline.
Rationale	Email is already required for notification functionality and provides a single controlled identity mechanism without an additional external identity-provider dependency.
Trade-offs	The team must manage credential security and password handling directly.
Risks	Poor authentication or password-management implementation could expose user accounts.
Evidence	Authentication Functional Requirement, Out-of-Scope section and NFR-101.
Later Consequence	Architecture and implementation must support secure credential handling, server-side authentication and role/account association.

Field	Entry
Decision ID	DEC-007
Description	Department-Based Request Access and Management-Controlled Contact Information
Context	CivicConnect must limit service-request visibility according to operational responsibility while protecting requester contact information.
Constraints	Requests may contain sensitive contact and location information. Staff should only access requests relevant to their department.
Alternatives	1. All Staff may view all requests. 2. Restrict requests by department but allow unrestricted requester-contact access. 3. Restrict requests by department and require Management approval for requester contact details.

Decision	Requests are made available only to the relevant department. Staff may accept responsibility only for eligible requests within that department. Requester contact information remains restricted until authorised by the relevant manager.
Rationale	This reduces unnecessary exposure of requester information while supporting operational ownership.
Trade-offs	The approach introduces additional permissions, approval logic and Management involvement.
Risks	Incorrect categorisation could route requests incorrectly. Incorrect access-control implementation could either expose requester information or block legitimate users.
Evidence	FR-207, FR-305, NFR-101 and the M2 persistence model.
Later Consequence	M2 data design now represents roles, departments, assignments, category routing and contact-information approval. M3 must implement and verify allowed and denied access cases.

Field	Entry
Decision ID	DEC-008
Description	Partial Mobile Responsiveness Scope
Context	Requesters are likely to submit and track requests from mobile devices, while Staff and Management screens may contain denser operational information.
Constraints	Full responsiveness across all roles would increase design and testing effort within the limited project schedule.
Alternatives	1. Make every interface fully responsive. 2. Target desktop only. 3. Prioritise Requester-facing responsive design and defer Staff/Management mobile scope.
Decision	Requester-facing functionality will support mobile browser use. The extent of Staff and Management mobile responsiveness remains deferred.
Rationale	Requesters have the strongest immediate need for mobile access, while Staff and Management interfaces are likely to require denser desktop-oriented layouts.
Trade-offs	Staff and Management may have a less optimised mobile experience.
Risks	Later evidence may show that Staff or Management require more mobile functionality, causing additional redesign.
Evidence	NFR-401, In-Scope Mobile Accessibility and Deferred Scope.
Later Consequence	Requester-facing responsive behaviour remains part of implementation. Staff and Management mobile scope must be revisited when their final workflows and interfaces are better established.

Field	Entry
Decision ID	DEC-009
Description	Select Modular Monolith Architecture
Context	CivicConnect requires clear responsibility boundaries for authentication, request processing, workflow, auditing, notifications and reporting, while being implemented by a three-person team under a limited schedule.
Constraints	The architecture must support ASR-01 to ASR-05, remain low-cost, avoid unnecessary deployment complexity and support maintainability and testing.
Alternatives	1. Traditional Layered Monolith. 2. Microservices / Distributed Architecture.
Decision	CivicConnect will use a Modular Monolith architecture. It will remain a single deployable application while major responsibilities are separated into internal modules.
Rationale	The Modular Monolith provides stronger internal separation than a basic monolith without introducing the network, deployment and data-consistency complexity of microservices.
Trade-offs	Individual modules cannot be independently deployed or scaled.
Risks	If internal boundaries are not maintained, the application could gradually become tightly coupled. A failure in the single application process may also affect the complete system.

Evidence	Architecture comparison, Sections 5.1-5.6, architecture diagrams and architecture ADR.
Later Consequence	M3 implementation must preserve the defined internal boundaries between presentation, request handling, business rules, persistence and supporting services.

Field	Entry
Decision ID	DEC-010
Description	Select PostgreSQL as the Relational Persistence Technology
Context	CivicConnect must reliably persist users, requests, assignments, workflow history, comments, audit information and departmental relationships.
Constraints	The persistence technology must support relational integrity, transactions, concurrency protection, efficient queries and low-cost deployment.
Alternatives	1. Relational PostgreSQL persistence. 2. A document-oriented database. 3. Simplified file/in-memory persistence.
Decision	CivicConnect will use PostgreSQL as its persistent relational database and node-postgres as the database access layer.
Rationale	PostgreSQL supports relational constraints, transactions, row-level locking and the structured relationships needed for CivicConnect's request lifecycle and access-control model.
Trade-offs	PostgreSQL introduces reliance on a managed database environment for production and requires appropriate schema and transaction design.
Risks	Poor transaction or concurrency handling may result in lost updates or inconsistent request state. Provider-specific storage and recovery capabilities may also be insufficient.
Evidence	Section 6 Data & Persistence Baseline, ERD/schema and technology-selection section.
Later Consequence	M3 implementation must connect the application to PostgreSQL and implement transaction and concurrency controls. The database technology is decided, but the managed provider remains deferred between candidates such as Neon and Supabase.

Field	Entry
Decision ID	DEC-011
Description	Select JavaScript-Centric MVC Technology Stack
Context	The project requires a practical web technology stack that the three-person team can implement and maintain within the milestone schedule while supporting authentication, database access, responsive pages, testing and deployment.
Constraints	The stack must remain maintainable, low-cost, lightweight enough for limited hosting environments and compatible with the team's capability.
Alternatives	1. React/Next.js SPA with separate API backend. 2. Java Servlets / Spring Boot. 3. JavaScript-centric server-rendered MVC application.
Decision	CivicConnect will use Node.js v24, Express.js, EJS, Tailwind CSS and node-postgres as its primary application stack.
Rationale	A unified JavaScript stack reduces learning and integration overhead. Server-side rendering avoids a separate frontend deployment and reduces unnecessary client-side state complexity.
Trade-offs	EJS provides a less dynamic client experience than a SPA architecture and generally relies more heavily on full-page server-rendered interactions.
Risks	Poor separation of responsibilities could still produce tightly coupled code even within an MVC structure. Runtime or dependency differences could also create environment inconsistencies if versions are not controlled.
Evidence	Section 7 Technology Stack Decision; ADR-004; .nvmrc; package.json; application repository.
Later Consequence	Node.js v24 is pinned through .nvmrc. M3 implementation will continue using the selected Express/EJS structure and PostgreSQL integration rather than introducing a separate SPA frontend.

Field	Entry
Decision ID	DEC-012
Description	Decouple Core Request Transactions from External Side-Effects
Context	Status changes and request-resolution operations must update persistent CivicConnect state but may also trigger secondary behaviour such as email notifications and audit processing.
Constraints	Failure or delay in an external service such as SMTP must not cause a valid request-state transaction to fail or remain open unnecessarily.
Alternatives	1. Direct synchronous service orchestration. 2. Observer / Event-Based internal side-effect handling.
Decision	CivicConnect will use an Observer / Event-Based Approach for secondary side-effects. RequestWorkflowService commits the primary transaction first, then publishes an in-memory event which supporting listeners such as NotificationService and AuditLogService respond to.
Rationale	This keeps the core request state independent from slower or less reliable supporting functions.
Trade-offs	In-memory events are lightweight but are not durable across application-process failure.
Risks	An event may be lost if the Node.js process fails after the database commit but before listeners complete. Notification failures may therefore occur even though the underlying request state is correct.
Evidence	Initial Design Decision 1; ADR-005; src/services/emailService.js structural stub; RSK-013.
Later Consequence	M3 must implement the EventEmitter-based behaviour and ensure that email/audit failures do not roll back valid request transactions. The platform remains the source of truth even if a notification fails.

Field	Entry
Decision ID	DEC-013
Description	Centralise Request Lifecycle State Mutation
Context	The request lifecycle includes controlled transitions such as SUBMITTED, ASSIGNED, IN_PROGRESS, RESOLVED and CLOSED. Duplicating transition logic across controllers would increase inconsistency and security risk.
Constraints	Workflow rules must remain maintainable, testable and consistently enforced across the application.
Alternatives	1. State Pattern with separate state classes. 2. Procedural Centralised Workflow / Transition Service.
Decision	Request lifecycle changes will be controlled through a single RequestWorkflowService containing a central allowed-transition configuration / finite state machine. Controllers will delegate mutation requests to this service.
Rationale	The centralised service provides consistent transition validation without the additional boilerplate of separate state classes for a relatively small fixed lifecycle.
Trade-offs	The service may become too dense if too many responsibilities are added.
Risks	A poorly structured central service could become a new monolithic component.
Evidence	Initial Design Decision 2; ADR-006; src/services and src/controllers structure.
Later Consequence	Validation rules should be extracted where appropriate so that RequestWorkflowService remains an orchestrator rather than accumulating unrelated logic. All request state mutations in M3 must pass through this controlled boundary.

Field	Entry
Decision ID	DEC-014
Description	Select Nodemailer with SMTP as the Application-Level Email Integration
Context	FR-104 requires requesters to receive feedback when request status changes. The email mechanism must integrate with the Node.js application without creating unnecessary architectural complexity.
Constraints	The solution should remain low-cost, configurable across environments and independent from the core request transaction.
Alternatives	1. Managed REST-based transactional email API. 2. Broker-based asynchronous messaging. 3. Direct SMTP integration using Nodemailer.
Decision	CivicConnect will use Nodemailer with SMTP as its application-level email integration mechanism. Ethereal will be used for development/testing.
Rationale	Nodemailer integrates naturally with Node.js, introduces minimal vendor-specific application logic and allows SMTP configuration to vary between environments.
Trade-offs	Direct SMTP depends on hosting-provider networking restrictions and SMTP-provider reliability.
Risks	Current deployment research shows that Render Free blocks outbound SMTP and Railway restricts SMTP to higher plans. Therefore the current production SMTP direction may conflict with low-cost hosting constraints.
Evidence	Section 9 Initial API / Integration Decisions; ADR-005; deployment compatibility research; RSK-008 and RSK-013.
Later Consequence	The application-level mechanism is selected, but the final production email provider/configuration remains deferred to M3. The production solution must be proven compatible with the selected host.

Field	Entry
Decision ID	DEC-015
Description	Establish Deployment Environment Progression and Single-Instance Baseline
Context	CivicConnect must progress from local development toward production without assuming that behaviour proven locally or in CI automatically proves production compatibility.
Constraints	The team has limited infrastructure capacity and must avoid premature commitment to a paid or incompatible cloud environment.
Alternatives	1. Deploy directly from development to production. 2. Introduce a containerised/distributed deployment pipeline immediately. 3. Progress through controlled environments before production commitment.
Decision	CivicConnect will use the progression Development -> Test/CI -> Local Staging -> Production. The current deployment baseline is a single application instance, consistent with the Modular Monolith architecture.
Rationale	Local staging allows the team to validate the integrated application, configuration, database connectivity and supporting services before selecting and committing to production infrastructure.
Trade-offs	This introduces an additional staging step and delays final cloud-provider selection.
Risks	Environment-specific configuration differences may cause behaviour that works in development or CI to fail in staging or production.
Evidence	Section 10 Deployment Compatibility, RSK-014 and Section 15 Architecture, Technology and Initial Design Baseline.
Later Consequence	Render and Railway remain candidate production application hosts, while Neon and Supabase remain candidate PostgreSQL providers. Final hosting, database provider, attachment storage, production email configuration, production secrets, Continuous Deployment, monitoring, recovery validation and horizontal scaling remain deferred until M3 evidence is available.

Field	Entry
Decision ID	DEC-016
Description	Add an Operator role and external requester support to CivicConnect.
Context	The M2 baseline assumed all requesters would self-register and use the portal. During M3 planning, the team identified that this excludes community members who use telephone, email, WhatsApp, in-person, or paper channels. Historical and ongoing requests received through these channels could not be captured in CivicConnect, recreating the original fragmentation problem.
Constraints	External requesters are not system users. They must not register, log in, or receive accounts. Operator-assisted capture must preserve the same core request information as self-service. Existing contact-access approval rules still apply.
Alternatives	1. Exclude external and operator-assisted requests. 2. Require all requesters to use the self-service portal. 3. Add an Operator role and external requester records.
Decision	Add the OPERATOR role. Add an external_requests table for external requester identity and contact details. Extend service_requests with external_requester_id and created_by. Add a check constraint ensuring exactly one requester source: either requester_id or external_requester_id. Add an operator submission path and an operator logged-request list.
Rationale	Enables CivicConnect to serve as the single source of truth for all service requests, regardless of submission channel. Supports inclusion, business continuity, traceability, and accountability by recording which operator captured each request and how it was received.
Trade-offs	Introduces a new role, additional data model changes, new UI screens, extra validation, and additional privacy obligations. Operator edit capability is intended but currently affected by defect #122.
Risks	POPIA and privacy risk for external requester details. Duplicate or inaccurate requests if operators mistype. Operator cannot currently edit a request they logged.
Evidence	Section 1.2.7; Section 2.1.1 item 16; Section 6.1.11; Section 11.5; Section 18.2.1; PR #87; submission.test.js.
Later Consequence	RTM adds FR-401 to FR-404. Defect #122 remains open and urgent.

Field	Entry
Decision ID	DEC-017
Description	Introduce an additional database entity to allow management approval process that allows or denies staff access to requester contact details.
Context	Staff may need requester contact details to clarify information or arrange service delivery. However, being assigned to a request should not automatically grant access to personal information. The system needs controlled access when needed, with accountability for who requested access and who approved it.
Constraints	Must comply with privacy expectations and POPIA. Access must be limited to the assigned staff member, the relevant department, and the specific service request. Reassignment must clear current approval. All decisions must be auditable.
Alternatives	1. Allow staff unrestricted access to requester contact details. 2. Deny staff access entirely. 3. Introduce a manager-controlled approval workflow per service request.
Decision	Add the request_contact_access_requests table. Support statuses PENDING, APPROVED, REJECTED, and CANCELLED. Provide a manager inbox for reviewing access requests. Allow manager approval, rejection, direct authorisation, and revocation. Clear current approval when a service request is reassigned. Record audit events for contact-access actions.
Rationale	Gives department managers responsibility for deciding when requester contact details may be shared. Supports service delivery while protecting requester privacy and maintaining accountability.
Trade-offs	Adds manager workload and may delay staff contacting requesters. Introduces additional permission checks and test requirements.
Risks	Incorrect approval could expose personal information. Future query or interface changes could bypass restrictions. Historical access requests may become outdated after reassignment. A rejected request cannot currently be resubmitted by the same staff member for that service request.
Evidence	Section 6.1.12; Section 11.4; Section 18.2.2; contact-access tests; PR #97.

Field	Entry
Later Consequence	Contact visibility depends on approval, current assignment, and department access checks. Audit logging is required. Service request operational status remains separate from contact-access status.

Field	Entry
Decision ID	DEC-018
Description	Select durable external S3-compatible object storage for service request attachments.
Context	M2 deferred the final image storage decision. Local application-instance storage is not durable and may be lost on restart or redeployment. Storing large binary images directly in PostgreSQL does not scale efficiently and increases database size and performance cost.
Constraints	Must remain free or low-cost. Must keep large binaries out of PostgreSQL. Must support secure, time-limited access. Must work with the existing Node.js and Express stack.
Alternatives	1. Store images directly in PostgreSQL. 2. Store images on the local application filesystem. 3. Store images in external S3-compatible object storage.
Decision	Use Cloudflare R2 or S3-compatible object storage via @aws-sdk/client-s3 and @aws-sdk/s3-request-presigner. Store only the storage key path in PostgreSQL. Serve images through secure, time-limited presigned URLs.
Rationale	Provides durable, scalable, and cost-effective attachment storage. Avoids database bloat and keeps attachment access controlled.
Trade-offs	Introduces an additional external service and operational dependency. Requires credential management, bucket configuration, and backup coordination.
Risks	Storage provider outage. Misconfigured access control. Orphaned database records or missing files. Provider quotas or retention limits.
Evidence	Section 6.1.7 M3 update; Section 7.3; attachment integration tests; PR #110 and PR #112.
Later Consequence	Backup and recovery must coordinate database records with object storage. Reconciliation procedures are required. Final production storage provider remains subject to operational validation.

Field	Entry
Decision ID	DEC-019
Description	Use a PostgreSQL transaction and a year-specific advisory transaction lock during request-number allocation.
Context	M2 left the final concurrency control approach undecided. Concurrent submissions could otherwise allocate duplicate request numbers or produce missing persistence records.
Constraints	Must work within free-tier PostgreSQL limits. Must preserve correctness under concurrent submissions. Must remain practical for a three-person team. Must not introduce unnecessary distributed-system complexity.
Alternatives	1. Optimistic concurrency control with a version column. 2. Pessimistic row locking with SELECT FOR UPDATE. 3. PostgreSQL transaction with a year-specific advisory transaction lock for number allocation.
Decision	Use a PostgreSQL transaction and a year-specific advisory transaction lock during request-number allocation. This serialises conflicting number-allocation work within the defined lock scope.
Rationale	Supports unique request-reference assignment under concurrent submissions. PT-001 created 90 distinct request references with no observed duplicate or missing persistence records.
Trade-offs	Lock contention is a possible performance contributor. The approach is evidence for the exercised path, not proof of all concurrency interleavings.
Risks	Lock contention may affect latency. Broader concurrent updates still require transaction and concurrency design..
Evidence	Section 6.4 M3 Concurrency Implementation; PT-001 results;

Field	Entry
Later Consequence	Continue monitoring concurrency and performance. Broader concurrency control still needs validation.

Field	Entry
Decision ID	DEC-020
Description	Select Render Free Web Service, Neon PostgreSQL, and Neon Object Storage for the M3 staging environment.
Context	M2 deferred final production hosting and managed PostgreSQL provider selection. M3 required an externally accessible, controlled staging environment to validate the integrated release candidate under deployment-like conditions.
Constraints	Must remain free or low-cost. Must support Node.js and Express. Must provide managed HTTPS and an externally accessible URL. Must keep persistent data outside the application instance. Must work with GitHub-based deployment.
Alternatives	1. Use a local-only staging environment. 2. Use Render or Railway with Neon or Supabase. 3. Use a paid hosting tier.
Decision	Use Render Free Web Service for the application, Neon PostgreSQL for persistent data, and Neon Object Storage for attachments. Deploy from the staging branch with auto-deploy on commit. Use NODE_ENV=staging as the runtime mode.
Rationale	Provides a cloud-hosted staging environment that is closer to real-world deployment than local testing. Avoids hosting cost. Integrates with GitHub and the existing Node.js and Express architecture.
Trade-offs	Free-tier limitations include reduced resources, cold starts, in-memory sessions, and blocked outbound SMTP. The staging environment does not prove production-scale reliability.
Risks	SMTP notifications fail on Render Free because outbound SMTP ports are blocked. Provider resource limits. Cold-start latency. Shared Neon resources may affect development data.
Evidence	Section 13.1; Section 14.2; Section 14.3; Section 14.5; staging URL https://civicconnect-staging.onrender.com/.
Later Consequence	Production hosting, production database provider, attachment storage provider, production email delivery, secrets, monitoring, and rollback validation remain deferred. The release candidate is classified CONDITIONALLY READY.

Field	Entry
Decision ID	DEC-021
Description	Abandon the dev branch for M3 and use testing as the primary integration branch.
Context	M1 planned main and dev. M2 added testing and staging. In M3, using dev would have added an extra integration step and an additional pull request for every change, slowing development and preventing continuous testing and integration.
Constraints	Three-person team. Peer review required. CI gate required. Stronger controls must apply closer to release. Avoid unnecessary branch-management overhead.
Alternatives	1. Keep dev as the integration branch. 2. Merge feature branches into dev, then dev into testing. 3. Create feature branches from testing and merge directly back into testing.
Decision	Stop using dev for M3. All feature, fix, and documentation branches are created from testing and merged back into testing via pull request. Promotion path is feature branch to testing, then staging, then main. testing requires 1 independent approval and a passing CI gate. staging and main require 2 independent approvals and a passing CI gate.
Rationale	Avoids an unnecessary extra integration step. Every change is validated by the full CI pipeline before joining the shared integration branch. Supports continuous integration and continuous development.

Field	Entry
Trade-offs	testing becomes the primary integration branch. Branch management and promotion overhead remain. Reduced approvals on testing provide less manual review than main or staging.
Risks	Poor branch management may cause divergence, merge conflicts, or confusion about the current integration state. Reducing approvals increases reliance on CI and peer review.
Evidence	Section 13.5; Section 16.1; PR #117; PR #120.
Later Consequence	dev is no longer part of the workflow. Branch protection and governance documentation must reflect this.

Field	Entry
Decision ID	DEC-022
Description	Expand the CI pipeline from a single build-and-test job into a multi-job verification gate.
Context	M2 established an initial CI quality gate. M3 required stronger automated verification for static analysis, dependency security, secret scanning, database checks, and multiple test suites before merging into protected branches.
Constraints	Must use GitHub Actions. Must remain free or within included usage. Must inject secrets through GitHub Actions repository secrets. Must not deploy in CI. Must not require production infrastructure.
Alternatives	1. Keep a single build-and-test CI job. 2. Rely on manual checks. 3. Implement a multi-job CI pipeline with independent verification jobs and a final gate.
Decision	Implement independent CI jobs: lint, dependency-audit, secret-scan, unit-tests, functional-tests, db-test, integration, e2e, and ci-gate. The ci-gate job aggregates results and fails if any dependency job fails, is cancelled, or is skipped. The CI gate is the single required status check for branch protection.
Rationale	Provides stronger automated verification. Enforces secret scanning, dependency auditing, static analysis, and regression testing before merge. A skipped job cannot count as a pass.
Trade-offs	More CI time and complexity. Database-backed jobs consume free-tier Neon compute and connection limits. Green CI does not prove production readiness.
Risks	CI minutes exhaustion. Flaky tests. Secret leakage if misconfigured. Integration and E2E tests may be limited by database availability.
Evidence	Section 13.6; Section 16.6; .github/workflows/ci.yml; shared setup action; PR #119 CI block demonstration.
Later Consequence	Maintain the pipeline. Performance testing remains outside CI and is handled separately through PT-001. TD-09 records the manual performance-test integration limitation.

Field	Entry
Decision ID	DEC-023
Description	Use Grafana k6 to conduct a controlled performance test of concurrent service request submission against the Render-hosted CivicConnect staging application.
Context	NFR-201 requires 95% of standard API read and write requests to complete in less than 800 ms under normal system load. Service request submission (FR-101) was selected because it represents a critical business operation involving request validation, application processing and database persistence. Delayed submissions may negatively affect requester experience and encourage repeated submission attempts.
Constraints	Limited M3 schedule, three-person development team and free-tier infrastructure. The assessment requires one justified performance-sensitive operation rather than enterprise-scale benchmarking. A manageable workload of five concurrent virtual users was selected. The existing Render staging deployment and Neon Staging database were used without introducing additional hosting infrastructure.
Alternatives	1. No performance testing: Rejected because it would not satisfy M3 requirements or provide measurable NFR-201 evidence. 2. Test only the application in an isolated local environment: Not selected as the final verification strategy because local hosting would not exercise the deployed Render application. 3. Conduct a controlled load test against Render staging: Selected because it provides direct performance evidence for the hosted M3 release candidate.

Field	Entry
Decision	Implement PT-001 using Grafana k6 v2.2.0 to evaluate authenticated POST /requester/submit operations against the Render staging deployment. Execute a preliminary one-user sanity run followed by a five-user concurrent workload. Measure submission response times, success/error behaviour and throughput, and independently verify database persistence. Maintain the test scripts on staging_test/performance_test pending controlled integration into staging.
Rationale	The selected approach provides measurable evidence for NFR-201 while exercising an important business operation through the deployed application and Neon PostgreSQL database. Using the existing staging deployment increases the relevance of the results to the M3 release candidate without requiring additional infrastructure. Separate database reconciliation strengthens the evidence by verifying that successful HTTP submissions correspond to persisted requests.
Trade-offs	The test focuses on one operation and a short, closed-loop workload rather than broad performance coverage. It does not establish maximum capacity, sustained performance or compliance across every system operation. The initial health check and authentication preparation occur before the measured workload, so cold-start responsiveness is not evaluated. Performance-testing integration into staging and CI is deferred.
Risks	Free-tier resource constraints, cold starts and changing database conditions may affect performance. The five-user workload may not represent realistic municipal traffic. Although PT-001 passed, other operations and higher workloads remain unverified. The initial health check took approximately 22.5 seconds, outside the measured submissions. Deferred test integration may also allow future performance regressions to go undetected.
Evidence	PED Section 13.10; PT-001; tests/performance/request-submission.js; tests/performance/README.md; Render-hosted k6 execution summaries, resource measurements and PostgreSQL persistence evidence in tests/performance/evidence/. Related technical debt: TD-09.
Later Consequence	PT-001 passed for the documented Render staging workload. The full run processed and persisted all 191 submissions, recorded zero submission errors and achieved a p95 response time of 453.98 ms against the NFR-201 target of 800 ms. The result provides positive performance evidence for the tested operation, but broader performance compliance and production-scale readiness remain unverified. Further testing and controlled integration into staging/CI are deferred under TD-09.

Field	Entry
Decision ID	DEC-024
Description	Accept the Render Free-tier SMTP limitation for M3 and classify the release candidate as CONDITIONALLY READY.
Context	Status-change notification emails cannot be sent from the free Render deployment because Render blocks outbound SMTP ports 25, 465, and 587. This is a platform limitation, not a code defect. The email flow works locally with Ethereal and Gmail SMTP. Failed email sends do not roll back core request transactions.
Constraints	Free-tier budget. M3 timeline. No production release intended. Core request transactions must remain unaffected by email failure.
Alternatives	1. Upgrade to a paid Render plan immediately. 2. Switch to an HTTPS transactional email API. 3. Accept the limitation for M3 and classify the release candidate as CONDITIONALLY READY.
Decision	Accept the SMTP limitation in M3. Keep the release candidate classified as CONDITIONALLY READY. A path to READY requires resolving urgent defects, enabling and verifying email delivery in staging, and integrating PT-001 into a controlled verification workflow.
Rationale	The core system is functional. SMTP notification is a secondary concern and does not roll back core transactions. The project is constrained by budget. The classification transparently reflects residual operational risk.
Trade-offs	Requesters will not receive email notifications on the current Render staging environment. The system is not approved for unrestricted municipal production use.
Risks	User dissatisfaction due to missing notifications. Operational impact for requesters. Urgent defect backlog remains. Infrastructure and budget constraints may delay READY status.
Evidence	Section 15; Issue #124; DEC-012; release evidence summary.
Later Consequence	Production email delivery remains deferred. SMTP-enabled hosting or an HTTPS transactional email API must be selected before production. CONDITIONALLY READY is not production approval.

Field	Entry
Decision ID	DEC-025
Description	Adopt a formal change management standard for all changes made after a baseline has been approved.
Context	CR-001 and CR-002 were raised after the M2 baseline. The team needed a controlled path to ensure changes are not silently absorbed into the PED, codebase, database schema, or other controlled artefacts.
Constraints	Every change must be traceable from request through impact analysis, decision, authorisation, implementation, verification, and baseline update. Change records must connect intent, affected artefacts, implementation evidence, and resulting baseline.
Alternatives	1. Manage changes informally. 2. Use a formal change request and impact analysis process. 3. Only document major changes.
Decision	Adopt the change path: Change Request to Impact Analysis to Decision to Authorise to Implement to Verify to Update Baseline. Record each change in the change register and expand it into a detailed controlled change record.
Rationale	Ensures that changes are controlled, justified, authorised, verified, and traceable. Prevents uncontrolled scope creep and silent baseline changes.
Trade-offs	Adds documentation and review overhead. Requires discipline to keep registers and baselines synchronised.
Risks	The process may not be followed consistently. Registers may become out of sync with release branches, as noted in TD-01.
Evidence	Section 18; CR-001; CR-002; change register.
Later Consequence	All future changes must follow the same controlled path. Baseline updates must reference affected sections and evidence.

Please note that due to the space constraints in Microsoft Word the format in which the Engineering Decision Log is displayed in this PED is not the same as the table format we will record and work with in the Engineering_Decision_Log.md file in our GitHub repository.

12. Risk & Forward Engineering Considerations

12.1 Initial Risk Register

In accordance with the CivicConnect Master Project Brief, the Risk Register is maintained as a live engineering artefact and records project-specific risks together with their cause, probability, impact, priority, mitigation, contingency, owner and current status. Risks are identified from the current CivicConnect scope, requirements, constraints and engineering decisions and will be reviewed and updated as new evidence becomes available throughout subsequent milestones. Where a risk occurs, it will be treated as an issue and linked to the relevant engineering actions or decisions.

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-001	Schedule overrun due to scope and learning curve The team may fail to complete the committed CivicConnect scope within the required milestone schedule.	The project contains multiple user roles, authentication, departmental access, dashboards, notifications, image uploads, responsive requester-facing functionality, database integration and deployment work, while the team consists of only three members and must complete the remaining implementation within a limited milestone schedule.	Medium	High	Critical	Maintain the baselined scope; prioritise Must-have requirements; use the selected JavaScript/Node.js stack to avoid unnecessary learning overhead; track milestone work through GitHub issues; keep work visible through PRs; avoid implementing deferred or out-of-scope features without formal change analysis.	If schedule variance becomes significant, defer or remove lower-priority Should/Could requirements through controlled change rather than reducing essential testing, security or quality controls.	Development Team Product Owner	Resolved
RSK-002	Uncontrolled scope growth Additional feature requests or ideas may expand the committed project scope and threaten delivery of the baseline requirements.	The Product Owner may want additional functionality, and the team has already identified several deferred features that could appear valuable later.	Medium	High	High	Maintain the approved baseline scope; clearly distinguish in-scope, out-of-scope and deferred functionality; require formal change requests and impact analysis before accepting additional features; update affected requirements, RTM, decisions and risks through controlled change.	Reject or defer proposed changes where schedule, cost, quality or security impact cannot be accommodated. If a change is necessary, update the controlled artefacts before implementation.	Product Owner Development Team	Closed
RSK-003	Unauthorised access to requester information Requester contact information or service-request data may be exposed to users who are not authorised to access it.	CivicConnect has Requester, Staff, Management and Admin roles, department restrictions, staff assignment rules and management-controlled access to requester contact details, creating complex server-side authorisation requirements.	Medium	High	Critical	Implement authentication, secure password hashing, server-side RBAC and department checks; centralise access-control rules; restrict requester contact information by default; use secure session configuration; test permitted and denied scenarios; keep secrets out of source control.	If an access-control flaw is found, restrict or disable affected functionality, correct the authorisation logic, investigate any exposed information and verify the fix before restoring access.	Development Team Management	Resolved
RSK-004	Incorrect request categorisation or departmental routing A service request may be categorised incorrectly and therefore become unavailable to the department responsible for resolving it.	Requesters select a category when creating a request, and the category determines which department can access and handle the request.	Medium	High	High	Use a controlled category list with clear labels/descriptions; validate category-to-department mappings; allow Management to redirect or reassign requests where necessary; test department access rules.	Allow Management to correct or redirect incorrectly categorised requests and record the correction through the audit/history trail.	Requester Management Development Team	Resolved

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-005	Unsuitable production deployment platform or provider selection The selected production platform may not adequately support CivicConnect's operational requirements and constraints.	The application architecture and technology stack are now selected, but the final PaaS host and managed PostgreSQL provider remain deferred. Candidate providers differ in memory, free-tier limits, SMTP support, database pooling, backup/recovery and operational behaviour.	Medium	High	High	Keep final provider selection deferred until M3 evidence exists; validate the integrated Node.js application on representative infrastructure; compare Render/Railway and Neon/Supabase against runtime support, PostgreSQL connectivity, secrets/configuration, SMTP compatibility, availability, recovery and total operating cost.	If a provider proves unsuitable, stop production release and move to the next evaluated candidate or adjust the supporting service configuration before substantial production dependence is created.	Development Team	Closed/ Risk accepted
RSK-006	Free-tier resource or service limitations CivicConnect may exceed the limits of free-tier hosting, database, storage, CI or supporting services.	The project must remain free or low-cost, while application hosting, PostgreSQL, image storage, email delivery and CI consume limited resources. Current provider research shows material quota and plan restrictions.	Medium	High	High	Monitor resource consumption and provider quotas; measure application memory and database usage; use connection pooling; enforce image limits; keep binaries out of the application instance; compare actual monthly cost before production; select low-cost alternatives only where they still satisfy required functionality.	Reduce non-essential resource consumption, tighten upload limits, move to another evaluated provider or accept a justified paid tier through controlled decision if the free option cannot meet the baseline.	Development Team Product Owner	Resolved
RSK-007	Image upload creates storage or security problems User-uploaded images may consume excessive storage or introduce unsafe or unsupported content.	Image uploads are in scope, while the final file-size limit, storage provider and complete validation approach are still deferred to M3.	Medium	Medium/ High	High	Allow only supported image types; validate file type and size server-side; enforce upload limits and authorisation for retrieval; store metadata and stable file references; monitor storage quotas; test the final storage mechanism before production.	Temporarily reduce supported file sizes/types or disable FR-106 upload functionality until storage/security constraints are corrected and verified.	Development Team Requester	Resolved
RSK-008	Email notification dependency or delivery failure Requesters may not receive status notifications reliably even when the underlying request is updated correctly.	Email delivery depends on Nodemailer plus the final SMTP provider, host networking rules, provider sending limits, authentication and external service availability.	Medium	Medium	Medium	Keep notification processing separate from the core request transaction; log delivery success/failure; test with Ethereal during development and the final provider in staging/production; keep credentials externalised; allow requester notification preferences; verify host SMTP compatibility before production.	Retain the correct status within CivicConnect if email fails; allow users to view status in the platform; retry or manually resend where appropriate; switch provider/configuration if required.	Development Team Requester	Risk accepted during M3

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-009	Team-member availability or knowledge concentration Progress may be delayed if important project knowledge or work becomes concentrated in one team member who becomes unavailable or falls behind.	The project is completed by only three team members and members may become primary contributors to specific technical or documentation areas.	Medium	High	High	Maintain readable documentation and code; require peer review; ensure more than one member understands major requirements, decisions and artefacts; use GitHub issues/PRs to keep work visible; avoid isolated ownership of critical areas.	Reassign affected work using documented artefacts, GitHub history and peer-reviewed contributions; reduce lower-priority work if necessary to protect milestone delivery.	Development Team	Resolved
RSK-010	Loss or corruption of platform data Service-request records, assignment information, status history, audit information or other persistent data may be lost, corrupted or become inconsistent.	PostgreSQL is selected, but the final managed provider and verified backup/recovery process are still deferred. Provider free tiers differ in backup/restore capability, and recovery has not yet been proven.	Low/Medium	High	High	Use PostgreSQL integrity constraints and transactions; validate writes; maintain audit/history records; keep database persistence independent of the application instance; target RTO 8 hours and RPO 24 hours; select a provider that can support recovery needs; perform at least one restore test before production.	Pause production writes if corruption is detected; restore from the latest verified backup/recovery point; reconnect the application through updated configuration; run smoke tests before resuming service; prevent release if recovery cannot be demonstrated.	Development Team Management	Resolved
New Risks identified in M2									
RSK-011	Lost update / inconsistent request state due to unresolved concurrency control Two users may modify the same request at the same time and one change may overwrite another.	Staff and Management can perform overlapping operations such as status updates, assignment/reassignment and contact-access approval. The final concurrency mechanism is still to be proven during M3.	Medium	High	High	Treat concurrency control as required; retain a version column on service_requests; evaluate optimistic locking against SELECT FOR UPDATE; wrap status transitions in atomic transactions; create simultaneous-update tests.	Reject stale/conflicting updates, ask the user to refresh and retry, and correct affected records using request_status_history and request_assignment_history where necessary.	Development Team	Resolved
RSK-012	Attachment storage is undecided and images may be lost or unavailable Uploaded images may be stored non-durably, exceed provider limits or delay FR-106.	The PED requires durable attachment storage, but the final mechanism/provider is deferred. Local application-instance storage is not durable across restart or redeployment.	Medium	High	High	Store only metadata and a stable file reference in request_attachments; do not rely on application-instance storage for production; prototype a durable external-storage approach early in M3; enforce file size/type limits; verify access control, retention, recovery and cost.	Temporarily disable or limit FR-106 until a durable storage option is confirmed; migrate any temporary/local files to the selected durable storage before production.	Development Team	Resolved
RSK-013	Email notification failure or silent loss due to in-memory events Notifications may be lost or duplicated even though the core request update succeeds.	The selected Observer/Event-Based approach uses an in-memory Node.js event emitter for secondary actions. Events do not survive a process crash, while SMTP delivery also has external failure modes.	Medium	Medium	Medium	Log all notification outcomes; assign a notification/event identifier to reduce duplicates; attempt a controlled retry for transient failures; test with Ethereal and the final SMTP provider; ensure the platform remains the source of truth for request status.	Mark failed notifications for audit/manual retry; allow the requester to view the correct status inside CivicConnect; switch SMTP provider/configuration if needed.	Development Team	Risk accepted, depended on RSK-008

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-014	Deployment environment or configuration mismatch CivicConnect may work in development or CI but fail in local staging or production because environment-specific configuration differs.	The deployment path spans Development -> Test/CI -> Local Staging -> Production. Database URLs, SESSION_SECRET, SMTP credentials, NODE_ENV and provider-specific settings are externalised and may be missing, invalid or inconsistent between environments.	Medium	High	High	Pin Node.js v24 through .nvmrc; use package-lock.json/npm ci; document all required environment variables; keep secrets external to source control; create consistent environment templates; test startup with missing/invalid configuration; validate the integrated application in local staging before production.	Block deployment, compare the affected environment with the validated staging configuration, correct the mismatch, rerun health/smoke tests and only continue once configuration parity is restored.	Development Team	Open - Monitoring
RSK-015	Insufficient verification / test coverage before integration or release Defects may pass the current quality gate because important functionality is not yet covered by automated or systematic tests.	At the end of M2, automated verification is mainly the /health test. Authentication, workflow, database operations, concurrency, UI interactions, attachments and email behaviour are not yet fully implemented or tested.	Medium	High	High	Expand Jest unit/integration tests during M3; add tests for authentication/authorisation, request workflow, database transactions, validation, concurrency and notification failure; add end-to-end and responsive/browser checks where practical; map tests back to the RTM and acceptance criteria.	Prevent release where critical requirements lack verification evidence; prioritise tests for Must-have/security/data-integrity paths; fix failed behaviour and rerun the full relevant test set before promotion.	Development Team	Resolved
New Risks identified in M3									
RSK-016	Outbound SMTP Blocking on Free-Tier PaaS (Email Notification Failure) Automated status change emails fail to deliver when deployed to staging.	Render Free and Railway Free block outbound traffic on standard SMTP ports to prevent spam abuse.	High	High	Medium	The implementation of paid Render services will be considered.	Users won't receive updates on their request status via email; they will manually have to check the status by logging into their accounts.	Development Team	Open-Active
RSK-017	Session Loss and Re-Authentication on Cloud Container Restart Logged-in citizens, staff, and managers are suddenly logged out and forced to re-authenticate.	express-session currently uses the default in-memory MemoryStore. PaaS free tiers (e.g. Render) spin down inactive containers after 15 minutes and restart them on new deploys.	High	Low / Medium	Medium	Session cookie duration is set to 8 hours with httpOnly and sameSite: 'lax' for security; forms contain client-side validation to avoid losing inputs prior to submit.	Implement a persistent session store (such as connect-pg-simple using the existing Neon PostgreSQL database or Redis) before production multi-user rollout.	Development Team	Open-Active
RSK-018	Serverless PostgreSQL Cold-Start Latency (Scale-to-Zero) Initial page loads or request submissions experience 1–3 second delays after periods of inactivity	Neon Free Tier uses serverless compute that suspends (scales to zero) after 5 minutes of inactivity to conserve the 50 CU-hours monthly quota.	Medium	Low / Medium	Medium	Node-postgres connection pool uses connectionTimeoutMillis: 10000 to prevent dropouts, and /health ping can warm up the instance.	Monitor cold-start latency against NFR-201 (<800ms); if wake-up times degrade user experience in staging, evaluate connection keep-alive pings or an always-on database instance.	Development Team	Open - Monitoring

Risk ID	Risk / Description	Cause	Probability	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-19	Asynchronous Event Loss on Sudden Process Termination An email notification or audit log event is lost if the Node.js process crashes immediately after database commit.	In-memory EventEmitter (eventBus.js) is decoupled from the SQL transaction. If the host process is killed before the listener executes, in-flight events are not persisted in a durable queue.	Low	Medium	Medium	The primary business operation (status update, resolution comment, assignment) is committed to PostgreSQL first, ensuring core platform data remains consistent.	In a future release, introduce a persistent transactional outbox table in PostgreSQL to guarantee at-least-once delivery across process restarts.	Development Team	Open - Accepted

12.2 Forward Engineering Considerations

The considerations below identify why each concern is relevant to the current engineering baseline, which later decisions or activities it may influence, what information is still required before a responsible decision can be made, and the potential consequence of ignoring the concern.

Security, Authentication and Access Control

Security and access control already influence the CivicConnect baseline because the system will store requester information and provide different levels of access to Requesters, Staff and Management. Requests are also restricted according to department, while requester contact information may only be released to Staff after Management approval. This means that later architecture and implementation decisions must support controlled access rather than treating all authenticated users in the same way. The current requirements already establish department-based access and management-controlled contact information.

- **Later decision/activity influenced:** Authentication architecture, role-based access control, department-based permissions, database relationships, API access rules, security testing and deployment configuration.
- **Information still missing:** Final authentication technology, detailed role and permission structure, session or token mechanism, password-storage approach and the technical method used to represent Management approval of contact-information access.
- **Risk if ignored:** Security may be introduced too late in the design process, which could require significant redesign or result in unauthorised users gaining access to requests or requester information

Data Persistence, Integrity, Auditability and Recovery

Reliable data storage is very important when it comes to the CivicConnect project because currently the operational processes suffer from requests being lost, duplicated, overlooked or poorly tracked. The new system must therefore preserve request information, assignments, status changes, resolution details and lifecycle history reliably.

- **Later decision/activity influenced:** Database technology, schema design, relationships, transaction handling, audit logging, backup and recovery planning.
- **Information still missing:** Final database technology, persistence model, audit fields, retention requirements, backup frequency and recovery approach.
- **Risk if ignored:** Poor data design could recreate the original business problem by allowing request records or accountability information to become incomplete, inconsistent or unrecoverable.

Testability, Verification and CI Quality Gates

The requirements and acceptance criteria defined during M1 will determine whether the team can later prove that CivicConnect works correctly. The current PED already requires measurable evidence, automated testing and Continuous Integration rather than relying only on claims that the code works.

- **Later decision/activity influenced:** Test strategy, unit/integration/system testing, testing framework selection, GitHub Actions configuration, merge quality gates and release evidence.
- **Information still missing:** Final technology stack, testing tools, build process, CI workflow, performance-test conditions and quality thresholds.
- **Risk if ignored:** Requirements may be implemented without reliable verification evidence, while vague acceptance criteria could become difficult to test later.

Deployment, Storage, Free-Tier Limits and Operational Cost

CivicConnect must operate within free or low-cost constraints, but the final hosting, database and storage environment has not yet been selected. Features such as image uploads, persistent data, email notifications and authentication may all consume limited platform resources. The PED already identifies free-tier limits as an engineering constraint.

- **Later decision/activity influenced:** Hosting platform, database service, image-storage approach, email service, deployment pipeline and operational-cost decisions.
- **Information still missing:** Storage quotas, database limits, bandwidth limits, image-size limits, expected usage volumes, email-service restrictions and final deployment platform.
- **Risk if ignored:** The selected solution may work during development but exceed free-tier limits, create unexpected costs or require late redesign of storage and deployment components.

Responsive UI and Stakeholder-Specific Mobile Use

Requester-facing mobile accessibility is already part of the baseline, while the extent of mobile responsiveness for Staff and Management has deliberately been deferred. This means the team must support requester mobile use without prematurely committing to making every interface fully responsive.

- **Later decision/activity influenced:** Wireframes, responsive layouts, browser support, UI architecture, usability testing and refinement of NFR-401.
- **Information still missing:** Staff and Management mobile-use requirements, final dashboard complexity, target screen sizes and supported browser/version range.
- **Risk if ignored:** The team could either overcommit to full mobile responsiveness and increase development effort unnecessarily or create requester-facing screens that are difficult to use on mobile devices.

13.Development and Application Evidence (M3 Release Candidate)

13.1 Identified Release candidate for M3

GitHub repository	https://github.com/Waldo-Blom/SEN381_Bugbusters
M3 Release candidate	Branch staging, commit # f6ecc48, V1.0.1. Link: https://github.com/Waldo-Blom/SEN381_Bugbusters/releases/tag/V1.0.1 *
CI / test evidence	See section "15.6 CI / Automated Checks" of PED
Staging environment (Render URL)	https://civicconnect-staging.onrender.com/

* Please note that this release candidate is for code only, all the documentation for the project such as RTM, risk register, AI usage register etc lives on the main branch and can be found here https://github.com/Waldo-Blom/SEN381_Bugbusters/tree/main/docs . This is done as we treat main as our production, we have not yet released to production and that is why no code is yet committed to main.. Also to note this is not a candidate for public release we are not saying that this is a release intended for public distribution, this candidate is still in staging and not yet deployed to prod.

Additionally , PT-001 was executed against a locally launched application source reported at 8d63e6744c80025a7a6d0d4bd785c48ab79db58f on staging_test/performance_test. This has been noted in our technical debt register under TD-09.

13.2 M3 Overview and Current Status

At the end of M2, CivicConnect was an EJS navigational prototype with no live backend. The M3 release candidate is an integrated, working service-request platform.

- **Authentication:** session-based authentication, with requireAuth, requireRole and requireDepartment middleware.
- **Persistence:** live Neon PostgreSQL database
- **Request lifecycle:** a centralised workflow state machine with transactional status changes.
- **Contact-access governance:** a manager-approved workflow for requester contact details.
- **Attachments:** S3-compatible object storage with short-lived signed URLs.
- **Notifications and audit:** email notifications and an append-only audit trail, driven by an in-process event bus (see section 13.9 for caveat to this).
- **Reporting:** manager dashboard and reporting.
- **Operator role (CR-001):** operator-assisted capture for external requesters.

The agreed M3 scope is frozen at the release candidate commit.

Status: the release candidate is classified **CONDITIONALLY READY** (see 13.10). The system is not called "production ready".

13.3 Repository / Module Structure

In M2 our applciation prototype was a MVC folder structure (https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/43) however in M3 we made sure to follow the modular monolith as we documented in M2 (https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/52).

The repo folder structure is as follows:

```
project-root/
├── .github/        CI workflow, issue templates, Node setup action
├── db/             Database schema seed and utility scripts
├── docs/ (The updated documentation lives on the main branch under this folder)
├── public/         Static front-end assets (JS, CSS)
├── src/
│   ├── config/     Database, environment, session, object storage
│   ├── modules/    Feature modules (audit, auth, notifications, pages,
│   │               reporting, requests, users, workflow)
│   ├── shared/     Middleware, errors, events, validation, layouts
│   ├── app.js      Express app setup
│   └── server.js   Server entry point
├── tests/          End-to-end, functional and integration tests
└── (root files)    package.json, README.md, ESLint, Prettier, Jest, .env.example
```

For the full detailed folder structure, please view it here on github: https://github.com/Waldo-Blom/SEN381_Bugbusters/tree/staging

13.4 M2 Baseline Conformance and Controlled Changes (Assessment Area A)

13.4.1 Baseline conformance

M2 basline decision	Status at M3	Evidance
Modular Monolith (DEC-009)	Implemented as planned. Single deployable app, modules under src/modules/.	Repository structure
PostgreSQL + node-posgres	Implemented. Connection pool (max 5) with a UTC-aware type parser. Hosted on Neon.	db/scripts/test-db.js, db/seeds/Final_database_schema.sql
Nodejs v24, Express EJS, Tailwind	Implemented. Pinned via .nvmrc, CSS built with npm run build:css.	.nvmrc, package.json
Observer / event based effects (DEC-012, ADR-005)	Implemented. Notification and audit failures do not roll back request transactions.	shared/events/eventBus.js, notifications.test.js
Centralized workflow (DEC-013, ADR-006)	Implemented.	workflow module: src/modules/workflow module
Nodemailer / SMTP (DEC-014)	Implemented with an Ethereal (dev) and Gmail (prod) transport factory.	email.service.js, notifications.test.js

	Production delivery is limited by host SMTP blocking (see RSK-016).	
Single instance bassline	Preserved. Request handling is stateless. Sessions are held in the application instance.	app.js
Concurrency control (deferred in M2)	Implemented for requester number allocation Submission transaction uses a year-specific PostgreSQL advisory transaction lock.	requests.model.js
Attachment storage (deferred in M2)	Resolved in M3. S3-compatible object storage, with only the storage key stored in PostgreSQL. Done via Neon object storage	object-storage.js, attachments.service.test.js

13.4.2 Significant deviations and controlled changes

For significant changes we implemented change control, for this please see section 18. Change Management Standard & Controlled Changes

13.5 Collaborative Construction and Configuration Control (Assessment Area B)

This section summarises how we implemented controlled change during M3 construction. The full governance model (branch rules, pull request process, issues, milestones and labels) is in **Section 16 (GitHub Governance)**, which has been updated for M3. This section states the M3 position and where the evidence can be found.

Branching strategy and deviation from the M1/M2 plan

The M1 baseline planned a main and dev structure. During M2, a testing branch and a staging branch were added (DEC-002). In M3 the team did not use dev for development. All feature, fix and documentation branches were created from testing, and all pull requests targeted testing.

Reason: We wanted continuous integration and continuous development. With feature branches merging into testing, every change was verified by the full CI gate before it joined the shared integration branch. A separate dev branch would have added second integration that would have slowed down development.

How changes move between branches: feature branch, testing, staging, main. Stronger controls apply as code moves closer to release: one approval and a CI gate on testing, then two approvals and a CI gate on staging and main. The testing to staging promotion was made through PR #117 and a Patch on PR #120, which required two independent approvals and a passing CI gate.

Branch	Role in M3	Required approvals	Required Checks
Feature / fix / docs branches	Short-lived. Created from testing, merged back into testing by PR.	n/a. The changes on this branch is the developers own branch where they make commits and does not require approval form anyone	CI runs on each PR into testing

Testing	Continuous-integration branch. All code was integrated and verified here.	1 independent approval	CI gate and PR approval
Staging	Release-candidate validation. The M3 submission baseline	2 independent approvals	CI gate and PR approval
Main	Protected stable branch. Treated as the production branch	2 independent approvals	PR approval
Dev	Created in M1 but not used for M3	1 independent approval	PR required

Peer review

Every substantive change entered a protected branch through a pull request reviewed by team members other than the author. Reviewers were expected to go through the changes and comment, not only approve. For some of our changes we also held code reviews on discord where we would sit in a meeting, show what we changed, why we changed it and how we tested the feature. A few examples of reviews that changed the code:

- Staging Patch V1.0.1 - https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/120
- CivicConnect Implementation to Staging for M3 Validation - https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/117
- FR-106: Service request image attachments- https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/110

Configuration and secrets

No secrets are committed to the repository. This is enforced in three ways:

1. Detection: Gitleaks scans the full Git history on every CI run (secret-scan job).
2. Injection in CI: CI_SESSION_SECRET and NEON_DATABASE_URL are supplied as GitHub Actions repository secrets.
3. Environment-specific values: each environment takes its own values from environment variables. The .env file was used for local development and testing and was never committed to the repository

The Node.js version is pinned through .nvmrc and dependencies are installed with npm ci from package-lock.json, so development, CI and staging use the same runtime and dependency versions.

Progressive history

The repository history is progressive and not a single late upload. Work was delivered through feature branches and pull requests into testing across M1 to M3. Application prototype was created on PR #43, the structure was improved to follow modular monolith in M3 in PR #52 (https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/52), the development then continued on, and we had our release candidate in PR #120 (https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/120).

Release candidate identification

The M3 release candidate is on the staging branch at commit f6ecc48, tagged https://github.com/Waldo-Blom/SEN381_Bugbusters/releases/tag/V1.0.1 . Commits made to staging after that tag are not part of the M3 submission (Submission Baseline Rule), and the tagged commit is the version demonstrated during the presentation. Also to note this is not a intended release for

production as there are still known defects documented in section Defects, Findings and Technical Debt (Assessment Area E).

13.6 CI and Automated Verification (Assessment Area C)

This section summarises the automated verification used to control the M3 release candidate and what the results do and do not demonstrate. The full CI configuration (triggers, shared setup, secret handling, branch-protection integration and gate evidence) is in Section 16.6 (CI / Automated Checks), which has been updated for M3. This section gives the engineering summary and the test evidence.

Pipeline summary

The pipeline is defined in .github/workflows/ci.yml and runs on pushes and pull requests to the protected branches and on manual dispatch. In M3 all feature branches were created from testing and all PRs target testing (see 13.5), so every change runs the full pipeline before it joins the integration branch. A newer run for the same ref cancels an older in-progress run. Every job runs on a fresh ubuntu-latest runner through a shared setup action (Node.js version from .nvmrc, clean npm ci install, Tailwind build where needed).

The pipeline has grown from the single build-and-test job of M2 into independent jobs covering static analysis, dependency and secret scanning, four automated test suites, a database check, and a final gate.

Job	What it checks	What a pass demonstrates	What it does not demonstrate
Lint	ESLint (with Prettier ingegration) and prettier –check	Code meets the style and static-analysis rules	Logic correctness
dependency-audit	npm audit --omit=dev --audit-level=high	No known high or critical vulnerabilities in production dependencies	Unknown vulnerabilities, or low and moderate findings
secret-scan	Gitleaks scan of the full Git history	No secrets committed in any commit	Secrets handled correctly at runtime
unit-tests	Jest unit project (src/**/__tests__/). Each test isolates its unit by mocking the database and other modules. The full unit suite runs on every push and PR, regardless of which files changed.	Business rules hold for each unit, and no change has broken previously verified unit behaviour	Behaviour against a real database. The database is deliberately mocked in unit tests so that each test checks business logic only, and a failure points to the unit rather than to the network or the data. Mocking keeps the tests fast, and independent of shared state, and it avoids consuming free-tier Neon compute and connection limits (RSK-006) on every push.
functional-tests	Black box tests	Functional scenarios are tested by sending HTTP requests through the Express application and verifying observable outcomes, including response status codes,	Database and persistence-layer functions are mocked to isolate the application workflows. Consequently, these tests verify application-level functional behaviour but do not establish the correctness of SQL queries.

		redirects, rendered pages, validation messages, authentication and session behaviour.	
db-test	npm run db:test connects to the CI Neon database, confirms it is the dev database and verifies all 11 expected tables	Schema and connectivity are correct	Data correctness
integration	Jest integration project against Neon (attachments, contact access, health, manager list, reporting, requests, staff access). Runs after db-test.	Modules and PostgreSQL cooperate correctly, including department scoping and contact-access rules	Behaviour on the staging or production database. This was tested on staging rather than the testing branch
e2e	Starts the server in production mode, polls /health (which runs SELECT 1 on Neon), then runs the critical requester journey. Runs after db-test.	The integrated system completes register, login, submit and track	Real browser behaviour. (This was tested by us manually where we tested the application and the UI on multiple different browsers and devices)
ci-gate	Aggregates all jobs, runs with if: always() and fails on failure, cancelled or skipped	Every configured check reported success. A skipped job cannot count as a pass.	Anything not configured as a check

13.6.1 Test inventory

Suite	Test Cases	Focus
Unit	169	Auth, workflow rules, attachment validation, reporting stats, error middleware, datetime, object storage, audit, notifications, user management
Functional (black-box)	5	FR-101, FR-102, FR-103, FR-105 requester journeys
API / integration	39 (database-backed suites require DATABASE_URL)	Staff access, contact access, attachments, reporting aggregates, manager list, health
E2E / system	1	Register, login, submit, track
Negative / failure / unauthorised (overlaps the above)	47	Cross-department 403, CSRF 403, expired session 401, invalid image 400/413, invalid

		UUID 400, duplicate email 409, unauthorised upload 403, dependency failure 500
Performance	1	See section 13.9

Unit tests are located under each separate module under a “__tests__” folder, for example for test related to the auth module, unit tests are located at `src/auth/__tests__` and for notifications the tests are located at `src/notifications/__tests__`. e2e , functional and integration tests are located at `src/__tests__`, for performance test see section 13.10

13.7 Quality Strategy Applied (Risk-Based)

CivicConnect applies a **risk-based quality strategy**: test effort is allocated according to the likelihood and impact of failure, not evenly across every module. The risk register identifies the highest-risk areas as unauthorised access to requester information, incorrect departmental routing, image-upload storage/security, data integrity and loss of request lifecycle evidence.

The requests module is by far the largest module in the codebase and carries the largest share of tests, because it contains the core domain logic: submission, categorisation, assignment, workflow transitions, contact access, attachments and lifecycle history. Submission of a service request is the primary user-facing outcome of the system, so defects there would directly break the business need CivicConnect exists to solve.

Risk-to-quality mapping:

Risk / qaulity concern	Why it matters	Qaulity response	Primary evidance
RSK-003 - Unauthorised access to requester information	Critical privacy and security risk. Contact details must only become visible after management authorisation and only to the correct assigned staff member.	Decision-table, negative and integration tests for contact-access approval/rejection, department scoping,	tests/integration/contact-access.integration.test.js, tests/integration/staff-access.integration.test.js, tests/integration/attachments.integration.test.js
RSK-004 - Incorrect request categorisation or routing	A request routed to the wrong department may be overlooked or handled by unauthorised staff. Submission must reject unknown or inactive categories before persistence.	Functional submission tests validate category existence and required fields. Manager-list integration tests verify department-scoped reads and category/status/search filters.	tests/functional/requests.functional.test.js, tests/integration/manager-list.integration.test.js, src/modules/requests/__tests__/submission.test.js
RSK-007 - Image upload creates storage or security problems	Uploads can consume storage, introduce unsafe content or expose attachments to the wrong user.	Attachment integration tests cover MIME/extension/content validation, 5 MB boundary, unauthorised retrieval, upload restrictions and correct uploader identity.	tests/integration/attachments.integration.test.js, src/modules/requests/__tests__/attachments.test.js

RSK-010 - Loss or corruption of platform data	CivicConnect depends on request, assignment, status and audit history being consistent.	Transaction-aware integration tests for workflow transitions, status history, assignment history, contact approval persistence and reporting aggregates. Health endpoint verifies database reachability.	tests/integration/requests.integration.test.js, tests/integration/reporting-stats.integration.test.js, tests/integration/health.test.js
RSK-008 - Email notification dependency or delivery failure	Email is useful but must not cause the underlying request update to fail or be rolled back.	Component/contract tests for notification listener and email rendering.	src/modules/notifications/__tests__/notifications.test.js, src/modules/notifications/email.service.js

13.8 Black-Box, Integration and System Validation (Assessment Area D)

Black-box validation treats CivicConnect as a system where behaviour must match the requirements, business rules, interface contracts and accepted user journeys. The central question is:

Given the specification and observable behaviour, which inputs, combinations, states, interactions and failure conditions could reveal that CivicConnect does not behave as required?

The primary black-box evidence is stored in:

- tests/functional/ – route-level functional tests using Supertest, real Express routes, middleware, validation, sessions and rendered views.
- tests/integration/ – API, database, workflow, attachment and reporting integration tests. Some use real PostgreSQL temporary tables when DATABASE_URL is configured (in our GitHub secrects this is a secret so on a CI run this is configured and does run);
- tests/e2e/ – browser-like critical journeys through the complete Express route, middleware, service, view and session stack.

13.8.1 Functional and E2E black-box records

ID	Requirement	Black-box type	Test basis	Input / condition	Expected observable result	Evidence location	Verdict
BB-01	FR-101, FR-105	E2E / system	Registration, login and submission	Register, sign in, submit	Registration accepted; submission redirects; listing	tests/e2e/critical-journeys.e2e.test.js – “registers, signs in, submits a report, and tracks its generated reference”	Pass condition

			acceptance criteria	“Street light out”	contains CC-2026-0001 and title; detail shows Submitted		met by suite
BB-02	FR-101	Functional black-box, EP valid	Valid submission generates a unique request reference	Valid category, title, description, location, priority	302; one record; requestsModel.create called once	tests/functional/requests.functional.test.js – “FR-101 accepts valid details once…”	Pass condition met by suite
BB-03	FR-101, NFR-501	Negative, EP invalid	Required fields and category must be valid	Blank each required field; unsupported category ID	400; human-readable error; no request created; no persistence	tests/functional/requests.functional.test.js – “FR-101 rejects every missing required field…”	Pass condition met by suite
BB-04	FR-103, NFR-101	Decision table, negative/security	Request history is scoped to authenticated requester	Signed-in requester with own and other records; signed-out request	Own request visible; other requester’s request not visible; signed-out gets 302/401	tests/functional/requests.functional.test.js – “FR-103 limits history…”	Pass condition met by suite
BB-05	FR-105, NFR-101	Use-case, state, negative	Registration and authentication rules	Valid registration; valid login; incorrect password	Registration 302; login 302 to /requester/my-requests; wrong password 401; no session	tests/functional/requests.functional.test.js – “FR-105 registers a requester…”	Pass condition met by suite

13.8.2 Integration, API and state-transition records

Please note this is not every single Integration, API and state transition test, the detailed test can be found under `src/tests/integration`. Below highlights some of the most important of those tests and how they link to our functional requirements

ID	Requirement	Black-box type	Test basis	Input / condition	Expected observable result	Evidence location	Verdict
BB-06	FR-106, FR-101	API / integration, use-case, persistence	Supported image must be associated with the correct request	Multipart request with valid PNG	302; one attachment record; key matches requests/{requestId}/uuid.png; uploader is authenticated user; request inserted before attachment	tests/integration/attachments.integration.test.js – "creates a request and associates a selected image…"	Pass condition met by suite
BB-07	FR-106, NFR-501	Negative, BVA, EP	JPEG/PNG /WebP only; 5 MB maximum	Fake image; 5 MB + 1 image	Fake image 400; oversized 413; no DB connection; no storage upload	tests/integration/attachments.integration.test.js – "rejects %s without creating a request"	Pass condition met by suite
BB-08	FR-106, FR-207, NFR-101	Decision table, negative/security	Attachment view/upload depends on role, ownership and request access	Requester/operator/staff/management with wrong access; anonymous; staff/management view and upload	Unauthorised retrieval 403; anonymous 401; authorised staff/management can view (302) but cannot upload (403)	tests/integration/attachments.integration.test.js – "denies attachment retrieval…" and "allows an authorised %s…"	Pass condition met by suite
BB-9	FR-305	State transition, use-case, decision table, integration	Contact access starts PENDING, manager approves, contact becomes visible only to assignee	Staff requests twice; manager approves; duplicate approval	Pending message shown; inbox shows staff/reference; approval 302; contact visible only to assignee; duplicate 409; DB approval set; audit actions recorded	tests/integration/contact-access.integration.test.js – "requests once, loads the manager inbox, approves…"	Pass condition met by suite
BB-10	FR-305, NFR-101	State transition, negative	Rejected access must stay denied and not	Staff requests; foreign manager rejects; correct manager rejects;	Foreign manager 403; manager 302; denial message; no contact data; resubmit 403;	tests/integration/contact-access.integration.test.js – "rejects access, shows the denial message…"	Pass condition met by suite

			reveal contact data	resubmission attempted	audit REQUESTED/REJECTED		
BB-11		State transition, integration, negative	Unassigned SUBMITTED request can be self-assigned; colleague cannot take over	SUBMITTED, unassigned; staff assigns; colleague assigns	Staff assign 302; colleague 403; one assignment-history row	tests/integration/staff-access.integration.test.js – "records self-assignment and prevents a colleague…"	Pass condition met by suite
BB-12	FR-202, FR-204	State transition, negative, decision	Resolution requires a comment; progress must follow allowed transitions	Resolve without comment; start progress; resolve with comment	Without comment 400; with comment status Resolved; comment and timeline recorded; resolved_at set	tests/integration/requests.integration.test.js; tests/integration/staff-access.integration.test.js	Pass condition met by suite
BB-13	FR-206, FR-301, FR-302, FR-304	State transition, integration, reporting	Resolve/close/reopen must update counts, deadlines, SLA and history	Overdue request resolved, closed, reopened; fixture statuses	Open/overdue/resolved/closed counts change; resolved_at/closed_at correct; SLA totals update; history matches	tests/integration/reporting-stats.integration.test.js; tests/integration/reporting-activity.integration.test.js	Pass condition met by suite
BB-14	FR-203, FR-207	API / integration, decision table	Manager list is scoped to owned department and filters	Status, category and search filters	Only owned request returned; invalid/foreign filters rejected; % and SQL-like input treated literally	tests/integration/manager-list.integration.test.js – "combines status, category and search filters…"	Pass condition met by suite
BB-15	NFR-501	API / integration, negative	Health endpoint reports	Database reachable;	200 {status:"ok", database:"connected"};	tests/integration/health.test.js	Pass condition met by suite

			database availability	database unavailable	503 {status:"error", database:"unavailable"}		

13.9 Defects, Findings and Technical Debt (Assessment Area E)

Note that the RTM is above in section “11.RTM & Traceability Foundation”

All defects, findings and technical debt identified during M3 verification are recorded as GitHub Issues in the project repository, and the five issues below are the complete set documented for the release candidate. Using GitHub Issues keeps each item linked to the pull requests, tests and commits that relate to it.

Each issue follows the project’s issue template: summary, steps to reproduce, expected and actual behaviour, acceptance criteria or verification, and evidence. Each is labelled by task type (for example bug) and by priority, using the project’s priority labels.

Issue	Title	Type	Priority	State
https://github.com/Waldo-Blom/SEN381_Bugbusters/issues/121	[FEATURE] Export log btn on "Report & Audit" screen for the Manager does not work	FEATURE	medium	open
https://github.com/Waldo-Blom/SEN381_Bugbusters/issues/122	[BUG] Operator can't edit a request that they logged	BUG	urgent	open
https://github.com/Waldo-Blom/SEN381_Bugbusters/issues/123	[FEATURE] Password reset functionality still needs to be implemented	FEATURE	high	open
https://github.com/Waldo-Blom/SEN381_Bugbusters/issues/124	[BUG] SMTP (email send of notifications) does not work on Render	BUG	urgent	open
https://github.com/Waldo-Blom/SEN381_Bugbusters/issues/125	[BUG] Temporary fix of time zone configuration	BUG	low	open
https://github.com/Waldo-Blom/SEN381_Bugbusters/issues/126	[NFR-201] Request-submission performance - Render staging retest passed	Test	high	open

Summary of each item

#121: Manager’s “Export log” button on Report & Audit screen does not work (feature, priority: medium)

The Manager’s Export log button on the Report & Audit screen currently produces no usable export of the audit log. The issue is a feature request rather than a defect: the button is present, but the end-to-end export action has not been implemented. The expected behaviour is that the manager receives a readable CSV file containing the audit-log entries they are authorized to view, respecting the current department scope and any filters applied on the page. The acceptance criteria require that the export returns a valid file in the agreed format, that exported entries match the visible filters and authorized scope, and that empty results and export errors are handled clearly. Test coverage for export content and access controls is also required.

Impact: Managers cannot extract audit-log data for reporting or compliance purposes, forcing manual review of the on-screen log.

Recommended fix: Implement the server-side export endpoint and wire it to the button, applying the same authorization and filter logic used on the Report & Audit screen. Add unit and integration tests covering authorized export, filter matching, empty results, and error handling.

Open verification items: The issue's acceptance criteria list export format validation, filter/scope matching, empty-result handling, and test coverage as items still to be confirmed.

#122: Operator cannot edit a request they logged (bug, priority: urgent)
An operator cannot edit a service request that they previously logged. The issue is classified as urgent because it blocks a core operational workflow: operators need to correct request details when an error is found after submission. When the operator attempts to edit the request, the UI returns a 403 (Forbidden) error, even though the operator should have edit access to requests they own. The expected behaviour is that the operator who logged the request can edit the request.

Impact: Operators cannot correct mistakes in requests they created, which may result in inaccurate request data and force workarounds such as creating duplicate requests or contacting an administrator.

Recommended fix: Review the access-control logic for the request-edit endpoint and correct the authorization check so that the operator who logged the request is recognized as an authorized editor when the request is in an editable state. Add tests covering authorized editing, denied access, invalid input, and lifecycle restrictions.

Open verification items: The acceptance criteria require tests for authorized editing, denied access, invalid input, and lifecycle restrictions; these tests have not yet been confirmed as passing.

#123: Password reset functionality still needs to be implemented (feature, priority: high)
The password-reset feature is incomplete. Although the login screen links to a password-reset page and the UI appears, there is no end-to-end functionality behind it: users cannot request a reset, receive a reset link, or set a new password. The expected behaviour is a full password-recovery flow: the user requests a reset using their email address, the system provides a response that does not reveal whether the account exists, a short-lived single-use reset link is sent via email for a matching account, the user can set a new password meeting the existing password policy, the token cannot be reused after success or expiry, and the user can sign in with the new password while the old password no longer works.

Impact: Users who forget their password have no way to reset their password.

Recommended fix: Implement the password-reset request form and server-side handling, generate a secure single-use expiring token, send the reset link through the configured email notification mechanism, validate the new password against the existing policy, and add tests for valid, expired, reused, invalid, and unknown-account reset requests. Confirm that a successful reset allows login with the new password.

Open verification items: The issue's "Current status" section notes that the password-reset page/entry point already exists and that the team should confirm which elements are already present before development begins.

#124: SMTP email notifications do not work on Render (bug, priority: urgent)
Status-change notification emails cannot be sent from the free Render deployment because Render's free tier blocks outbound traffic on SMTP ports. This is a platform limitation and not a code defect. It is the realised instance of the risk recorded in the Risk Register (RSK-016, and the free-tier risk RSK-006), and the deployment research. The email flow itself was verified separately with both Ethereal (preview message captured) and Gmail SMTP (email received in a real inbox), and notification unit

tests pass (src/modules/notifications/**tests**/notifications.test.js). Switching from Ethereal to Gmail required only environment variable changes, with no code change. A failed email send does not roll back the request or status update, so the core transaction is unaffected (DEC-012).

Impact: Requesters will not receive email notifications on the current Render staging environment.

Recommended fix: Move to a paid Render plan that permits SMTP and configure the SMTP settings, then re-test on the staging branch before any production deployment.

Open verification items: The issue's checklist still lists capturing the Render SMTP-block error log, confirming that a failed send does not roll back a status update on Render, and confirming that requesters with notifications switched off receive nothing.

#125: Temporary fix of time zone configuration (bug, priority: low)

The current implementation forces displayed times to South African time (Africa/Johannesburg). This is a temporary patch, not a complete approach for an application that may be used by people in other countries or time zones. The production database is hosted by Neon in Virginia, USA, and its PostgreSQL session timezone is UTC/GMT, while the team and current users are in South Africa. Several SQL statements use LOCALTIMESTAMP, which PostgreSQL evaluates as the current date and time without a timezone using the database session timezone. Application's conversion/display code assumes UTC for these legacy values and formats them in Africa/Johannesburg. The team has not yet determined the best long-term approach, and the temporary nature of this fix was explicitly noted in PR #116.

Impact: Users in other time zones will see times displayed in South African time rather than their local time, which may cause confusion around deadlines, response times.

Recommended fix: Investigate and agree on an approach suitable for users in different time zones, preserving event instants and displaying them in the selected or appropriate timezone in both local and deployed environments. Duration calculations must remain correct across timezone boundaries and daylight-saving changes. The affected LOCALTIMESTAMP usages are documented in the issue and span the audit, reporting, requests, assignment, contact-access, and request-model modules.

Open verification items: The acceptance criteria require reproducing and identifying the affected timestamp paths, agreeing on a timezone approach, verifying correct display in local and deployed environments, and confirming that duration calculations remain correct across timezone boundaries and daylight-saving changes.

#126 [NFR-201] Request-submission performance - Render staging retest passed
PT-001 targeted the deployed CivicConnect Render staging service (https://civicconnect-staging.onrender.com). **The hosted test met the configured NFR-201 latency threshold for the selected operation and workload.** All 191 full-run submissions returned the expected behaviour and were reconciled with 191 persisted database records. No submission errors were recorded. The measured full-run p95 was **453.98 ms**, below the established **800 ms** criterion.

For more info please see the issue created

Documented technical debt

Technical debt is recorded by type so that the cause, consequence and management approach remain traceable. Each item links to the GitHub Issue, risk register entry or architectural decision that created or governs

ID	Type of Technical Debt	Item	Description	Origin	Impact	Current Mitigation	Recommended Action
TD-01	Documentation	Registers out of sync with release branch	The live engineering registers (AI Usage, Risk, Decision Log, RTM) are maintained on main, while the M3 release candidate and its code reside on staging. The staging branch does not carry the most current register versions.	Deliberate branch separation: main is treated as the production/documentation baseline; staging is the release candidate branch.	Reviewers inspecting the release branch see a documentation snapshot that predates the latest register updates.	Registers are version-controlled on main; links to the live documents are embedded in the PED.	Merge main documentation into staging at release promotion, or adopt a single source of truth (e.g. publish registers from main only and link from both branches).
TD-02	Code & Implementation	Operator cannot edit logged requests	Operators receive HTTP 403 when attempting to edit a request they created. The authorisation check does not recognise the	Access-control logic in requests.attachments.model.js and requests.service.js does not include an operator-ownership path for edit.	Blocks a core operational workflow (Issue #122, urgent). Operators cannot correct mistakes and may create	Documented as GitHub Issue #122 (urgent); duplicate-request risk noted.	Extend the edit authorisation predicate to allow the operator who created the request when status is Submitted. Add tests for authorised/denied edit.

			operator as an authorised editor.		duplicate requests.		
TD-03	Code & Implementation	Export log button not implemented	The Manager "Export log" button on the Reporting & Audit screen is present but has no server-side endpoint.	UI placeholder; export endpoint never implemented.	Managers cannot extract audit data for reporting or compliance (Issue #121, medium).	Button exists but produces no usable export.	Implement a CSV export endpoint respecting department scope and active filters. Add unit + integration tests.
TD-04	Code & Implementation	Password reset incomplete	The login screen links to a reset page, but no end-to-end flow exists (no token generation, no email, no password update).	Password-reset page was scaffolded early; full flow deferred.	Users who forget their password cannot recover their account (Issue #123, high).	Page exists as UI only.	Implement request form, single-use expiring token, email dispatch via existing notification service, password policy validation, and tests for valid/expired/reused/unknown-account flows.
TD-05	Code & Implementation	Timezone handling is a temporary patch	Display code forces Africa/Johannesburg on legacy timestamp without time zone columns that store UTC wall time. SQL uses LOCALTIMEST	Neon database host in Virginia (UTC) vs South African team; legacy schema uses timestamp not timestamptz.	Users outside SA see SAST times; duration calculations may be fragile across DST boundaries (Issue #125, low).	src/shared/datetime.js centralises conversion; documented in docs/timestamps.md.	Migrate event columns to timestamptz, update SQL display conversions, and write timestamps explicitly as UTC instants.

			AMP in several modules.				
TD-06	Architecture & Design	In-memory event bus	eventBus.js extends Node's EventEmitter; audit and notification events are not durable.	Observer pattern selected in ADR-005 to decouple side-effects; persistent queue rejected as over-engineering.	If the process crashes after a DB commit but before listeners execute, the email/audit event is lost (RSK-019).	Core business operation (DB commit) is not rolled back; failures are logged.	Introduce a transactional outbox table in PostgreSQL to guarantee at-least-once delivery for audit and notification events.
TD-07	Architecture & Design	Single-instance deployment baseline	The application is designed as a single-instance modular monolith; horizontal scaling and external session storage are deferred.	Deliberate architecture decision (DEC-009, DEC-015) to keep operational complexity proportionate.	Cannot scale horizontally without moving sessions out of process memory.	Single instance is sufficient for the expected low-to-medium workload.	Re-evaluate only when workload evidence shows the single instance is insufficient; then externalise session storage and consider multiple instances.
TD-08	Test	Mocked DB in unit tests	Unit tests mock pg and the model layer; SQL correctness is not verified by unit tests.	Unit test isolation strategy.	SQL regressions are only caught by integration tests (which require DATABASE_URL).	Integration tests run against real PostgreSQL in CI when the secret is configured.	Maintain integration coverage for all new SQL; avoid adding untested raw queries.
TD-09	Test	Performance testing kept	Due to time constraints, the team focused on	Deliberate delivery-time trade-off to preserve the M3	Manual execution is required; future	PT-001 test script, recorded execution results and	Review implementation and integrate through a reviewed PR. Status:

		outside staging	deploying CivicConnect's core functionality. PT-001 has been implemented and executed on a dedicated branch from staging, but is not integrated into staging verification or CI.	staging baseline. subsequent Render-hosted PT-001 passed with p95=453.98 ms and 191/191 persisted. The change has not yet been integrated into staging as of yet.	performance regressions may not be detected automatically.	persistence reconciliation.	Open, integration deferred.
TD-10	Infrastructure & Deployment	SMTP blocked on Render free tier	Outbound SMTP ports 25/465/587 are blocked on Render Free. Gmail SMTP cannot deliver from staging.	Free-tier hosting restriction (RSK-016, Issue #124, urgent).	Requesters receive no email notifications in staging.	Email flow verified locally with Ethereal and Gmail; failures do not roll back DB transactions.	Upgrade to a paid Render plan, switch to an HTTPS transactional email API, or select a host that permits SMTP.
TD-11	Infrastructure & Deployment	No production monitoring/observability	Only /health exists. No centralised logging, alerting or uptime monitoring.	Observability excluded from M2/M3 CI scope.	Failures in production would not be detected proactively.	/health endpoint and host-level logs.	Add structured logging, error alerts and uptime monitoring after production host selection.
TD-12	Infrastructure & Deployment	Neon cold-start latency	Neon Free scales to zero after inactivity; first request may take 1–3 s.	Serverless PostgreSQL free-tier behaviour (RSK-018).	First page load after idle may exceed NFR-201 target.	connectionTimeoutMillis: 10000; /health can warm the instance.	Monitor cold-start latency in staging; consider keep-alive pings or an always-on database instance if it degrades UX.

TD-13	Security	Password reset not implemented	Users cannot recover a forgotten password; no token-based reset flow exists.	Feature deferred (Issue #123, high).	Users are locked out; support burden increases.	Login page links to a placeholder reset page.	Implement the full token-based reset flow with rate limiting and single-use expiry.

13.10 Performance and Reliability (Assessment Area F)

13.10.1 Performance Test Purpose and Scope

A controlled performance exercise was conducted during Milestone 3 to evaluate CivicConnect's behaviour when several authenticated requesters submit municipal service requests concurrently. In keeping with Assessment Area F, the team selected one performance-sensitive operation rather than attempting enterprise-scale benchmarking or testing unrelated functions.

Service request submission was chosen because it is the central business operation specified by FR-101. Requesters depend on prompt confirmation that an issue has been captured. Excessive delay may create uncertainty, lead to repeated attempts and impair perceived reliability. The exercise therefore assesses submission response time, correct handling of concurrent requests and successful persistence to PostgreSQL.

Test characteristic	Recorded scope
Test ID and name	PT-001 — Concurrent Service Request Submission
Test type and tool	HTTP-level performance/load test; Grafana k6 v2.2.0
Operation	Authenticated requester submission — POST /requester/submit
Requirements	FR-101 (primary); FR-105 (supporting authentication); NFR-201 (performance)
Execution date	16 October 2026
Assessed application	CivicConnect on Render staging
Overall outcome	PASS for the selected operation and documented hosted workload

The one-user sanity run and five-user full run are two workload configurations of the same scenario, not two separate performance scenarios. The test was intended to generate manageable, credible M3 evidence rather than determine the platform's maximum capacity.

13.10.2 Performance Criteria and Test Design

NFR-201 in Section 3.2 specifies that 95% of standard API read and write requests must complete in less than 800 milliseconds under normal system load. PT-001 applies this numerical criterion to the submission POST using the custom k6 response-time metric pt_submission_ms and threshold p(95)<800.

A p95 of 453.98 ms means that approximately 95% of measured submission responses completed within that duration. The requirement does not demand that every individual response be below 800 ms. Because "normal system load" is not formally quantified in the PED, the five-user configuration is a justified assessment workload, not an established system-wide operational load profile.

Evaluation dimension	Expected evidence
Response-time criterion	Custom submission p95 below 800 ms
Submission validity	At least one recorded attempt and zero detected submission failures
HTTP result	Expected 302 response with redirect to /requester/my-requests
Data integrity	Successful responses reconciled with distinct persisted database records
Completeness	No unexpected, interrupted or unaccounted attempts

The nonzero-attempt and zero-error checks are validity gates specific to PT-001, not additional formal NFRs. HTTP success is initially classified by status and redirect, but database persistence is verified separately; an accepted HTTP response alone does not prove that the corresponding request was saved.

13.10.3 Test Implementation and Application Workflow

The standalone test is maintained at tests/performance/request-submission.js, with instructions in tests/performance/README.md. It uses Grafana k6 independently of the project's Jest suites and is not included in the ordinary GitHub Actions CI gate. Environment variables provide the approved

endpoint, designated accounts, category, run ID, workload settings and execution safety confirmations. Existing authentication, authorisation, CSRF checks and business logic remain enabled.

Stage	Test behaviour
Configuration	Validate approved target and essential parameters.
Preparation	Authenticate each requester, retrieve the submission form and capture the session cookie and CSRF token.
Measured workload	Submit unique synthetic service requests, classify responses and record custom metrics.
Reporting and verification	Export k6 summaries, reconcile database records and retain separate execution metadata.

The test follows the application's actual sequence: POST /auth/login; GET /requester/submit; retrieve CSRF token and confirm an active category; then send a multipart form to POST /requester/submit. The payload includes an active category UUID, unique title, synthetic description and location, MEDIUM priority and _csrf. Attachments are excluded.

The submission passes through the existing requests route, controller, service and model, including input validation, category lookup, database connection acquisition and transaction-controlled insertion. Request-number allocation uses a year-specific PostgreSQL advisory transaction lock. Concurrent requests can therefore exercise transaction and connection contention, though the test does not independently quantify either mechanism.

Automatic HTTP redirect following is disabled, so the 302 response and Location header are checked directly. Authentication and form preparation are excluded from the custom submission latency metric. The test uses direct HTTP calls: it does not evaluate browser rendering, the redirected page or end-user device behaviour.

Each virtual user represents a separate authenticated requester session. During measured iterations, it creates a unique synthetic request, submits it, records the outcome and waits approximately one to two seconds before repeating.

13.10.4 Hosted Environment, Workload and Data Preparation

CivicConnect ran on the existing Render staging service at https://civicconnect-staging.onrender.com, connected to the approved Neon Staging PostgreSQL database. k6 executed on a Windows test computer and sent HTTPS requests to Render. The test computer was the load generator, not the application host.

Environment element	Recorded configuration
Render service / plan / region	civicconnect-bugbusters / Free / Virginia; one configured application instance
Deployed branch and application SHA	staging — f6ecc481b3819eb5edb1fa097453bba70c7b645f
Separate test-script branch and SHA	staging_test/performance_test — 87c7210dddefed7791102ece8467b0ce887e5ab0
Database	Neon Staging; civicconnect_dev; AWS US East (Ohio)
Neon configured limits	0.25–2 CU; configuration, not observed utilisation
Test runner	Windows 10.0.26200 x64; 8 logical CPUs; approximately 20 GiB RAM
Tools / runner runtime	k6 v2.2.0; Node.js v20.15.1 reported for test computer, not Render
Initial database contents	242 service requests and 72 users before hosted preparation
Synthetic accounts and attachments	Five new designated requester accounts; no attachment uploads

The Render deployment and external test script have different commit identifiers because the load generator is maintained separately from the deployed application. The recorded deployed SHA identifies the staging release candidate being exercised. The report records no deployment, restart, environment-variable change, schema change or object-storage change for this test.

Before the workload, the team confirmed the hosted service and database binding, selected an active category and created five synthetic requester accounts through the normal hosted registration endpoint. The accounts used example.invalid addresses; generated credentials were not placed in source control or evidence. The subsequent account records were verified in Neon Staging.

Parameter	Sanity check	Full workload
Virtual users	1	5
Ramp-up	1 second	10 seconds
Steady load	5 seconds	60 seconds
Ramp-down	1 second	10 seconds
Planned workload	7 seconds	80 seconds
Pacing	Approximately 1–2 seconds	Approximately 1–2 seconds
HTTP timeout	30 seconds	30 seconds

The k6 ramping-vus executor uses a closed-loop workload: each virtual user waits for a submission to finish before initiating another. Accordingly, achieved throughput can fall if responses slow. Active iterations are allowed to finish gracefully, which means total elapsed test time may exceed the planned workload stages.

A preliminary hosted health check returned successfully after approximately 22.5 seconds, before measured submissions and account preparation. The submission results therefore relate to a prepared service; they do not demonstrate the responsiveness of a cold or sleeping instance.

13.10.5 Test Data, Execution Results and Verification

Each submitted title was uniquely tagged using PT001-<runId>-vu<virtual-user-number>-i<iteration-number>. This permits run-level reconciliation, identifies the originating virtual user and avoids the application's short duplicate-submission guard. Following each workload, verification checked persisted row count, unique titles, distinct request references, requester ownership, category, iteration sequences and agreement with HTTP outcomes.

Both workloads were executed on 16 October 2026. The sanity workload provided an initial functional check; the five-user full workload supplied the principal M3 performance result.

Measurement	Hosted sanity	Hosted full run
Run ID	render-sanity-20261016115053593	render-full-20261016115053593
Overall k6 duration	11.38 s	90.13 s
Attempted / successful submissions	4 / 4	191 / 191
Failures / errors	0 / 0%	0 / 0%
Average response time	462.11 ms	353.62 ms
Median response time	345.89 ms	329.98 ms
p90 response time	681.89 ms	396.75 ms
p95 response time	752.43 ms	453.98 ms
Maximum response time	822.96 ms	824.84 ms
Overall successful submissions/s	0.3514	2.1193
Persisted records verified	4	191
Distinct titles / references	4 / 4	191 / 191
Unaccounted attempts	0	0
k6 exit code	0 (PASS)	0 (PASS)

The full run completed 191 submissions without detected failures. Each successful HTTP outcome corresponded to a unique, verified PostgreSQL record. No unaccounted requests were reported, and the recorded validation, authentication, authorisation/CSRF, rate-limit, server, timeout, transport and unexpected-response counters were zero.

Full-run criterion	Recorded evidence	Result
Attempts / HTTP behaviour	191 accepted submissions; expected redirect	PASS
Submission failure gate	0 failures	PASS
Database reconciliation	191 distinct persisted requests	PASS
No unaccounted attempts	0	PASS
NFR-201 submission p95 below 800 ms	453.98 ms	PASS

The full-run p95 of 453.98 ms was 346.02 ms below the 800 ms target (approximately 56.7% of that threshold). Some individual responses were above 800 ms—the maximum was 824.84 ms—but the defined criterion is percentile-based, not a hard limit on every request. The sanity run also passed, although four observations provide only limited percentile evidence.

The overall throughput value of 2.1193 successful submissions per second divides successful submissions by total k6 elapsed duration, including setup and graceful completion. It must not be described as steady-state throughput.

13.10.6 Engineering Interpretation, Resource Evidence and Limitations

PT-001 demonstrates that, under the recorded Render-hosted five-user workload, CivicConnect maintained submission correctness and PostgreSQL persistence while meeting the selected NFR-201 latency criterion. It provides credible evidence for the core requester submission operation on the identified staging deployment, supporting limited M3 release-readiness claims for that operation.

Render resource data were sampled at 30-second resolution over a window covering preparation, execution and surrounding time. The largest returned CPU sample was 0.08975 CPU units against a reported 0.15-CPU limit (approximately 59.8%); the largest returned memory sample was approximately 62.45 MiB against a reported limit of approximately 512 MiB. These are maxima among returned samples, not continuous peak measurements.

The requested Render HTTP request-count, latency and instance-count series returned no data points; the submission response-time and throughput measurements therefore come from k6. Neon utilisation was not independently recorded. These samples cannot prove sustained available capacity or identify resource bottlenecks.

The outcome has explicit boundaries: it covers one authenticated requester operation, five concurrent users, no attachments, no browser rendering and a short test interval. It does not establish maximum capacity, long-term availability, notification delivery, complete system-wide NFR-201 compliance or production-scale municipal readiness. The NFR's "normal system load" remains undefined. Closed-loop pacing, resource constraints, network and database state may influence the observations.

The approximately 22.5-second pre-workload health check is a relevant separate readiness concern, but it was excluded from measured submission latency. The performance pass must therefore not be presented as evidence of acceptable cold-start responsiveness. Likewise, success under a selected workload is not a guarantee that every future run will pass.

13.10.7 Evidence, Traceability, Technical Debt and Release Conclusion

The executable test and its instructions are maintained at tests/performance/request-submission.js and tests/performance/README.md. Hosted execution evidence is recorded in tests/performance/evidence/. The following artefacts are identified in the Codex execution record:

Artefact	Purpose
render-20261016115053593-execution.json	Environment, deployed revision, run outcomes and reconciliation metadata
render-sanity-20261016115053593-summary.json	Sanity k6 measurements and thresholds
render-sanity-20261016115053593-persistence.json	Sanity PostgreSQL reconciliation
render-full-20261016115053593-summary.json	Full-run k6 measurements and thresholds
render-full-20261016115053593-persistence.json	Full-run PostgreSQL reconciliation
render-20261016115053593-resources.json	Render resource observations and deployment verification
request-submission-evidence.md	Narrative test record and interpretation

The raw k6 summaries do not themselves query PostgreSQL and can retain pending-persistence wording. Subsequent verification is recorded in the separate persistence exports and execution metadata rather than by rewriting raw k6 output. Only verified and accessible artefacts should be linked from the final PED.

Requirement	Evidence and scope	Recorded status
FR-101	191 successful hosted submissions and database reconciliation	PASS for the exercised submission workflow
FR-105	Valid requester authentication and sessions as test preparation	Supporting evidence; not a comprehensive FR-105 test
NFR-201	Five-VU Render-hosted submission p95 of 453.98 ms versus 800 ms	PASS for this operation and workload only

The performance script was developed on staging_test/performance_test, separate from the deployed staging release candidate. Its integration into regular staging/CI verification was deferred pending review and possible scope refinement. This is already recorded in the PED as TD-09 (Section 13.9). The debt concerns repeatability and integration, not the absence of an executed performance exercise. Any future PR should follow the team's-controlled review and quality-gate process.

The hosted test created five synthetic requester accounts and 195 synthetic requests across its two executions; no automatic cleanup was performed. These records should remain identifiable and should be removed only through an approved, traceable cleanup procedure when no longer required for evidence.

14. Staging / Test Deployment

14.1 Purpose and rationale

The purpose of the CivicConnect staging environment is to provide a controlled, externally accessible deployment of the integrated release candidate project, separate from everyday development activities. For Milestone 3, the objective is not to establish a production-ready municipal service, but to demonstrate that an identifiable application version can be deployed with controlled configuration and verified in an environment that stages real-world operation.

The team selected **Render's free Node.js web service**, connected to the dedicated GitHub staging branch, with **Neon PostgreSQL** providing persistent database storage and **Neon Object Storage** managing uploaded attachments. This approach was chosen to satisfy the project's low to no-cost

infrastructure requirement while maintaining compatibility with the existing Node.js and Express architecture. Render also provides GitHub integration, managed HTTPS and an externally accessible URL without requiring the team to provision or maintain its own server.

Although M3 permits a simulated staging environment, the team deliberately chose a cloud-hosted deployment to simulate a real world deployment better and gather better integration and verification evidence. Unlike local or in-process testing, this environment exposes deployment-specific concerns such as reverse-proxy behaviour, secure session cookies, environment-variable configuration, external service connectivity and hosting restrictions.

The decision involved a deliberate trade-off between **cost, deployment realism and operational capability**. Render's free tier introduces limitations, including reduced resources, potential startup delays and restrictions on outbound SMTP connections that prevent the current email notification service from functioning in staging. Although the Mail service works 100% in our own local environment, the team accepted these limitations when deploying on render, because they do not prevent verification of the application's core service-request workflows. However, they remain documented constraints and would require mitigation before CivicConnect could be considered suitable for operational municipal use.

14.2 Deployment architecture and configuration

The deployment boundary is deliberately simple. The modular monolith serves all web routes and EJS views in one Node.js process. PostgreSQL and object storage are managed externally through Neon. This avoids additional inter-service deployment complexity for a three-person team, while retaining module separation in the codebase. Importantly, the application instance must remain replaceable: persistent request records belong in PostgreSQL and attachments belong in object storage, not on Render’s temporary filesystem.

A successful build alone is not proof of a working deployment. Operational checks must distinguish dependency installation and startup from application-to-database connectivity, HTTPS/session behaviour and complete user workflows. The /health endpoint supplies one technical check, but it is not a substitute for functional verification or a release identifier.

Component	Staging technology	Responsibility
Web hosting	Render free Web Service	Runs Node.js/Express modular-monolith application
Frontend	EJS and Tailwind CSS	Server-rendered views and styling
Database	Neon PostgreSQL	Persistent users, requests, departments and audit data
Attachments	Neon Object Storage	S3-compatible external object storage
Source control / CI	GitHub / GitHub Actions	Branch control, PRs and automated checks
Notifications	Nodemailer / SMTP	Email integration; outgoing SMTP restricted on free Render

Render is linked to the repository Waldo-Blom/SEN381_Bugbusters and configured to deploy from the staging brach. Render checks out the selected revision, installs dependencies and starts the server. The application uses the npm start command to run. Its prestart script builds Tailwind CSS before src/server.js runs. The server reads the Render-supplied PORT, checks database connectivity on startup, and exposes /health for application/database health checks.

Setting	Configured/reported value
Service / runtime	Render Web Service / Node.js

Setting	Configured/reported value
Compute plan	Free
Repository	Waldo-Blom/SEN381_Bugbusters
Source branch	Origin/Staging
Node version	24 (.nvmrc)
Build command	npm ci --include=dev && npm run build:css
Start command	node src/server.js
Health endpoint	/health
NODE_ENV	staging (runtime mode, not business production)
Auto-deploy	On Commit
Staging URL	https://civicconnect-staging.onrender.com/

14.3 Environment variables and secret control

With local deployment developers use a private .env file, when it comes to staging, staging values are configured in Render environment settings. Secrets must not be committed to GitHub or disclosed in screenshots. The .env.example file documents keys but contains no production credentials. Setting NODE_ENV=production allows production-mode cookie/proxy behaviour to be exercised in staging.

Variable(s)	Purpose
NODE_ENV; PORT	Runtime behaviour; hosting-assigned listening port
DATABASE_URL	Neon PostgreSQL connection string for the selected staging resource
SESSION_SECRET	Private session-cookie signing secret
AWS_ACCESS_KEY_ID; AWS_SECRET_ACCESS_KEY	Credentials for S3-compatible Neon Object Storage
AWS_REGION; AWS_ENDPOINT_URL_S3; STORAGE_BUCKET	Object storage region, endpoint and bucket
EMAIL_PROVIDER; EMAIL_USER; EMAIL_PASS	Notification provider/account credentials
EMAIL_HOST; EMAIL_PORT; EMAIL_FROM	SMTP host/port and optional sender address

AWS-prefixed names identify the S3-compatible API, but they do not imply a separately managed AWS bucket. The staging branch isolates source-code promotion, but database and attachment isolation also require separate Neon resources and credentials

14.4 Hosted end-to-end/system verification

The selected cross-role journey is risk-based rather than a collection of disconnected screen checks. It traverses the central CivicConnect purpose, citizen/requester submission, manager assignment, staff action and requester follow-up, while exercising a sensitive authorisation decision. It is particularly valuable because the new contact-access workflow requires the identity of the current assignee, department ownership and manager decision to agree. A mistake could expose personal information even while normal request handling appears functional.

Data and expected outcomes should be established before test execution. Use dedicated staging accounts for requester, manager, staff and unauthorised user. Capture the request identifier, attachment reference and start state; verify that the submitted record exists after creation, that contact fields are concealed before approval, and that approval does not grant permission to unrelated staff. Keep the observed result, date and deployed SHA linked to the test record. If a step fails, log a defect and do not mark the complete journey as passed.

The locally automated E2E suite and the hosted manual journey provide different evidence. The in-process suite can detect regressions in application logic repeatably, while the hosted journey verifies deployment integration and configuration. Neither automatically proves production-scale reliability. Notification email should have a separate result of unavailable/blocked due to the known hosting restriction, not be silently omitted from the overall quality assessment.

Automated in-process E2E tests with mocked storage do not by themselves prove the public Render system works. A critical journey must therefore be run through the deployed staging URL and recorded with expected and actual results. The planned journey is:

Automated in-process E2E tests with mocked storage do not by solely prove the public Render system works. A critical journey must therefore be run through the deployed staging URL and recorded with expected and actual results. The planned journey is:

- Requester signs in using normal authentication and creates a service request with an attachment.
- Responsible manager signs in and assigns the request to eligible staff.
- Assigned staff member confirms requester contact details are initially hidden.
- Staff requests contact access; manager approves; assignee verifies authorised visibility.
- Staff adds a comment and changes the request status through a permitted transition.
- Requester verifies updated request status/history, an unauthorised user is denied restricted access.

Test field	Evidence to record
Environment / revision	Render staging URL and verified deployed Git commit
Preconditions / test data	Controlled requester, manager and staff accounts; eligible department and attachment
Expected result	End-to-end flow completes; persistence, storage and access restrictions remain correct
Actual result / status	Record observed result: Pass
Execution date / link	10 October 2026

The above steps describe the verification procedure; do not record them as passed until observed on Render. SMTP email delivery is tracked separately as an accepted environment limitation, not silently counted as a successful hosted notification test.

14.5 Staging differences, constraints and residual risks

The most consequential accepted limitation is SMTP. The current Nodemailer transport requires outbound SMTP connections, while Render free web services restrict ports 25, 465 and 587. This means that an email failure in this configuration is not sufficient evidence of a defect in the notification business logic; equally, passing local notification tests is not proof that hosted notifications work. For M3 the team should disclose this difference explicitly and treat hosted notification delivery as unverified, rather than claim complete functional parity.

Cold starts, resource limits and in-memory sessions affect the interpretation of hosted test results. A first request after inactivity may be slower than a warm request; restart or redeployment can log users out; transient capacity restrictions may differ from those of a paid service. These factors should be distinguished from functional defects. Storage must remain external so that deployment replacement does not destroy attachments. Any shared Neon data resources also increase the risk of test activity altering development data.

Difference / limitation	Effect on interpretation
HTTPS and reverse proxy	Hosted secure cookies require appropriate trust-proxy handling; local HTTP is not equivalent

Difference / limitation	Effect on interpretation
NODE_ENV=production	Changes session and other runtime behaviours
Free-tier spin-down/cold starts	Initial response after inactivity may be slower
Limited compute resources	Results are not representative of sustained municipal demand
In-memory session storage	Users may need to sign in again after restart/redeployment
Ephemeral application filesystem	Persistent attachments belong in external object storage
SMTP egress on ports 25, 465, 587	Render free service blocks these ports; Nodemailer/Gmail SMTP notifications cannot be relied upon
Possible shared Neon resources	Staging tests could influence development data if resources are not isolated

The team selected the free service despite SMTP limitations because the objective was a working, accessible staging candidate at no hosting cost. Email notifications remain a known functional limitation of this deployed configuration. Successful local SMTP testing is not evidence of successful delivery from Render. The limitations are reflected in the Risk Register and Technical Debt Register as appropriate.

14.6 Path to real-world deployment

For real operational use, the core modular monolith can be kept but, in the future, it may be better to take a microservices approach. A more proportionate next step is to maintain the architecture while upgrading operational controls: a reliably available paid service, separate production resources, persistent sessions, secret rotation, monitored database and object backups, release-tag promotion, migration review, structured logging, alerting and scheduled recovery testing. This protects the existing design investment while addressing the actual limitations observed in staging.

Before municipal adoption, representative users and administrators should validate core workflows and access rules against acceptance criteria. Performance testing should use realistic request volume and data size. Failures should be investigated rather than concealed by overprovisioning. Notification delivery requires either supported SMTP egress or a transactional email HTTPS API, with retry/error reporting and delivery monitoring. POPIA-sensitive data should not be copied to demonstration environments without appropriate safeguards. Bata testing should also be done before the final Public release.

Operational concern	Recommended production improvement
Availability / capacity	Move to appropriately sized paid/managed hosting and monitor service availability
Notifications	Use supported SMTP connectivity or an HTTPS transactional email API
Sessions	Use persistent shared server-side session storage
Isolation	Separate development, staging and production databases, buckets and credentials
Release control	Promote approved tagged commits after CI and staging evidence
Database evolution	Maintain version-controlled migrations and compatibility checks
Protection / recovery	Implement secure backups, tested restores and a rollback playbook
Monitoring	Centralise application logs, error alerts, performance and delivery monitoring
Validation	Conduct security, acceptance and representative load tests before rollout

The current modular monolith architecture can remain. The major difference is production-grade infrastructure, separation, monitoring and operational controls. The staging deployment demonstrates feasibility within academic constraints but does not establish production readiness.

15 Backup, Rollback and Recovery Plan

15.1 Scope and implementation status

This section proposes how CivicConnect would protect and recover application data, uploaded attachments, source code, deployments and runtime configuration in a real operating environment. The project's financial and academic constraints do not justify a complete production backup and disaster-recovery implementation during M3. The measures below are proposed operating procedures, not claims that automated backups or full restores have already been configured and validated.

Please note the following is an engineering proposal for a future operational service, not a claim that independent daily exports, backup storage, regular restore drills or guaranteed recovery times already exist. This distinction matters under M3: the team is required to articulate a simple, defensible rollback/recovery approach, not purchase or operate enterprise disaster-recovery services. Unimplemented safeguards remain residual risks in the release assessment.

15.2 Database and attachment backup and recovery

15.2.1 Database protection

Neon PostgreSQL stores CivicConnect's structured application data, including user accounts, service requests, assignments, comments, contact-access decisions and audit records. To protect this information against accidental deletion, corruption and database failures, a real-world deployment would combine Neon's native point-in-time recovery capabilities with independent PostgreSQL backups.

These approaches address different recovery scenarios. Neon's history-based recovery could enable faster restoration of recent changes within the available retention period, while independent pg_dump exports would provide an additional recovery option if the original Neon database project became unavailable. Independent exports would include both the database schema and stored records and would be securely stored outside the primary Neon project.

The availability of Neon's recovery features depends on the subscribed plan and branch configuration and would need to be verified before operational use. Independent backups would require automated scheduling, encryption, restricted access, retention management and monitoring to ensure they are created successfully and remain recoverable.

Proposed Backup Frequency and Retention

Backup type	Frequency	Retention	Purpose
Independent database backup	Daily	30 days	Recover from recent data loss or database corruption
Long-term database backup	Monthly	3 months	Provide additional historical recovery options
Pre-migration backup	Before significant schema changes	Until the related release is verified	Protect against data loss during database migrations
Neon native recovery	Provider-managed history, where supported	According to the Neon plan and branch configuration	Recover earlier database states within the available history window

These intervals represent proposed operational targets and would need to be reviewed against organisational requirements, storage costs and applicable data-retention policies

15.2.2 Database recovery procedure

The recovery approach would depend on the nature and extent of the incident. For isolated data loss, such as accidentally deleted service requests, selective recovery from a historical database state or backup would be preferred to preserve legitimate transactions made after the incident. For widespread

corruption or significant data loss, a full database restore may be necessary, although this could result in the loss of valid transactions created after the selected recovery point.

1. Contain the incident: Prevent further harmful database changes where necessary and preserve the current database state and relevant logs.
2. Assess the damage: Identify affected records or tables and select an appropriate recovery point or independent backup.
3. Restore in isolation: Recover the required data into a separate recovery environment before modifying the live database.
4. Select the recovery method: Prefer selective record recovery for smaller incidents and full restoration where extensive damage makes it necessary.
5. Verify integrity: Validate important records, relational relationships, service-request functionality, permissions and application compatibility.
6. Restore service: Reconnect the application only after successful verification and document the recovery decision and any lost transactions.

Recovery validation must extend beyond successful SQL execution. The restored data must support the application's actual business workflows and preserve its security restrictions. The database schema must also remain compatible with the selected application version, while attachment references must correspond to files available in Object Storage.

15.2.3 Attachment backups and consistency

CivicConnect uses Neon Object Storage for uploaded attachments, while PostgreSQL maintains the records linking those files to service requests. Because these components are stored separately, restoring the database does not automatically restore the corresponding attachments.

In a real-world environment, the proposed approach would involve daily independent copies of uploaded files to protected backup storage. These backups would preserve original object keys and maintain a manifest containing file identifiers, sizes and checksums to support integrity verification.

Database and attachment backups would need to be coordinated to prevent inconsistencies, such as restored database records referencing missing files. This could be achieved by temporarily pausing relevant writes and uploads during a consistent backup or implementing a reconciliation procedure for changes occurring during the backup process.

During recovery, restored attachment references would be checked against the available objects, and authorised file access would be tested through the application. Unmatched files would be retained until their status had been investigated to avoid accidental data loss.

15.2.4 Schema/version compatibility

Database recovery must also account for compatibility between the restored schema and the application version. For example, CivicConnect's contact-access approval functionality introduced the request_contact_access_requests table and related permission fields. Restoring a database to a point before these structures existed could cause the current application to fail or prevent the contact-access workflow from functioning.

Conversely, rolling back the application does not automatically reverse database changes, and removing newer tables could destroy valuable access decisions or audit history.

To manage this risk, a real-world implementation would maintain version-controlled database migrations and link each application release to its compatible schema version. Non-destructive corrective migrations would be preferred where practical to preserve valid data while resolving compatibility issues.

15.3. Codebase, application and deployment recovery

15.3.1 GitHub and Source Code Protection

GitHub provides CivicConnect's primary mechanism for source code protection and version control. Its commit history, pull requests and release references allow developers to identify previous application versions and reverse changes that introduce defects.

However, Git history alone does not constitute a complete backup strategy. Repository deletion or loss of access could affect the team's ability to retrieve code, while Git backups do not automatically preserve GitHub issues, pull-request discussions, repository settings or secret values.

A real-world strategy would therefore maintain independent, dated Git mirror backups outside the primary GitHub account. Stable application releases would also be identified through Git tags and associated deployment records. Sensitive credentials would be maintained separately in a secure secrets-management solution rather than included in repository backups.

15.3.2 Application rollback

An application rollback would primarily address failures introduced by a new software release. Before initiating rollback, the responsible engineer would determine whether the problem originates from application code, deployment configuration or an external dependency such as Neon PostgreSQL. If a code regression were confirmed, the preferred approach would be to restore the last verified working deployment rather than unnecessarily recovering the entire database.

1. Identify the failed Render deployment and its associated Git commit.
2. Locate the last verified working release and confirm compatibility with the current database schema.
3. Restore the selected version using Render's rollback capabilities, where available, or redeploy its recorded Git revision.
4. Verify application startup, database connectivity and critical functionality.
5. Confirm that authentication, service requests, department restrictions, attachments and reporting operate correctly.
6. Record the recovery outcome and introduce a reviewed revert or corrective pull request to keep the deployment branch consistent with the recovered version.

The team must consider that restarting or redeploying the application will clear existing in-memory sessions, requiring users to sign in again. A previous deployment should only be selected as a rollback target if it has been verified as compatible with the hosted configuration. This is particularly relevant to CivicConnect because staging introduced changes to secure-session behaviour behind Render's reverse proxy.

15.3.3 Configuration recovery

Recovering a working application version is insufficient if the required environment configuration is unavailable. CivicConnect depends on configuration values for Neon PostgreSQL, Object Storage, session security, email integration and application runtime settings.

In a real-world deployment, the team would maintain a documented inventory of required environment variables, secure copies of sensitive credentials and instructions for recreating the hosting configuration.

If the Render service became unavailable, a replacement service could be configured using the selected Git release, locked dependency versions, documented build/start commands and securely stored environment settings. The recovered application would then be reconnected to the validated

database and storage services. Where an incident involved possible credential exposure, affected secrets would be rotated rather than simply reused.

15.4. Whole-System Recovery and Risk Management

15.4.1 Coordinated System Recovery

Certain incidents may affect multiple CivicConnect components simultaneously. In these cases, restoring individual services independently could produce an inconsistent system, even when each component appears operational. A coordinated recovery process would therefore align the application release, database recovery point, uploaded attachments and deployment configuration.

1. Containment: Identify the incident, suspend harmful operations where necessary and preserve relevant logs.
2. Recovery planning: Determine the affected components and select compatible application, database and attachment recovery points.
3. Data recovery: Restore affected records and attachments into an isolated environment and verify their consistency.
4. Application recovery: Restore the matching application release and required deployment configuration.
5. Integrated verification: Test database connectivity, authentication, service-request processing, access restrictions, attachments and reporting.
6. Return to service: Restore availability once the recovered system has been verified, then monitor for further errors.
7. Incident review: Document the cause, recovery actions, potential data loss and required corrective measures.

Recovery planning must also consider actions that cannot be reversed through database restoration. For example, email notifications may already have been delivered, while historical database recovery could remove more recent audit records. Such evidence should be preserved where necessary, and external actions should not be replayed without verification.

15.4.2 Proposed Recovery Objectives

In a real-world environment, recovery targets would help define acceptable service interruption and data loss. The Recovery Point Objective (RPO) represents the maximum acceptable period of data loss, while the Recovery Time Objective (RTO) defines the target duration for restoring service.

Objective	Proposed target	Engineering consideration
Database RPO	Up to 24 hours	Assumes successful, recoverable daily backups; more recent Neon history may reduce data loss
Full-system RTO	Within 4 hours	Depends on the incident scope, backup availability and recovery complexity
Application rollback	Within 1 hour	Assumes a verified, compatible application release is available

These are preliminary planning objectives, not demonstrated service-level guarantees. Before operational adoption, they would need to be validated through recovery exercises and adjusted according to business requirements, infrastructure capabilities and available resources.

15.4.3 Recovery Risks and Mitigations

The effectiveness of a recovery strategy depends on more than the existence of backup files. Backups must be protected, monitored and verified to ensure they can be used when required.

Risk	Potential consequence	Proposed mitigation
Failed or inaccessible backup	Required data cannot be recovered	Monitor backup execution and periodically test restoration
Inconsistent database and attachments	Missing files or invalid references	Coordinate backup points and reconcile attachment manifests
Application/schema incompatibility	Recovered application fails	Maintain versioned migrations and release compatibility records
Missing configuration or credentials	Application cannot reconnect to services	Maintain protected configuration records and secrets
Outdated recovery point	Loss of valid recent activity	Select the safest recovery point and prefer selective restoration where possible
Unauthorised backup access	Exposure of requester information	Apply encryption, least-privilege access and retention controls
Untested recovery process	Recovery targets cannot be achieved	Conduct periodic isolated recovery exercises

Because CivicConnect processes requester information, backup storage and recovery access would also need to comply with applicable privacy and data-protection requirements, including POPIA.

15.4.4 Recovery Responsibilities and Verification

In an operational environment, responsibility would be assigned for backup monitoring, recovery execution and independent verification. Regular recovery exercises would confirm that backups remain usable and that the proposed recovery objectives are realistic. Each exercise would record the selected backup, application version, recovery duration, data-loss interval, verification results and any corrective actions required.

Verification would focus on whether the recovered application can perform its intended business functions.

Verification area	Expected result
Application health	Application starts and connects to PostgreSQL
Authentication	Authorised users can sign in successfully
Service requests	Requests can be created, viewed, assigned and updated
Department restrictions	Users remain restricted to authorised departmental information
Contact-access approvals	Requester contact details remain protected by approval and assignment rules
Attachments	Files remain accessible to authorised users
Reporting	Management dashboard retrieves consistent information
Audit history	Available audit records remain consistent with the recovery point

These checks provide stronger assurance than simply confirming that a database backup was successfully imported or that the application process started.

15.5. Milestone 3 Scope and Release-Readiness Implications

The Milestone 3 requirements call for a credible rollback and recovery approach rather than a fully implemented enterprise disaster recovery solution. Given the project's financial and academic constraints, independent automated database exports, attachment backup infrastructure and comprehensive recovery exercises are proposed for future operational deployment rather than implemented during M3.

The existing GitHub version history and Render deployment records provide a foundation for application-level recovery. However, reliable recovery of application data and attachments would require the additional safeguards described in this plan.

The absence of implemented and independently verified full-system recovery capabilities therefore remains a residual operational risk and should be reflected in the PED release-readiness assessment and relevant Risk or Technical Debt Registers.

15.6. Conclusion

The proposed strategy combines controlled GitHub releases, Render deployment rollback, Neon database recovery capabilities, independent database and attachment backups, secure configuration management and coordinated verification. Together, these measures provide a practical approach to protecting CivicConnect against data loss, deployment failures and infrastructure disruptions.

Although comprehensive recovery infrastructure falls outside the implemented M3 scope, the plan establishes the engineering controls, dependencies and verification activities that would be necessary before CivicConnect could be considered suitable for reliable real-world municipal operation.

16 Release-Quality Review & Residual Risk - Assessment area G

16.1 Release evidence summary:

Evidence area	Result	Interpretation	Concern / action
Build & CI	**Pass**	The GitHub Actions CI pipeline successfully enforces ESLint, Prettier, Gitleaks (secret scanning), dependency auditing, and runs the full automated test suite (unit, functional, integration, E2E) on all PRs to the testing and staging branches.	Block/fix if mandatory gate fails. No blocking issues.
Automated regression	**Pass**	Critical behaviours (auth, request submission, workflow transitions, department scoping) are protected by 169 unit, 5 functional, and 39 integration tests.	Investigate gaps/failures. Regression suite provides a strong safety net, but minor defects (e.g., operator editing) evaded initial coverage.
API/Integration	**Partial**	Core API and database integrations (PostgreSQL, Neon Object Storage) are behaving correctly. However, SMTP integration fails in the deployed staging environment.	Hosting platform restriction (Render Free Tier blocks SMTP ports 25, 465, 587) prevents email notifications from working in staging.

Evidence area	Result	Interpretation	Concern / action
System/E2E	**Partial**	The critical requester journey is complete and working. However, this is the only e2e system test that we currently have. We would like to implement a staff e2e automated test in the future. Manual e2e testing has however been completed for all the stakeholder journeys.	Additional automated e2e testing required.
Static/security	**Pass**	Gitleaks scan confirms no secrets committed. npm audit shows no high/critical vulnerabilities. Role-Based Access Control (RBAC) and department scoping are verified via integration tests.	Fix, mitigate or record. Security posture is strong; no immediate action required.
Performance	**Pass**	191/191 persisted; p95 453.98 ms against <800 ms NFR-201 threshold	Retain limitations: single operation, five VUs, pre-warmed service, cold-start not measured and TD-09 open.
Known defects	**Open**	5 documented defects/findings (#121 to #125). Issue #122 (Operator cannot edit logged request) and #124 (SMTP email fails on Render) are urgent and impact core workflows.	Block or residual risk. Urgent defects block a "READY" status. Workarounds exist (e.g., manual DB edits, local email testing), but these must be resolved.
Acceptance	**Partial**	Core functionality is demonstrated in the hosted staging environment (e.g., submission, assignment, contact-access approval).	Formal sign-off and final RTM evidence are pending M4. Stakeholder validation of the staging build is required.

16.2 Final Classification & Rationale

Required final classification: CONDITIONALLY READY

Engineering Argument:

CivicConnect is classified as CONDITIONALLY READY. The system architecture, core functional workflows, security controls, and CI/CD pipeline are demonstrably functional, and meet the architectural guardrails established in the baseline (ASR-01 through ASR-05). We have successfully transitioned from the M2 navigational prototype to a fully integrated, database-backed platform.

However, the system is not yet ready for production release due to identified defects and project constraints:

1. Known Defects & Limitations: Five defects remain open, two of which are classified as urgent. Issue #122 (Operator cannot edit their own logged requests) blocks a core operational workflow. Issue #124 (SMTP email delivery on Render) is a known limitation due to the free-tier hosting budget constraints set for this project. While email failures do not roll back core transactions (DEC-012), this is a significant operational limitation for our service-request platform.
2. Infrastructure & Budget Constraints: The selected free-tier hosting environment (Render) restricts outbound SMTP traffic, preventing the Nodemailer/Gmail SMTP integration from functioning in the deployed staging environment. Resolving this requires either migrating to a paid hosting tier.
3. Performance Validation: On 16 October 2026 the Render-hosted PT-001 workload used five concurrent k6 virtual users and recorded 191 successful submitted and persisted requests, no submission errors, and p95=453.98 ms against the 800 ms NFR-201 target. This is a PASS for this endpoint and recorded workload, not all application operations, long-duration conditions or cold-start readiness.
4. Acceptance Criteria: While the system is functionally complete, formal acceptance evidence and stakeholder sign-off are pending M4.

Path to READY:
If the team were granted an additional approximate timeframe of 2 weeks and be allocated a budget of $25 (in order to upgrade to the render paid plan), we believe we could classify the system to READY for release. This timeframe would be allocated to:

- Resolving urgent defects (#122, #124) and closing remaining open issues.
- Migrating to a paid Render hosting tier (or configuring an alternative SMTP provider) to enable and verify email delivery in staging. And properly testing this in a staging environment.
- Integrate PT-001 into a controlled verification workflow through peer-reviewed change.

Until these specific residual risks are mitigated, the CONDITIONALLY READY classification accurately reflects the system's engineering quality, constrained by the deliberate budget limitations and the outstanding defect backlog.

17. GitHub Governance

Link to our repo: https://github.com/Waldo-Blom/SEN381_Bugbusters

GitHub is a tool but just because you make use of GitHub does not mean that you applied proper software engineering. In order to apply software engineering, you need to apply software engineering practices. The following below describes exactly how we will use GitHub throughout the project to ensure these best practices.

17.1 Branches

We will have two main branches that we will use `main` and `dev`. From these two branches we will branch out further into subbranches referred to as feature branches.

M2 review: After research conducted in assignment 3, we introduced a testing branch to run our automated CI checks. All current code resides on testing. We also created a staging branch, but its implementation is deferred to M3 as it depends on our deployment environment. The staging branch currently contains only documentation, though branch protection rules are already in place. main also currently contains only documentation. All code is on testing.

M3 review: During M3, the team decided to stop using the dev branch entirely. The original M1 plan had main and dev, with feature branches merging into dev and then dev merging into testing. This introduced an unnecessary extra integration step and an additional pull request for every change. Moving from dev to testing would have slowed down development and prevented continuous testing and integration. By not utilizing dev, all feature branches now branch directly from testing and merge back into testing via a pull request. This allows every change to be validated by the full CI pipeline before it joins the shared integration branch, which directly supports the team's goal of continuous integration and continuous development. The dev branch is therefore no longer part of the workflow.

According to our project brief the main branch should be treated as a fully protected branch that does not allow commits on the branch unless through a pull request and detailed review. We achieve this through applying a ruleset to the `main` and `dev` branch with the following settings:

- **Requires a pull request before merging**: All commits must be made to a non-protected branch and submitted via a pull request before they can be merged into main. Direct development on main it is therefore not possible.
- **Requires approvals**: Pull requests targeting main require a minimum number of approvals. We have set the required number of approvals before merging to 2 thereby matching the project master brief.
- **Force pushes are blocked**: The "Allow force pushes" setting is disabled, so force pushes to main are not permitted.
- **Deletions are blocked**: When the "Allow deletions" setting is disabled, the main and dev branch can't be deleted (Just to avoid accidental deletion of our production and development branches)
- **Do not allow bypassing the above settings:** This setting just makes sure no matter the role that the above-mentioned settings can be bypassed by admin privileges or roles.

M2 review: The team identified a collaboration risk: requiring 2 independent reviewers on a 3-person team could create bottlenecks if either reviewer is unavailable, potentially stalling progress and leading to larger, riskier merges. The dev branch approval requirement has therefore been adjusted to 1 independent approval to mitigate this risk while maintaining peer review and code quality. The same applies for the testing branch. The staging branch however follows the same rules as main requiring 2 independent reviews.

The testing branch follows the same protection as dev (1 independent approval) and additionally requires the CI status check to pass before merging. The staging branch has branch protection rules in place, but CI integration will be completed in M3 when deployment is established, along with the configuration of the "require deployments to succeed before merging" rule.

For the full ruleset of our repo please see: https://github.com/Waldo-Blom/SEN381_Bugbusters/settings/rules

Branch	Role	Required approvals	Required CI checks	Additional protections
Main	Production / documentation baseline	2 independent approvals	Yes (CI gate)	Force pushes blocked, deletions blocked, no bypassing settings

Branch	Role	Required approvals	Required CI checks	Additional protections
Staging	Release candidate / staging deployment	2 independent approvals	Yes (CI gate)	Force pushes blocked, deletions blocked, no bypassing settings
Testing	Continuous integration branch	1 independent approval	Yes (CI gate)	Force pushes blocked, deletions blocked, no bypassing settings
Feature / fix / docs	Short-lived development branches	None (author's own branch)	CI runs on PR into testing	Branches are deleted after merging

Let's take a practical example:

Waldo was assigned the following task to complete: "Documentation for "Problem and business need"". Now he has to go and make a change on the `problem-and-business-need.md`file (stored in the docs folder). Instead of just working on the branch `main` (which is against the project spec and not good software engineering practices (as you would never develop straight on the live version of compony software)), he instead creates an entire new branch called `docs/waldo-m1-artifacts `. He then in his own local environment makes changes to that mentioned .md file, either through single or multiple commits (whatever fits his use case).

Note: We use naming prefixes (like docs/ and feature/) to categorize the type of work. GitHub utilizes these prefixes by allowing us to filter branches by name, making it easy to see all documentation branches or all feature branches at once, this helps to improve navigation and organization in the repository (Github, 2025).

Here are other examples of feature branches from our naming convention:

- docs/ai-usage-register-m1-waldo
- feature/NFR-improve-login-loading-time
- feature/FR-implement-login

As the project evolves, we might add additional branching prefixes, we will identify where certain changes align and agree on a prefix for that code change type. When a new one is added we will also update it in the `Github-governace.md` file, under the docs folder of our repo.

Note branches get deleted after each milestone, this is done to keep the repo clean and structured, so we don't end up with for example 70 feature branches that are no longer used. But before deleting a feature branch it is first merged with the relevant branch either `main` , `dev` or `testing` depending on the scenario and then that branch absorbs the feature branch commits and changes through the PR, nothing is lost and the commit history is kept, that branch just can be accessed anymore but this is fine.

See our current branches here: https://github.com/Waldo-Blom/SEN381_Bugbusters/branches

17.2 Pull requests (PR's) and Code review process

Marco and Christian then first have to review this change within the pull request, treating the Pull request (PR) as a place to do a code review, meaning they look at Waldo's proposed changes and then comments and gives feedback. After both Marco and Christian approve this change made by Waldo (Waldo can't approve his own changes, he only verifies that his work is indeed correct before opening a pull request). This applies for all members, the scenario above is just for demonstrative purposes.

As per our Ruleset, the main branch requires at least 2 approvals (enforced automatically). We do not allow self-approval on any branch that requires reviews. Any merge conflicts are then resolved (if there are any).

Example PR: https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/2

Also note that in some cases we held code reviews on discord where the author would show the team / team member what they did, showcasing what was changed, how it linked to a specific requirement, how that requirement was met and how the behaviour was tested:

Example PR as evidence: https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/110

17.3 Issues

When referring to issues in GitHub we don't necessarily mean it in terms of a software bug (Github, 2025). This refers to a specific task that has to be completed (Github, 2025). For our project we start by identifying what the deliverables are for a specific milestone and then assign we distribute the workload and assign specific tasks to specific people. But just because one person is in charge of a specific part of the workload does not mean that person is the only one working on that, we look at what that person has done and then review it and incorporate our feedback.

Example issue: https://github.com/Waldo-Blom/SEN381_Bugbusters/issues/3

17.4 Milestones

The milestones on our GitHub page relate to the actual milestones of this project i.e. Milestone 1 on GitHub consists of the issues that are related to Milestone 1 project deliverables.

Every issue must be assigned to only one milestone (M1, M2, M3, or M4). Assigning issues to milestones allows us to track our overall progress, the milestone view automatically shows what percentage is complete (based on closed vs. open issues) and helps us see if we are on track or falling behind.

Link Milestones: https://github.com/Waldo-Blom/SEN381_Bugbusters/milestones

17.5 Labels

We use labels in our PR's and issues just to mark issues more clearly, for example:

- By priority:
  - `priority: Urgent` – Blocking issues, critical bugs etc
  - ` priority: High ` - important but not a blocking issue
  - `priority: Medium ` - normal priority
  - `priority: Low ` - Nice to have, can defer

- By the task type:
    - `Feature` - New functionality added
    - `Documentation` - Improvements or additions to documentation
    - `Bug` - Something isn't working
    - `Enhancement` - Improve something / change an existing behaviour
- By the type of work:
    - `Front end` - UI / Client side changes
    - `Back end` - Server / API work
    - `Database` - Data layer / queries

In the future as the project evolves, we might add additional labels as we see fit in order to provide extra clarity where necessary. Or we might remove some that are not used.

Link to the labels: https://github.com/Waldo-Blom/SEN381_Bugbusters/labels

17.6 CI / Automated Checks

The project uses a GitHub Actions Continuous Integration (CI) pipeline, defined in .github/workflows/ci.yml as a workflow named **CI**. It has grown from the single build-and-test job of M2 into a set of independent jobs covering static analysis, dependency and secret scanning, four automated test suites, a database check, and a final CI gate check.

17.6.1 Triggers and concurrency

The pipeline runs on pushes to dev, testing, staging and main, on pull requests targeting any of those branches, and on manual dispatch. Runs are grouped by workflow and Git ref, and a newer run cancels an older in-progress run for the same ref, so obsolete commits do not consume runner time.

17.6.2 Shared setup

Every job runs on a fresh ubuntu-latest runner and uses a action (.github/actions/setup-node/action.yml). This action:

1. Sets up Node.js using the version pinned in .nvmrc.
2. Installs dependencies with npm ci --include=dev, giving a clean, reproducible install from package-lock.json.
3. Builds the Tailwind CSS (npm run build:css). The dependency audit and database check skip this step because they do not need it.

17.6.3 Jobs

Static analysis (lint)	`npm run lint` (ESLint with Prettier integration) and `npx prettier --check . ` for formatting.
Dependency check (dependency-audit)	`npm audit --omit=dev --audit-level=high`. High or critical vulnerabilities in production dependencies fail the job.
Secret scan (secret-scan)	Gitleaks 8.18.4 scans the full Git history (fetch-depth: 0) for committed secrets, with output redacted.
Unit tests (unit-tests)	Jest unit project (src/**/__tests__/). Database operations are mocked, and DATABASE_URL is a harmless local placeholder.
Functional tests (functional-tests)	Jest functional project (tests/functional/). Black-box requester workflows (registration, login, request submission, lookup and history) run against the real Express app via Supertest, with the model layer mocked.

Neon database check (db-test)	`npm run db:test` connects to the CI Neon database, confirms it is civicconnect_dev, verifies that all 11 expected tables exist in the public schema, and runs basic row-count reads.
Integration and API tests (integration)	Jest integration project (tests/integration/), covering attachments, contact access, health, manager listing, reporting, requests and staff access. Database-backed suites run against Neon. Runs only after `db-test` succeeds.
E2E on simulated staging (e2e)	Starts the server in production mode, polls /health (which runs SELECT 1 against Neon) then runs the Jest e2e project (tests/e2e/) for the critical requester journey. The server log is printed on failure and the server is always stopped afterwards. Runs only after `db-test` succeeds.

Each job has its own timeout (10 minutes, or 15 for integration and E2E). A timeout fails that job and therefore the gate.

The test suites differ in scope. Unit, functional and E2E journey test make use of mock data. Only the integration suite and the database and health checks make use of the real Neon database.

Please note on previous commits there is also a performance test that "ran", it was for a blank folder that was planned to be implemented on staging, for performance testing on staging we rather made use of render for more accurate results in our simulated deployment environment (aka staging branch).

CI gate

The final CI gate job waits for all eight checks and runs with if: always(). It fails if any dependency result is failure, cancelled or skipped, so a skipped job can never count as a pass. A green gate means every check reported success.

17.6.4 Integration with branch protection

The CI gate job is the single required status check for branch protection. This replaces the earlier CI / build-and-test check from M2. Branch-specific review requirements (such as the 1 independent approval on testing described in Section 14.1) apply in addition to the CI requirement, so no change that fails linting, scanning, tests or the database check can be merged into a protected branch. Requiring this check is configured in the GitHub repository settings or ruleset, not in the workflow file itself.

17.6.5 Secret handling

No secrets are committed to the repository, and Gitleaks now enforces this on every run. Two secrets are injected through GitHub Actions repository secrets:

CI_SESSION_SECRET supplies the SESSION_SECRET required by express-session in the unit, functional, integration and E2E jobs.

NEON_DATABASE_URL supplies the CI database connection for db-test, integration and e2e.

17.6.6 Scope and deliberate exclusions

The pipeline is a verification pipeline only. It contains no deployment, release, container build, database migration or artifact-upload job. Performance testing is not part of CI and is handled separately through Render. As with any automated pipeline, a green run shows that the configured checks passed. It does not prove that every behaviour is tested or that the production environment is identical to the testing environment.

17.6.7 Evidence and traceability

The workflow configuration is at .github/workflows/ci.yml and the shared setup action is at .github/actions/setup-node/action.yml. The test suites live in src/**/__tests__/ (unit), tests/functional/, tests/integration/ and tests/e2e/, with Jest projects defined in jest.config.js.

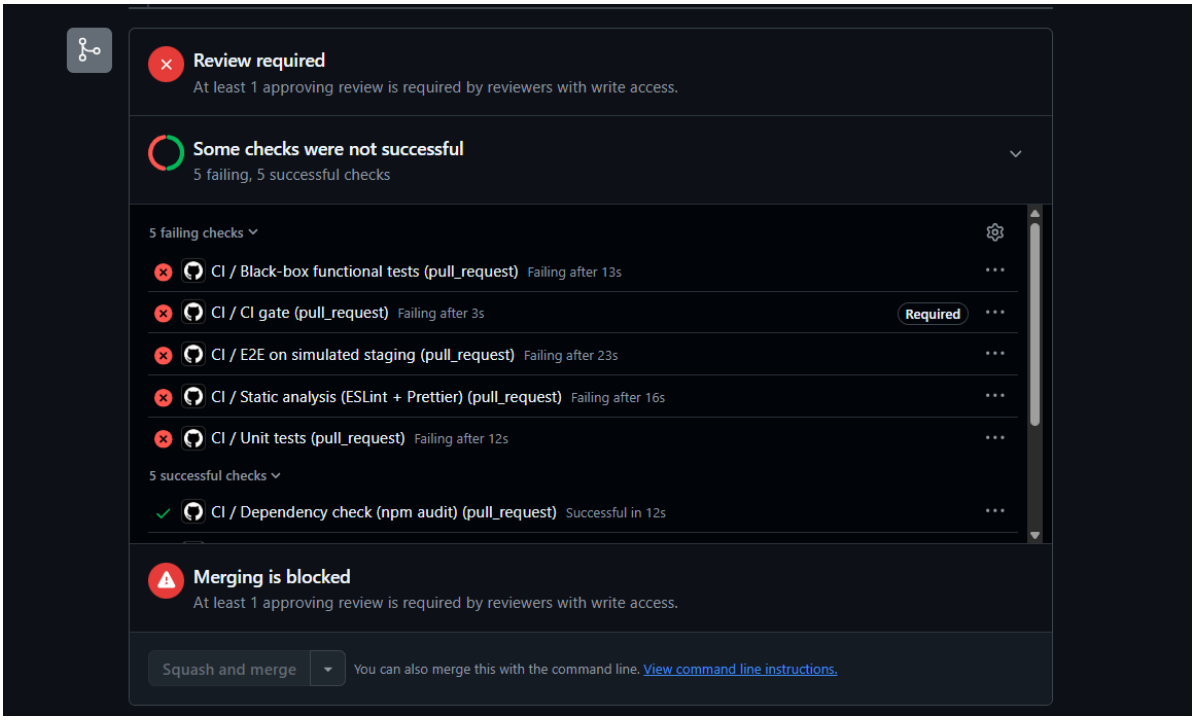
17.6.8 Evidence of the CI gate blocking a failing change:

CI block for M2: https://github.com/WaldoBlom/SEN381_Bugbusters/pull/47

This PR was opened with a deliberately failing test to show that the required status check blocks merging. It will be closed without merging, confirming that no failing code could enter the protected branch.

CI block for M3: https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/119

To demonstrate the CI gate, we opened Pull Request #119. This change is designed to fail the unit, functional, lint and E2E jobs. Because the CI gate requires every job to succeed, a single failure fails the gate and blocks the merge, so no incorrect change can enter a protected branch. The PR will be closed without merging, so the regression never reaches testing.



18. Architecture, Technology and Initial Design Baseline

18.1 Baseline Identification

- Baseline: CivicConnect PED v2.0
- Milestone: M2
- Status: Approved for controlled construction

18.2 Baseline synthesis and operational contract

The CivicConnect PED v2.0 baseline establishes the engineering guardrails that will guide controlled implementation during Milestone 3. The baseline defines the architecture, persistence technology, application technology stack, initial design responsibilities, integration mechanism, verification

approach and current deployment direction while deliberately retaining provider-specific operational decisions until sufficient implementation and staging evidence exists.

- **Architecture baseline**: CivicConnect uses a Modular Monolith deployed as a single application. Internally, responsibilities are separated through clear presentation, controller/request-handling, service/business-logic and persistence boundaries.
- **Data and persistence baseline**: PostgreSQL is the selected relational persistence technology. Transaction handling, integrity constraints and concurrency protection will preserve request and workflow consistency. The final managed PostgreSQL provider remains deferred.
- **Technology baseline**: CivicConnect uses Node.js v24, Express.js, EJS and Tailwind CSS, with node-postgres for database access. Node.js v24 is pinned through .nvmrc to support runtime consistency between development and CI.
- **Initial design baseline**: Core request workflow and persistence remain separated from secondary external side-effects. Notification activity must not cause an otherwise valid request transaction to fail.
- **Integration baseline**: Nodemailer using SMTP is the selected application-level email integration mechanism. Ethereal will be used for development/testing. The final production SMTP provider/configuration remains deferred because it must be verified against the networking restrictions of the selected production host.
- **Verification and CI baseline**: GitHub Actions, Jest and ESLint provide the initial automated quality gate. The staging branch requires peer review and a successful CI workflow before changes can progress.
- **Deployment baseline**: The current environment progression is Development → Test/CI → Local Staging → Production. CivicConnect will initially be validated as a single application instance in a dedicated local staging environment before moving to a managed PaaS production environment.
- **Production infrastructure baseline**: Render and Railway remain candidate application-hosting providers, while Neon and Supabase remain candidate managed PostgreSQL providers. Final selections are intentionally deferred until M3 evidence confirms compatibility, resource usage, cost, availability, backup/recovery and operational suitability.
- **Operational baseline**: CivicConnect targets proportionate, low-cost and best-effort availability for its initial low-to-medium workload. The system does not currently require enterprise high-availability architecture or horizontal scaling.

18.3 Open / Deferred Decisions

The following deployment and operational decisions remain deliberately deferred until sufficient M3 implementation and staging evidence is available:

- **Production application hosting provider:** Render and Railway remain candidate managed PaaS providers. The final production host will only be selected after validating Node.js resource usage, PostgreSQL connectivity, environment-variable and secret handling, availability behaviour, operational cost and compatibility with the selected production email-delivery approach.
- **Managed PostgreSQL provider:** PostgreSQL has already been selected as the database technology, but the final managed provider remains deferred. Neon and Supabase remain candidates and must be evaluated against connection pooling, storage limits, availability, backup/recovery capability, cost and the project's current recovery targets before final selection.
- **Attachment / image storage mechanism:** FR-106 requires durable attachment storage. Local application-instance storage is not considered suitable for production because files may be lost during restart or redeployment. The final durable storage mechanism or provider remains

deferred until representative image-upload testing confirms storage requirements, file-size limits, performance impact, access control, backup/retention and cost.

- **Production email-delivery configuration/provider:** Nodemailer using SMTP remains the selected application-level integration mechanism and Ethereal remains suitable for development/testing. The final production SMTP provider/configuration remains deferred because it must be compatible with the selected production hosting environment and its outbound-network restrictions.
- **Production environment variables and secrets configuration:** Environment-specific configuration and secrets will remain external to source control. Final production values and provider-specific secret configuration will only be established once the production hosting, database and supporting services have been selected.
- **Automated production deployment:** GitHub Actions currently provides CI and verification, while production Continuous Deployment remains deferred. The team will first validate the integrated application in the dedicated local staging environment and establish a successful manual production deployment before automating deployment to the live environment.
- **Production monitoring and observability:** The current /health endpoint provides initial health verification, but the final logging, monitoring, uptime and failure-alerting approach remains deferred until the production hosting platform and its available operational tooling are selected.
- **Rollback and recovery validation:** Recovery targets and an initial recovery procedure have been identified, but the final production rollback and database-recovery process remains deferred until the selected hosting and database platforms can be tested through restore, redeployment and smoke-test evidence.
- **Horizontal scaling and external session storage:** The current deployment baseline remains a single application instance. Horizontal scaling and external session storage are deferred unless M3 workload or resource evidence demonstrates that the initial single-instance deployment is insufficient.

For detailed evidence requirements and expected finalisation points, refer to Section 10.5 – Deferred Deployment Decisions and Evidence Required.

19. Change Management Standard & Controlled Changes

This section defines how CivicConnect controls change after a baseline has been approved. Once a baseline is signed off, changes are not silently absorbed into the PED, the codebase, the database schema, or any other controlled artefact. Every change follows the same controlled path:

Change Request → Impact Analysis → Decision → Authorise → Implement → Verify → Update Baseline

19.1 Change register

Change ID	Date Raised	Title	Short description of change	Decision	Authorized By	Baseline Version	Detailed section
CR-001	10 October 2026	Operator Stakeholder and External Requester Support	Add operator role and external requester capture to support phone, email, and walk-in requests.	Approved	The whole team / Bugbusters	PED v3.0	17.2.1
CR-002	11 October 2026	Requester Contact Information Access Approval	Introduce manager-controlled approval, rejection, and revocation of staff access to requester contact details, with assignment-based restrictions and audit logging	Approved	The whole team / Bugbusters	PED v3.0	17.2.2

19.2 Detailed change management per summary

This section provides the detailed controlled change record for each change raised against the baselined Project Engineering Document. Where the summary register above provides a high-level index of every change, this section expands each one into its full engineering record so that the reason, impact, decision, authorisation, implementation, and verification of every change remain traceable end-to-end.

This matters because a change is only complete when its authorised intent, affected artefacts, implementation evidence, and resulting baseline can be connected, not simply because code was merged or a document was edited. Each sub-section below follows the same structure and corresponds to one change request: it begins with the change identifier and title, then records the requested change and business reason, the impact analysis across the affected engineering dimensions, the formal decision and any conditions attached to it, the verification evidence required to prove the change works, and the baseline update that incorporates the change.

This consistent structure ensures that every change, whether raised early in the project or late, is documented to the same standard.

19.2.1 Operator Stakeholder and External Requester Support (CR-001)

Requested change

Add a new Operator stakeholder role and external requester support to CivicConnect. Operators will be able to log service requests on behalf of external requesters who contact the organisation through traditional communication channels (telephone, email, WhatsApp, in-person, or paper) and who do not have a CivicConnect account.

Reason and expected value

The M2 baseline assumed that all requesters would self-register and use the portal. During M3 planning, we identified that this assumption excludes community members who continue to use traditional communication channels. Without this change:

- Historical and ongoing requests received through phone, email, WhatsApp, or in-person cannot be captured in CivicConnect.

- The organisation would still need to manage some requests outside the platform, recreating the original fragmentation problem described in the business need.

- Management reporting would be incomplete because it would only reflect portal-submitted requests.

Business intent: Enable CivicConnect to serve as the single source of truth for **all** service requests, regardless of submission channel, while preserving accountability by recording which operator captured each request and how it was received.

Affected capabilities

- Request Management (submission path)

- Authentication & User Management (new role)

- Audit & Request History (operator attribution)

Clarification and resolved scope

The following points were clarified during the change request review and are now considered in scope for CR-001:

- The Operator is an internal CivicConnect user role (OPERATOR) created to capture requests on behalf of community members who cannot or do not use the self-service portal.
- External requesters are not system users. They do not register, log in, or receive accounts. Their details are stored in a separate external_requests table and linked to service requests via external_requester_id.
- The operator-assisted submission path must capture the same core request information as self-service (category, title, description, priority, location, attachments where applicable) and must record the operator who captured the request (created_by).
- Operators can view and search requests they have logged (FR-404). They do not have access to requests logged by other operators or by self-service requesters unless explicitly authorised.
- An operator who logged a request is intended to be able to edit that request while it remains in SUBMITTED status, provided the request has not yet been assigned or progressed. (Note: This capability is currently affected by defect #122; it is part of the intended scope but not yet functioning correctly.)
- Management reporting must be able to distinguish between portal-submitted requests and operator-captured requests (channel visibility).
- The existing service_requests table is extended with external_requester_id and created_by. A check constraint ensures that every request has exactly one requester source: either an internal requester_id or an external_requester_id.
- The change does not include: external requester self-service, automated ingestion from email/WhatsApp/telephony, or a public API for external channels. Operators manually capture requests using the internal interface.
- All external requester contact information remains subject to the existing contact-access approval workflow (FR-305, CR-002).

Impact analysis

Area	Impact
Data model	Adds external_requests table (external requester identity and contact details). Adds OPERATOR to users.user_type enum. Modifies service_requests with external_requester_id (FK) and created_by (FK to operator). Adds check constraint service_requests_exactly_one_requester.
Authentication & Authorisation	Introduces OPERATOR role. RBAC middleware (requireRole) must permit operators to access the operator submission form and their own logged-request list, while denying access to staff/management-only functions.
Request Management	New submission path: operatorSubmitRequest. Validates external requester details and request data (FR-402). Creates the request in SUBMITTED state, linked to the correct category and external requester. Operators can list their logged requests (FR-404).
Audit & Traceability	created_by records the operator. Audit events should capture operator creation actions. request_status_history and request_assignment_history continue to apply. RTM adds Operator requirements (FR-401–FR-404).

Area	Impact
Reporting	Management dashboards and reports must be able to filter or distinguish requests by submission channel (self-service vs operator-captured).
UI / Presentation	Adjustment to EJS views: operator submission form (with external requester fields), operator logged-requests list.
Security & Privacy	External requester contact details are personal information. Access must remain restricted by FR-305 and CR-002. Operators capture data but do not automatically gain unrestricted access to view contact details for all requests. Input validation is required to prevent injection or malformed data.
Validation & Error Handling	validateExternalRequest schema enforces at least one contact method (email or phone), valid email format, and required request fields. Database constraint prevents requests without a requester source.
Testing	Unit and integration tests cover operator submission, external requester creation, check constraint enforcement, and access control for operator-logged requests. Current evidence: submission.test.js, PR #87. Defect #122 (operator cannot edit) remains open and is tracked separately.
Deployment & Infrastructure	No new infrastructure required. Uses existing Node.js/Express, PostgreSQL, and object storage. No impact on free-tier limits beyond normal request volume.
Risks	Data privacy (POPIA) for external requester details. Duplicate or inaccurate requests if operators mistype.

Baseline updated

Once all verification activities above have passed, this change is incorporated into PED v3.0. The following artefacts are updated as part of the baseline:

- Section 1.2 - Stakeholder Analysis (adds 1.2.7 Operator)

- Section 2.1.1 - Scope Baseline (adds operator-assisted capture)

- Section 3.1 - Functional Requirements (updates FR-101; adds FR-401–FR-404)

- Section 6.1 - Core Data Entities (adds `OPERATOR` enum, `external_requesters`, modifies `service_requests`)

- Section 6.2 - Data Model / ERD (adds structural notes)

- Section 11 - Traceability (adds Operator RTM)

- Section 11.5 - Decision Log (adds DEC-016)

- Section 12.1 - Risk Register (adds RSK-016, RSK-017, RSK-018)

A CR-001 Impact Summary table appears at the top of the affected sections for reader convenience.

19.2.2 Requester Contact Information Access Approval (CR-002)

Requested change
Introduce a management approval process that allows or denies staff access to requester contact details. The assigned staff member can request access for a specific service request, and the responsible department manager can approve or reject it. Managers can also authorise access directly or revoke previously granted access.

Reason and expected value
Staff may need requester contact details to clarify information or arrange service delivery. However, being able to work on a service request should not automatically give staff access to personal information.

This change provides controlled access when needed, protects requester privacy, and records who requested access and who made the decision.

Business intent
Give department managers responsibility for deciding when requester contact details may be shared with staff. This supports service delivery while maintaining accountability for access to personal information.

Approval applies to the assigned staff member working on a particular service request.

How it affected the system
The change affected the database, access rules, staff and manager interfaces, and audit logging.

The new request_contact_access_requests table stores the service request, requesting staff member, department, reason, request date, review decision, reviewer, review date, and review note. It supports the statuses PENDING, APPROVED, REJECTED, and CANCELLED.

The core **Service Request** entity now works with two related aspects of contact access:

- **Current permission:** contact_information_approved, contact_approved_by, and contact_approved_at represent the current approval and its authorisation details.
- **Access request records:** the new table links individual access requests and review decisions to the service request, staff member, department, and reviewing manager.

This introduces a one-to-many relationship between a service request and its contact-access request records. Contact-access status is separate from the service request's operational status.

Staff views restrict requester names, email addresses, and phone numbers until access is approved for the current assignee. Managers have an inbox for reviewing access requests. Reassigning a service request clears the current approval so permission does not transfer automatically to another staff member. Contact-access actions also generate audit events.

Changes to the project scope
The project scope expanded to include a contact-access approval workflow alongside the existing service-request workflow. This required:

- A new database entity and relationships.
- Staff access-request functionality and status feedback.
- Manager approval, rejection, direct authorisation, and revocation controls.
- Additional checks for department ownership, staff assignment, and active users.
- Audit logging and tests covering contact visibility and access decisions.

The change adds a separate permission decision to handling a service request.

Impact analysis

Area	Impact
Data model	Adds contact-access request records linked to service requests, users, and departments.
Staff workflow	Staff must obtain approval before viewing requester contact details.

Manager workflow	Managers take on an additional review responsibility.
Security	Contact visibility depends on approval, current assignment, and department access checks.
Maintenance and testing	Requires verification of approval, rejection, revocation, reassignment, and restricted visibility.

Possible risks

- **Service delays:** staff may need to wait for manager approval before contacting a requester.
- **Incorrect approval:** a manager could grant access when it is unnecessary.
- **Information exposure:** future changes to queries or interfaces could accidentally bypass contact restrictions.
- **Outdated decisions:** reassignment can leave historical access requests that no longer apply. The implementation resets approval and prevents review when the requesting staff member is no longer the eligible assignee.
- **Denied-request handling:** a rejected request cannot currently be resubmitted by the same staff member for that service request. A manager must directly authorise access if circumstances change.

AI Usage

Our AI usage is tracked as a live document on our github repository, it can be viewed here:

https://github.com/Waldo-Blom/SEN381_Bugbusters/blob/main/docs/registers/ai-usage-register.md

Baseline Sign off

Appendix D — Baseline Sign-Off Template

Project	CivicConnect
Baseline Type	PED Milestone 1
Version	Version1
Date	9 September 2026
Scope reviewed	(YES) NO
Requirements/traceability checked	(YES) NO
Risk review completed	(YES) NO
Repository/governance controls checked	(YES) NO
Outcome	(ACCEPTED) CONDITIONALLY ACCEPTED / REVISION REQUIRED